



UD1 INTERFACES DE USUARIO



Bienvenidos al módulo de desarrollo de Interfaces del ciclo de grado superior de Desarrollo de Aplicaciones Multiplataforma.

En la primera unidad de este módulo daremos un paseo por la historia de las interfaces para conocer de primera mano: qué son, cuándo surgieron, qué tipos hay y qué elementos la forman, para así dar nuestros primeros pasos en el diseño de interfaces.

¿Te apuntas?

1. INTRODUCCIÓN

Uno de los campos de la informática que se encuentra en constante evolución y experimentación es el del diseño y desarrollo de interfaces de usuario. Y esto no sólo es debido a la evolución de herramientas de desarrollo o aparición de nuevos estándares, sino también el desarrollo de interfaces hardware que dan la vuelta a la forma de interacción habitual con las máquinas.

En esta primera unidad haremos una introducción a las interfaces de usuario y en su historia y los elementos que habitualmente se utilizan dentro de una interfaz para conseguir una buena interacción hombre-máquina.

También haremos un repaso a algunas de las herramientas que podemos emplear para desarrollar los diferentes tipos de interfaz, tanto de software libre o privativo. Actualmente hay multitud de ejemplos de aplicaciones informáticas con las interfaces de usuario correspondientes, que ayudan en la vida cotidiana de los seres humanos. Naturalmente, ante un ordenador, se utilizarán muchas aplicaciones, pero ante un cajero automático en el que se quiere sacar dinero o hacer cualquier otra operación, habrá que interactuar con otro tipo de interfaz de usuario (que será interactiva, en este caso).

Hay, sin embargo, muchos más ejemplos: hoy en día ver cine y series, con plataformas de streaming, representa tener que utilizar una serie de menús y opciones. Este es otro tipo de interacción entre un hardware electrónico y las personas, mediante otro tipo de interfaz de usuario. También lo será un reproductor de MP3, que llevará una pequeña interfaz muy simple, pero necesaria para su funcionamiento. Y no es necesario que hablemos de las interfaces que llevan todos los aparatos dedicados al ocio, como las consolas de videojuegos. Y se podría hablar también de los teléfonos móviles.

Las interfaces de usuario están en continua evolución y se han de conocer una serie de herramientas que permitan su desarrollo.

2. ¿QUÉ SON LAS INTERFACES DE USUARIO?



Reflexiona

¿Te has preguntado alguna vez cómo conseguimos que nos entiendan los ordenadores? A estas alturas todos sabemos que ellos trabajan con 0s y 1s y nosotros con palabras. ¿Cómo conseguimos que sean unas herramientas tan útiles para nosotros?

El mundo de la informática ha evolucionado a la velocidad del rayo en las últimas décadas, y con ello, nuestra forma de interactuar con los ordenadores.

Inicialmente, los ordenadores no existían para el gran público, tan sólo las grandes corporaciones o investigadores tenían acceso a ellos. Además, debían programar las instrucciones de las operaciones que querían hiciese el ordenador sin obtener una respuesta inmediata. ¿Quién puede imaginarlo ahora que obtenemos todo a golpe de click o de pulsación?

Desde los primeros ordenadores hasta ahora, hemos necesitado una forma de comunicarnos con ellos, y precisamente, esa es la **función de las interfaces: permitir la interacción entre usuarios y ordenadores.**

Así, su desarrollo ha ido ligado a la evolución de los ordenadores personales y los [sistemas operativos](#) . Desde Windows, hasta Linux, OS o Android para los móviles, tienen su propia interfaz característica que les define y que nos hace, muchas veces, decantarnos por uno y otro sistema, en función de lo habituado que estemos a ellos. Así, para entender qué son las interfaces de usuario comenzaremos recordando el funcionamiento de un sistema informático. ¿Cómo consigue un usuario manejar como a él le interesa un ordenador? El conjunto de piezas que componen el hardware de un ordenador deberá llegar a funcionar según los deseos del usuario. Pero la relación entre uno y otro tendrá algunos actores intermedios, como se puede ver en la figura 1. Fig. 1.



Máquina

En **primer lugar** se encuentra el **sistema operativo**, encargado de la gestión y la coordinación de las tareas que llevan a cabo un intercambio de información entre los diferentes recursos. Se compondrá de un sistema de entrada/salida, de la gestión de procesos, de la gestión de la memoria principal o del sistema de archivos, entre otros. A la vez actuará de interfaz entre el hardware y las aplicaciones utilizadas.

En **segundo** lugar tendremos las **aplicaciones**. Éstas se encuentran por encima del sistema operativo (el cuál necesitarán, adaptándose a él) y por debajo de los usuarios. Las aplicaciones tienen unas funcionalidades concretas y ayudan a los usuarios a conseguir sus objetivos.

¿Dónde se encuentran las interfaces? *Las interfaces forman parte de las aplicaciones.* Son la parte de las aplicaciones con la que se relacionarán los usuarios.

En la figura 2 se puede ver un ejemplo de interfaz actual de un microondas presentado en el CES 2010, junto con una lavadora. Sus interfaces están basadas en Android, sistema operativo para dispositivos móviles basado en una versión modificada de Linux.



Hay muchos tipos de aplicaciones informáticas. Se pueden agrupar según la finalidad, el entorno de uso, si son de uso genérico o si son hechas a medida, pero también se pueden agrupar según sean aplicaciones de gestión empresarial, para el ocio, ofimáticas, de gestión del hardware, aplicaciones en un entorno cliente-servidor, en un entorno web...

2.1. Concepto de interfaz de usuario

Hay muchos tipos de interfaces de usuario en muchos ámbitos diferentes. Por esta razón es difícil encontrar una definición única que deje claro el concepto.

Una aplicación informática tendrá varias interfaces de usuario.

Una **interfaz de usuario** es un conjunto de elementos (que pueden pertenecer al software o al hardware) que ofrecen una información al usuario, y permiten, además la interacción (física o lógica) entre el usuario y el ordenador, por medio de un dispositivo periférico o un enlace de comunicaciones.

Dentro de las interfaces de usuario diseñadas para aplicaciones informáticas, nosotros nos vamos a fijar en las interfaces de usuario gráficas o GUI (graphical user interface). Las GUI permitirán al usuario interactuar con el sistema informático de más maneras que la que históricamente ha sido la única alternativa hasta hace unos años: teclear instrucciones complejas y difíciles de entender para el usuario no experto.

Las **interfaces gráficas de usuario (GUI)** son aquellas que utilizan elementos gráficos, como pueden ser menús, ventanas o diálogos, además del uso de otros recursos del sistema informático (periféricos como el teclado, el ratón o el sonido) para permitir al usuario interactuar con el ordenador de forma muy sencilla e intuitiva.

Una de las novedades más importantes de las interfaces gráficas de usuario, aparecida a mediados de los ochenta, fue la aparición del ratón como periférico.

Utilizarlo les permite a los usuarios poder precisar la ejecución de las instrucciones, para poder interactuar con las interfaces sin necesidad de teclear una sola instrucción.

Actualmente podemos encontrar un uso generalizado de las interfaces gráficas de usuario. La gran mayoría de los sistemas operativos tienen este tipo de interfaz, de tal manera que integrar una aplicación en el sistema operativo sea muy sencillo, y sólo haya que adaptar algunos parámetros. Incluso hay dispositivos que han desarrollado su sistema operativo propio, como los dispositivos móviles, con sus interfaces propias. En la figura 1.3 se puede ver un ejemplo con el iPad, de Apple.



Fig. 3. Interfaz gráfica iPad

Algunos ejemplos de interfaces gráficas de usuario los podemos encontrar en los sistemas siguientes:

- **Windows:** La interfaz de usuario de Windows ha ido evolucionando juntamente con la evolución del sistema operativo. No tiene nada que ver la interfaz del antiguo Windows 3.1 con la de Windows 95, Windows XP, el Windows Vista, Windows 7 o Windows 10. Todas tienen algunos denominadores comunes, como ofrecer un menú de inicio, una barra de tareas, un escritorio y algunas características como la ejecución de varias ventanas de forma simultánea o la configuración de muchas de sus funcionalidades.
- **MAC:** Los sistemas informáticos de los Mac llevan un sistema operativo macOS. El nombre de la interfaz gráfica de usuario que incorpora su sistema operativo se conoce como Aqua (desde el año 2000). Antes, con los sistemas operativos Mac OS 8 y 9 la interfaz gráfica de usuario se había llamado Platinum. Se trata de una interfaz que intenta ofrecer unos colores y unas texturas bastante atractivas para ayudar al éxito del sistema operativo. Aporta un Dock, una barra de accesos directos a las aplicaciones y en las carpetas de documentos, que facilita la navegación por el sistema. Incorpora también un menú siempre en la misma ubicación, que irá cambiando en función de la aplicación que esté activa.
- **Linux:** El sistema UNIX/Linux lleva una interfaz primaria o básica tipo texto y era el único sistema para poder interactuar hasta no hace mucho tiempo. Actualmente hay varias interfaces gráficas, como:
 - **X-Window:** Incorpora un modelo cliente-servidor que permite el uso directo por parte de las aplicaciones de los periféricos de entrada/salida.
 - **KDE:** Entorno de escritorio para varios sistemas operativos. Permite la ejecución automática de nuevos dispositivos de datos, incorpora un inicio rápido.
 - **GNOME:** Entorno de escritorio dentro del proyecto GNU. Nació como alternativa a KDE. Mejora la usabilidad y aporta elementos útiles para discapacitados.
 - **Open Look:** Muestra y ejecuta varias aplicaciones de manera simultánea en diferentes ventanas.

A continuación se muestran algunos ejemplos de interfaces para los tres sistemas operativos que se han explicado.

En la figura 4 se puede ver un ejemplo de interfaces en el sistema operativo Windows 3.11.





Fig. 6. Interfaz gráfica de un Mac

Cada interfaz tendrá su razón de ser y sus funcionalidades bien definidas, pero podemos establecer algunas **funciones principales** que pueden cumplir las interfaces gráficas de usuario. Podemos encontrar:

- **Configuración de las interfaces gráficas de usuario y del entorno de trabajo.**
- **Control de acceso a una aplicación informática.**
- **Sistemas de ayuda interactivos.**
- **Gestión y manipulación de directorios y archivos de un sistema operativo.**
- **Arranque y cierre de un sistema informático mediante un sistema operativo.**
- **Intercambio de datos entre diferentes aplicaciones.**
- **Comunicación entre sistemas informáticos.**
- **Ayuda al desarrollo de aplicaciones informáticas.**
- **Ayuda al diseño y desarrollo de interfaces gráficas de usuario.**
- **Gestión y manipulación de las funcionalidades que se puedan configurar en los sistemas informáticos.**

Todas estas funciones principales más habituales de las interfaces de usuario tienen cabida dentro del **objetivo principal** de una interfaz de usuario, que consiste en **comunicarse de manera sencilla y agradable con el usuario por medio de un dispositivo de entrada/salida**.

Pero no todas las interfaces han sido, ni son, sencillas y agradables de usar. Las interfaces de líneas de instrucciones (CLI, command line interface), son un tipo de interfaz más arcaico, pero con algunas funcionalidades mucho más prácticas para sus usuarios, normalmente usuarios más expertos.

¿Quién preferirá utilizar una interfaz de tipo CLI antes que una interfaz gráfica de usuario? Los usuarios con muchos años de experiencia, que tuvieron que aprender las instrucciones básicas para poder trabajar con los sistemas operativos y están más habituados a este tipo de interfaces, o, simplemente, las quieren usar, ya que una instrucción compleja será muy complicada de representar mediante los elementos que ofrecen las interfaces gráficas.

Otros dispositivos de hardware harán necesario el uso de estas interfaces. Por ejemplo, un router o un conmutador algo complejo ofrecerá unas posibilidades de programación que requerirán el trabajo mediante líneas de instrucciones.

Todos los sistemas operativos ofrecen la posibilidad de trabajar mediante un intérprete de instrucciones, por ejemplo el Shell de UNIX, o el símbolo del sistema de Windows, o PowerShell en servidores. Habrá que tener también en cuenta que no todas las funcionalidades se pueden representar mediante interfaces de usuario. Por ejemplo, las funcionalidades que requieren muchas instrucciones serán difícilmente programables mediante interfaces; en cambio, con una pequeña programación Batch en Windows o los Shell scripts en UNIX, se programarán con facilidad.

Uno de los puntos críticos será el diseño de las interfaces de usuario. Tendrá que cumplir todos los requisitos y ser sencilla de utilizar. Si en la fase de diseño no se consiguen estas premisas, difícilmente se obtendrá una interfaz adecuada para ser utilizada de manera agradable y sencilla para sus usuarios, sin que éstos necesiten horas de formación o un buen manual al lado mientras interactúan con la interfaz.

2.2. Evolución de las interfaces de usuario

Hablar actualmente de interfaces gráficas de usuario es una cuestión que parece que no tenga mucha razón de ser, ya que las interfaces son parte de nuestro día a día, como un elemento muy cotidiano de lo que no parece que sea necesario hablar. Pero no siempre ha sido así. No hace mucho tiempo no se podía hablar de interfaces gráficas, o, si se podía hablar, al menos no era del mismo tipo de interfaces de las que podemos disfrutar ahora.

A lo largo de los años las interfaces gráficas han ido de la mano con los sistemas operativos. A medida que éstos iban evolucionando, las interfaces de las aplicaciones que se ejecutaban sobre aquellos sistemas operativos habían evolucionado en la misma medida, o más, si eso era posible.

Los sistemas operativos gráficos acercaron la informática a muchos más usuarios de los que hasta ese momento habían usado los ordenadores. Ya no era necesario ser conocedor de las órdenes que había que introducir por la línea de instrucciones o tener conocimientos de BASIC. Por ejemplo, cuando apareció el Windows 3.1, el primer sistema operativo de gran consumo basado en ventanas, muchas aplicaciones se adaptaron a este nuevo tipo de interfaces y también se basaron en ventanas, y esto hizo mucho más fácil y accesible el uso de la informática al gran público.

Para poder comprender la evolución de las interfaces, es necesario también que nos fijemos en la evolución del hardware sobre el que ejecutamos los programas y en la evolución de las técnicas de ingeniería del software. A medida que el hardware evolucionaba en prestaciones y bajaba en coste había una lucha para conseguir el negocio de la informática de gran consumo a escala de software, es decir, en lo que respecta a sistemas operativos. Todo esto ha llevado a una lucha entre diferentes empresas para desarrollar los entornos más agradables y sencillos de utilizar, lo que ha provocado una continua evolución de las interfaces en los últimos años.

Empresas como Apple, Xerox, Be o Microsoft han sido las que más tiempo y dinero han dedicado a esta tarea durante las últimas cuatro décadas. Además, también han intercambiado algunas acusaciones y demandas entre ellas. A lo largo del tiempo se puede dividir la evolución de las interfaces de usuario en las **fases** siguientes:

Hasta 1970

CLI - Sólo existían Interfaces de usuario basadas en línea de instrucciones.

1970

Se considera el nacimiento de las interfaces de usuario

De 1980–1985

WIMP (Windows, icons, menus and pointer) - Evolución hacia las interfaces gráficas de usuario (GUI).

De 1996–2001

Interfaces gráficas de usuario no basadas en instrucciones

De 2002 – actualidad

Interfaces gráficas de usuario interactivas

Interfaces de usuario basadas en la línea de instrucciones

En los inicios de los sistemas informáticos no había ningún interfaz de usuario. Si se quería modificar la programación se accedía directamente al hardware para hacer las adecuaciones pertinentes. Posteriormente, con la programación por lotes tampoco había una interfaz que permitiera la interacción. El usuario tenía que indicar un conjunto de instrucciones al sistema que, una vez procesadas, ofrecería unos resultados.

A partir de los años sesenta, con los lenguajes de comandos, se llegó a la interfaz de línea de instrucciones. Esta interfaz permitía al usuario interactuar con el ordenador con una línea de texto, que serviría como texto que llevaría incluido una orden.

Hay que tener en cuenta que estamos hablando de una época en la que un único ordenador central daba servicio a muchos usuarios, que van enviando instrucciones a este ordenador central, que distribuye su tiempo entre las solicitudes.

En la figura 7 se puede ver un ejemplo de ese tipo de máquinas.



Fig. 7. Interfaz basada en línea de instrucciones

Nacimiento de las interfaces de usuario (1970)

En la década de los setenta se empieza a evolucionar hacia el uso de un ordenador por parte de un único usuario. Estos primeros ordenadores personales permitirán a usuarios almacenar información, editarla y compartirla de una manera sencilla. Las aplicaciones que se desarrollan siguen estos objetivos e intentan ofrecer unas interfaces de usuario adecuadas para conseguirlo. Se suele utilizar un diseño de interfaces de dos dimensiones con menús jerárquicos de pantalla completa. Se usan las teclas de función como una manera de interactuar con las aplicaciones y de completar los menús.

Asimismo la empresa Xerox desarrolló, después de muchos años de investigaciones en PARC (Palo Alto Research Center), el ordenador Xerox Star 1981. Dentro de estas investigaciones, en 1973 se desarrolló la primera interfaz gráfica de usuario. El objetivo era obtener un sistema informático lo suficientemente pequeño para poder ser transportable, para trabajar en una oficina, con un sistema operativo con interfaz gráfica y poder compartir información de manera sencilla.

En la figura 8 se puede ver un ejemplo de Xerox Star.

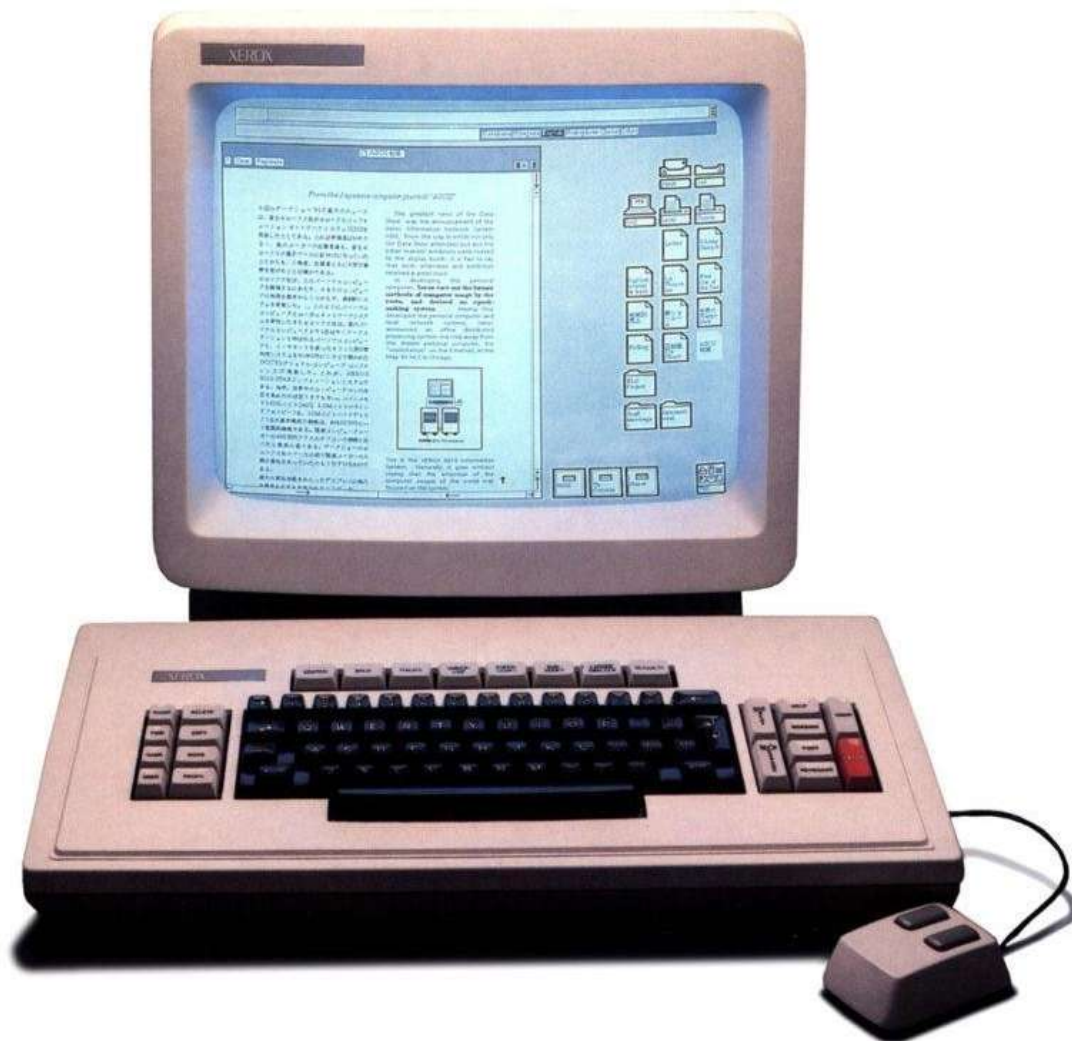


Fig. 8. Xerox Star

Evolución hacia las interfaces gráficas de usuario GUI (1980-1995)

En la década de los ochenta los ordenadores personales se hicieron más habituales para el gran público, aunque las interfaces gráficas de usuario

tardarían un poco en llegar. En esta fase se empiezan a utilizar las interfaces llamadas WIMP (windows, iconos, menús and pointer), Es decir, interfaces que disponen del movimiento de un puntero para la pantalla y que tendrán ventanas, iconos y menús. Se puede considerar que hasta el momento se trabajaba en dos dimensiones con las interfaces con líneas de instrucciones y, a partir de la aparición de las interfaces WIMP, se puede considerar que se utiliza una tercera dimensión que permite trabajar con más de una ventana (y aplicación) al mismo tiempo, superponiendo una encima de la otra.

Las GUI permiten a los usuarios trabajar directamente en aquellas funcionalidades que les interesan, y tienen que desplazarse hasta otras interfaces para localizar y poder hacer utilizar otras funcionalidades. Ofrecen una interacción con los usuarios que se basa en la manipulación directa, ofrece los elementos importantes para el usuario en cada interfaz y permite su manipulación.

Las primeras interfaces gráficas hemos visto que se empezaron a desarrollar a mediados de los setenta (Xerox), pero hasta mediados de los ochenta no las desarrollaron masivamente otras empresas, lo que provocó el uso más generalizado los ordenadores. En el siguiente listado podemos ver la evolución de las interfaces y algunos sistemas operativos que se fueron desarrollando:

- 1981: Xerox Star
- 1984: Apple Macintosh
- 1985: Commodore Amiga, Microsoft Windows 1.0
- 1990: Microsoft Windows 3.0
- 1992: IBM OS 2 2.0
- 1995: Microsoft Windows 95
- 1998: KDE/ Gnome
- 2001: Apple Mac OS X 10.0, Microsoft Windows XP

Interfaces gráficas de usuario no basadas en instrucciones (1996-2001)

A partir de 1995 hay un cambio muy importante en el uso de las interfaces de usuario. La aparición de sistemas operativos que interactúan con los usuarios de manera única y completa con GUI convirtió las interfaces en un elemento más de consumo y generalizó más aún el uso de los ordenadores personales dentro de las casas y los sistemas informáticos en las organizaciones.

Este gran cambio supuso un cambio en la manera de desarrollar aplicaciones para los nuevos sistemas operativos gráficos, y permitió también investigar y evolucionar mucho más en este campo.

Las interfaces gráficas de usuario continúan utilizando elementos de las interfaces del estilo WIMP, pero cada vez más evolucionados, con nuevos elementos y funcionalidades. Una de las novedades más importantes es la aparición de elementos multimedia que se pueden añadir a las interfaces, como sonidos (ya sean alertas que avisan al usuario de una mala interacción o un sonido de fondo que se puede escuchar durante el funcionamiento de la

interfaz).

Estos elementos se pueden considerar, utilizando el término con comillas, una "dimensión" más que añadir a las dos dimensiones que tenían las interfaces de estilo WIMP, aunque no se puede hablar propiamente de una tercera "dimensión", los elementos que se añaden de tipo multimedia, sonido o interpretación de voz o realidad virtual es hacia donde comenzaron a evolucionar las interfaces.

En la figura 9 se puede ver un ejemplo de sistema operativo con un entorno WIMP. El sistema operativo del ejemplo se llama Haiku, un proyecto de código abierto desarrollado en C++ que quiere recrear el sistema operativo BeOS.

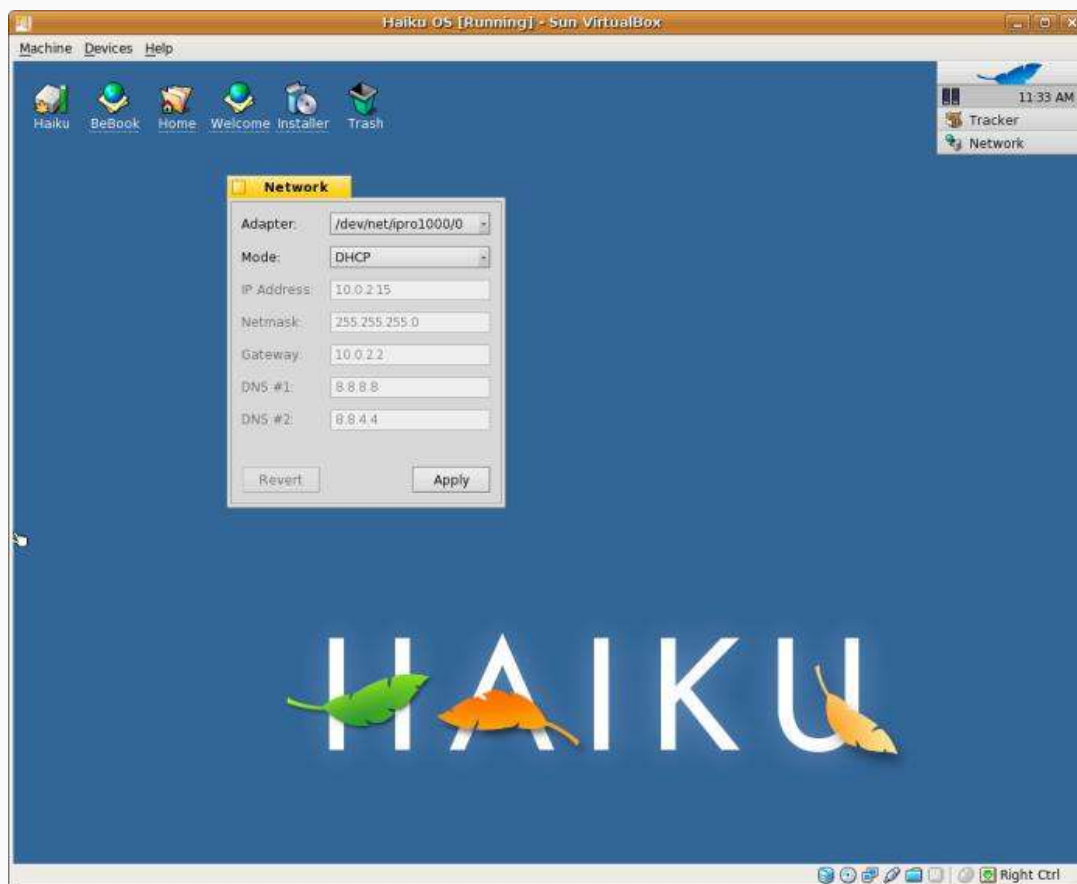


Fig. 9. Ejemplo de interfaz gráfica no basada en instrucciones

Interfaces gráficas de usuario interactivas (2002-ahora)

Las interfaces gráficas que nos encontramos actualmente (y la evolución hacia las que nos encontraremos en el futuro) van encaminadas a utilizar aún más dimensiones y otras evoluciones.

La primera nueva "dimensión" que se incorporó fue la de utilizar elementos multimedia. En algunos casos por necesidad y en otros para investigar hacia una evolución que haga aún más sencillas y útiles las interfaces. Un elemento multimedia puede ser la incorporación de imágenes en las interfaces, de sonidos o de animaciones. Por esta razón se podría considerar una tercera "dimensión" el concepto de tiempo (las animaciones utilizarán un tiempo estipulado para que se pueda completar) o incluso y todo una cuarta "dimensión" con el concepto de comunicación verbal. Hoy en día las personas con deficiencias visuales pueden utilizar una interfaz gráfica gracias a las indicaciones verbales que puede ofrecer

cada elemento de la GUI y dispositivos de entrada de datos específicos para ellos.

Pero tampoco se dejará de lado en el futuro el aprovechamiento de las nuevas posibilidades que puede ofrecer añadir una tercera dimensión real a las interfaces, gracias también a la evolución de los dispositivos de salida más habituales, las pantallas, que después de evolucionar a la tecnología de LED (también en las pantallas de ordenadores), llegarán más pronto que tarde a la tecnología 3D, algo muy aprovechable para las interfaces para explorar nuevas posibilidades.

2.3. Tipos de interfaces de usuario

Las interfaces gráficas se pueden clasificar en diferentes tipos en función de las características y funcionalidades que tengamos en cuenta. Si se habla de tipo de interfaces, podemos encontrar algunas que se utilizan en los diferentes sistemas operativos.

Por ejemplo, hay un tipo de interfaces basadas en WIMP. WIMP muestra un tipo de formato de interacción entre las personas o usuarios y las computadoras (y sus interfaces).

WIMP es un estilo de interacción que utiliza normalmente un **ratón** para controlar la ubicación de un cursor que se irá desplazando por las interfaces. Otro elemento importante del WIMP es la utilización de **ventanas** para presentar la información de manera organizada. También se usan **iconos y menús** para ofrecer las funcionalidades adecuadas a los usuarios.

A partir de este conjunto de elementos, las opciones de los menús y las que representan los iconos serán activadas a partir de los punteros que se moverán con el dispositivo físico (ratón), y se modificarán las informaciones para mostrar en las ventanas.

El estilo de interacción WIMP busca reducir el tiempo de aprendizaje que necesitará un usuario para utilizar de manera óptima una interfaz gráfica facilitando el recuerdo de las opciones que ofrece. A la vez será más sencilla de utilizar por parte de usuarios no expertos.

Si dividimos las interfaces en función de la forma de interaccionar con los usuarios, podremos encontrar los siguientes tipos de interfaces:

Las alfanuméricas

con las que habrá que trabajar con instrucciones, también conocidas como las interfaces de línea de instrucciones (CLI). Este tipo de interfaz será la única posibilidad de trabajar con estos sistemas, y hace servir toda la pantalla para este objetivo.

Las interfaces gráficas de usuario (GUI).

Este tipo de interfaces permiten una interacción más rápida, sencilla e intuitiva gracias al hecho de representar de manera gráfica los elementos de las interfaces.

Las interfaces desarrolladas para pantallas táctiles

Estas interfaces deben adaptarse a las necesidades de un dispositivo de entrada y de salida al mismo tiempo, que será una pantalla táctil, y a sus características específicas (utilizar iconos más grandes, al no poder trabajar con la misma precisión que un ratón, o no utilizar menús, por ejemplo).

Las interfaces desarrolladas para dispositivos móviles

Dispositivos como teléfonos móviles, tabletas u otras herramientas similares disponen de sistemas operativos y tipos de interfaces adecuadas a sus necesidades (pantallas de un tamaño limitado, pantallas táctiles...).

Si nos fijamos en cómo se ha desarrollado la interfaz, los elementos con los que se ha construido, podemos encontrar tres tipos:

De hardware

Se trata de interfaces que permiten la interacción entre el usuario y el sistema informático mediante elementos de hardware, dispositivos como pulsadores o lectores de códigos de barras u otros reguladores o instrumentos.

De software

Estas son las interfaces desarrolladas para ordenadores que basan su creación en desarrollo de software, mediante el cual se llevará a cabo toda la interacción con los usuarios.

De software-hardware

Son interfaces que compartirán los dos tipos explicados anteriormente. Por un lado interfaces de software con algún elemento o dispositivo de hardware necesario para recoger algún tipo de información específica (por ejemplo, un lector de código de barras).

Otra manera de clasificar las interfaces es según la arquitectura de las aplicaciones y su manera de trabajar:

Aplicaciones locales

Interfaces Winforms. Estas aplicaciones se desarrollan para trabajar en una única máquina un único usuario a la vez. Se habrá podido desarrollar en diferentes lenguajes de programación y se podrá haber vinculado a diferentes tipos de bases de datos, pero se encontrarán instaladas en una máquina que será el único lugar desde donde se podrá utilizar. Este tipo de aplicaciones, habitualmente, disponen de interfaces de tipo Winforms. Este nombre es el que se da a las [API](#)  incluidas en entornos integrados de desarrollo que facilitan la creación de interfaces basadas en ventanas y otros elementos de tipo WIMP.

Aplicaciones cliente-servidor

Este tipo de aplicaciones están basadas en el trabajo en más de una máquina, en la que una hará de servidor y el resto (una o muchas más) harán de clientes que acceden remotamente al servidor para acceder a los datos o a las funcionalidades. Hay que hacer una instalación en la máquina servidor y otra instalación en cada una de las máquinas cliente. También se podría dar el caso de trabajar con aplicaciones cliente-servidor en una única máquina. El tipo de interfaces que se utilizan habitualmente para este tipo de aplicaciones también serán las Winforms, aunque también se podrán utilizar interfaces de tipo Web.

Aplicaciones Web.

Son un tipo específico de las aplicaciones cliente/servidor. Este tipo de aplicaciones permite acceder a datos y funcionalidad por medio de las redes telemáticas, y el acceso más habitual se hace por medio de un navegador de Internet, que accede a un servidor, donde se localizarán los datos y la implementación de las funcionalidades. Las interfaces serán de tipo web, se ejecutarán las funcionalidades y la manipulación de datos en el servidor y se enviarán sólo las interfaces o datos para mostrarlos en los navegadores que acceden de forma remota. En los últimos tiempos se está mejorando este sistema para hacer que la transferencia de información sea cada vez más pequeña, e implicar entonces a la máquina cliente cada vez más.

Una última manera de clasificar las interfaces es a partir del software donde se desarrollarán:

Plataformas RichClient.

Son herramientas de desarrollo de software que contendrán todos los elementos necesarios para diseñar y desarrollar sin otra necesidad una interfaz gráfica de usuario. Además se podrán aprovechar de las bibliotecas y elementos existentes. Esta manera de trabajar facilita el desarrollo y la integración de los elementos y la aplicación desarrollada en su entorno. Se pueden desarrollar aplicaciones locales y cliente/servidor. Ejemplos de entornos RichClient son Visual Studio (de Microsoft) o, en código abierto, Eclipse o NetBeans.

Plataformas ThinClient.

Son herramientas de desarrollo de software que necesitan otras herramientas para poder diseñar y desarrollar interfaces gráficas de usuario y partes de software.

2.4. Elementos de las GUI. Propiedades y características

Estudiada la evolución de las interfaces gráficas de usuario a lo largo del tiempo, podemos observar como el denominador común ha sido llegar a ofrecer a los usuarios unas interfaces con una usabilidad óptima. Para conseguir esto se ha establecido que las interfaces deben cumplir una serie de características que, entre otras, serán:

Accesible e intuitiva

Una interfaz debe ser intuitiva en su uso, debe mostrar con claridad las funcionalidades que ofrece y facilitar llegar a las mismas de manera sencilla y clara.

Uso de metáforas

Para identificar las funcionalidades que representan o los objetivos que simularán, las interfaces deben utilizar metáforas que vinculen sencillamente icono o imagen con funcionalidad u objetivo.

Aprendizaje y uso fácil

Las interfaces deben ser fáciles de usar y también de aprender por parte de los usuarios principiantes.

Consistencia

Las interfaces deben seguir un mismo diseño y estructura entre ellas y también con otras interfaces de aplicaciones análogas. También deberán ser consistentes cuando se ejecuten en diferentes entornos.

Ofrecer el control de las interfaces

Una interfaz debe saber entregar su control al usuario que la utilizará.

Anticipación

Se deberán prever los posibles errores que pueda cometer un usuario o las necesidades que pueda demostrar y ofrecer soluciones antes de que aparezcan o controlándolos y ofreciendo soluciones.

Legibilidad

Deberán ser fácilmente interpretables y ofrecer una legibilidad adecuada a los usuarios.

Autonomía

Un usuario no debe necesitar más información o ayuda que la que una interfaz le ofrece o, la que puede encontrar a partir de las indicaciones que le indicará esta interfaz.

Reducir carga de memoria

Para poder utilizar una interfaz más de una vez no habrá de obligar a los usuarios a recordar la ubicación de las funcionalidades, sino que éstas deberán ser sencillas de encontrar. Esto hará que no sea indispensable memorizar muchas informaciones para aprender a utilizar una interfaz determinada.

Internacionalización de la interfaz

Debe permitir ser entendida y utilizable por usuarios de diferentes culturas e idiomas o bien haciéndola internacional con iconos fácilmente reconocibles o con la posibilidad de seleccionar el idioma con el que se querrá trabajar.

Valores iniciales

También conocidos como valores por defecto o estándar. Son los valores que en las interfaces que llevan un formulario incorporado aparecerán seleccionados inicialmente. Además, deben poder ser descartados de manera sencilla.

Ley de Fitts

Una interfaz debe conseguir optimizar la ley de Fitts. Esta ley, en ergonomía, moldeará el movimiento humano, haciendo una estimación del tiempo que puede necesitar un ser humano para mover un puntero desde una zona de la pantalla hasta otra teniendo en cuenta variables como los objetivos, la distancia hasta alcanzarlos y el tamaño que tendrán.

Las interfaces gráficas de usuario (GUI a partir de ahora) disponen de una serie de elementos propios, comunes a muchas de las GUI desarrolladas, que dispondrán de unas características y propiedades. Con estos elementos se facilitará el desarrollo de las GUI de las aplicaciones informáticas, lo que ofrece la posibilidad de cumplir la mayoría de las características definidas.

Para que el usuario pueda utilizar estos elementos, deberá utilizar alguno de los dispositivos de entrada/salida que se han ido desarrollando a lo largo de los últimos años. El teclado del ordenador ha sido el dispositivo indispensable, como lo es hoy en día el ratón. Pero hay otros como el touchpad (ratón táctil), la tableta gráfica, el trackball (bola), los joysticks, las pantallas táctiles o los micrófonos. Como dispositivos de salida, además de la pantalla, serán importantes los altavoces o los LED informativos.

Algunos de estos dispositivos serán indispensables para el funcionamiento correcto de algunos de los elementos que ofrecen las GUI. Cada interfaz gráfica

utilizará algunos elementos en función de su entorno de trabajo, y no todas tienen los mismos elementos.

Veremos que una interfaz gráfica está formada por un conjunto de ventanas, llamadas formularios, y que dentro de ellos podemos colocar diferentes elementos visuales, que se denominan controles o componentes con los que al interactuar damos órdenes o podemos recibir información:

- Ventanas
- Cuadros de diálogo
- Asistentes
- Menús
- Pestañas
- Barras de herramientas
- Iconos
- Entorno de trabajo
- Entornos de escritorio
- Controles
- Tipografía

Ventanas

Dentro de los diferentes elementos de las GUI, encontramos uno de los más importantes, que son las ventanas (windows). Las ventanas son, normalmente, bidimensionales y de forma rectangular, y se ubicarán en el escritorio del sistema operativo. Cuando tenemos abierta más de una ventana ubicada en un mismo escritorio, la que está activa es la que podremos ver, la que está más cerca de nosotros y el resto quedarán por debajo, escondidas. Hay que decir que en los sistemas operativos más modernos se empiezan a utilizar técnicas de transparencia para poder identificar algunos elementos de las ventanas no activas, ubicadas por debajo de la ventana activa.

Las ventanas nos ofrecen la posibilidad de trabajar al mismo tiempo con diferentes entornos, que podrán ser de la misma aplicación o de diferentes aplicaciones.

Por ejemplo, el hecho de trabajar con ventanas nos ofrece la posibilidad de tener dos documentos ofimáticos abiertos (una hoja de cálculo y un documento de texto) e ir alternando la información escrita en ambos documentos o ir copiando información de un documento a otro. En la figura 10 se puede encontrar un ejemplo de trabajo con ventanas en un entorno de trabajo con el sistema operativo Windows 10.





Fig. 10. Sistema operativo Windows 10

En un caso de utilización de una misma aplicación informática, el hecho de trabajar con ventanas nos permitirá tener más de una interfaz abierta al mismo tiempo, lo que nos permitirá comparar la ejecución de dos funcionalidades o tener dos interfaces diferentes abiertas a la vez, una para consultar datos y otra para modificar otros datos.

Las ventanas nos permitirán ofrecer información al usuario de manera organizada. Nos permitirán navegar por las funcionalidades de una aplicación visualizando la información de manera jerarquizada. Esto permite gestionar y manipular la información de manera bastante sencilla. Cada ventana puede ser manipulada como al usuario le interese, maximizándola o minimizándola, moviéndolas o cerrándolas, navegando en ella o haciéndolas más grandes... Las ventanas contendrán interfaces gráficas de usuario que incorporarán otros elementos propios de las GUI, como menús, pestañas, barras de herramientas, objetos gráficos o multimedia, que permitirán tanto la salida como la entrada de datos para que el usuario pueda llevar a cabo varios procesos.

A lo largo de los años las ventanas han ido evolucionando, y se ha llegado hoy día a lo se podría llamar un estándar de diseño de ventanas, con unos elementos básicos, los cuales utilizarán todas las ventanas, como son: el marco, la cabecera de la ventana, el espacio del contenido, la barra de desplazamiento y el pie de la ventana.

El marco ayuda a distinguir qué pertenece a la ventana y qué no. Además, sirve para redimensionar el tamaño de la ventana. Como prácticamente todos los elementos, se podrá configurar en la mayoría de los casos. La cabecera de la ventana es el lugar donde se encontrarán situados los iconos que identifican y permiten llegar a las funcionalidades principales de la aplicación que se ejecuta en esa ventana. El espacio de contenido variará en tamaño en función de las dimensiones de la ventana. Será el lugar donde se mostrará la información (imagen, contenidos multimedia, de texto...). La barra de desplazamiento o scroll aparecerá en el caso de que el espacio de contenido sea menor que la información para mostrar. De esta manera el usuario puede desplazarse por los contenidos no visibles inicialmente en la ventana. Finalmente el pie de la ventana se utiliza para ofrecer informaciones diversas relacionadas con la ventana o con la aplicación y los contenidos que se mostrarán.

Cuadros de diálogo.

Los cuadros de diálogo son un tipo de ventanas, también consideradas como ventanas secundarias. Los cuadros de diálogo aparecen por encima del resto de ventanas pidiendo una interacción concreta al usuario, que deberá contestar. Esta interacción podrá ser sólo informativa (un mensaje de aviso o de advertencia que habrá que confirmar que se ha leído para poder continuar) o un requisito concreto de algún dato o de alguna acción que habrá que contestar de

manera concreta para poder continuar utilizando la aplicación.

Un ejemplo de cuadro de diálogo lo encontramos al intentar finalizar una aplicación. Si ha habido cambios en el documento con el que se ha trabajado o con los datos con las que se ha abierto una aplicación, al intentar finalizar esta aplicación nos preguntará si la queremos finalizar sin guardar los datos o si lo queremos hacer. Este cuadro de diálogo no nos permitirá continuar trabajando con la aplicación hasta que no hayamos contestado su mensaje.

En la figura 11 se muestra un ejemplo de un cuadro de diálogo generado por la aplicación Microsoft Word.

Asistentes

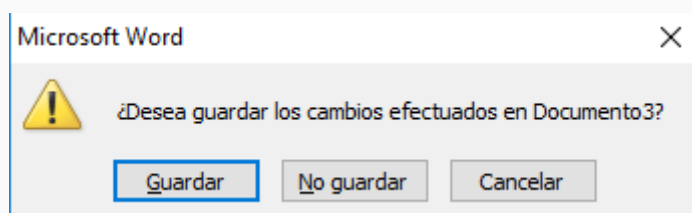


Fig. 11. Ejemplo de cuadro de diálogo

Los asistentes son ventanas que van apareciendo una tras otra hasta que se llega a un determinado objetivo. Como su nombre indica, asisten al usuario hasta completar una funcionalidad, preguntando paso a paso todas las informaciones necesarias hasta completar correctamente. De esta manera el usuario no se podrá olvidar de introducir una información o escogerá mal una opción.

Los asistentes son habituales en la instalación de software o como ayuda en la utilización de una funcionalidad muy concreta.

En la figura 12 se muestra un ejemplo de un asistente de instalación del software Mozilla Thunderbird.



Fig. 12. Ejemplo de asistente

Menús

Los menús son un elemento muy importante para las interfaces de usuario, ya que serán los encargados de mostrar todas las posibilidades de interactuar con la aplicación informática.

Los menús desplegarán todo el conjunto de funcionalidades que tendrá el usuario a su alcance. Cada opción de un menú dispondrá de un subconjunto de opciones, lo que convierte los menús en un árbol jerarquizado con un número de niveles que deberán haber decidido los desarrolladores. Cuanto más niveles tenga, más complicado será que un usuario llegue a una funcionalidad determinada.

En la figura 13 se muestra un ejemplo de un menú del software Mozilla Thunderbird.

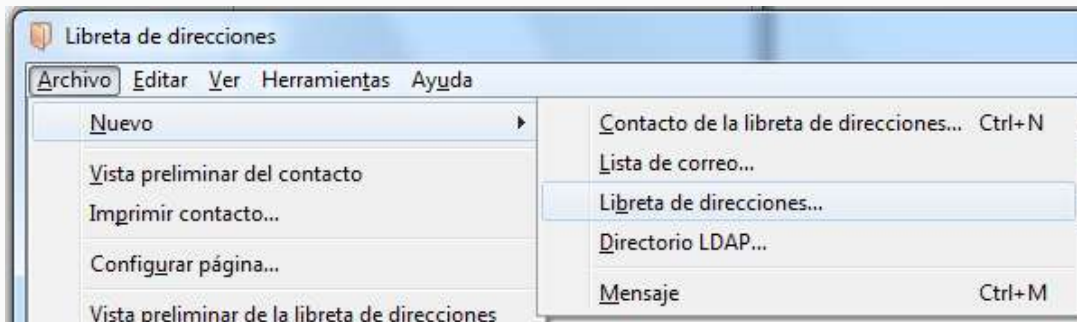


Fig. 13. Ejemplo de menú de Mozilla Thunderbird

Como se puede ver en la imagen, muchos menús indican las combinaciones de teclas que permiten llegar a la misma funcionalidad de manera más rápida. Un usuario habitual de una aplicación memorizará las funcionalidades más indispensables para él. A veces, en vez de la combinación de teclas puede haber otras informaciones gráficas que complementen la información en formato de texto que identifica una funcionalidad, o bien indicadores de cómo se encuentra una opción (si se puede ejecutar o no o si está activada o no) o la agrupación a la que pertenece.

Finalmente, hay que tener presente la evolución de los menús. En las últimas aplicaciones, como el paquete de ofimática Office, los menús han evolucionado hacia una muestra de funcionalidades mediante iconos, minimizando la presencia del texto. Esta evolución puede llevar a confusión a los usuarios acostumbrados a una determinada manera de trabajar. Se puede ver un ejemplo de menú de Office 2010 en la figura 14.



Fig. 14. Ejemplo de menú de Microsoft Office

Hay muchos tipos de menús, dependiendo de su entorno de ejecución o de la tipología de aplicación a la que pertenece la interfaz donde se encontrará el menú. Los menús más habituales que nos podemos encontrar son los menús contextuales, los menús de navegación y los menús jerárquicos:

- **Menús jerárquicos:** Menús representados en forma de árbol, con un número de niveles horizontal (opciones del menú) y un número determinado de niveles verticales (hasta donde podrá llegar el usuario navegando por el menú). Las opciones que se encuentran en un nivel tendrán correspondencia

entre ellas. Son utilizados de manera habitual en sistemas operativos y aplicaciones de toda índole.

- **Menús contextuales:** No se encontrarán visibles en la interfaz hasta que el usuario no provoque la activación. Son menús que se abrirán en una nueva ventana flotante, que variarán en función de la ubicación del ratón en el momento de llamarlos (normalmente haciendo clic con el botón derecho del ratón). De esta manera, será diferente el menú que aparecerá si se hace clic sobre el marco de la ventana o si se hace clic sobre el área de trabajo.
- **Menús de navegación (scrolls):** Se pueden considerar una evolución de los menús jerárquicos. Se trata de un tipo de menú que muestra sólo las opciones más utilizadas de los menús y oculta el resto, pero dejando visible un pequeño icono que, al ser seleccionado, muestra todo el resto de las opciones de menús. Esto nos da la sensación de poder navegar por el menú y sus ítems. Este tipo de menú se utiliza cuando son muchas las opciones que se quieren mostrar en una interfaz de usuario.

Hay otros tipos de menús. Uno que habrá que tener en cuenta por su importancia es el menú de inicio. Este menú lo desarrolló Microsoft para el sistema operativo Windows 95 y ha evolucionado hasta hoy día en toda la familia de sistemas operativos de Windows y en otros sistemas operativos que lo han adaptado, como, por ejemplo, GNU/Linux. Se trata de un menú jerárquico que ofrece el acceso a muchas de las funcionalidades y opciones de configuración del sistema operativo y llega a la mayoría de sus elementos y aplicaciones.

Pestañas

Las pestañas de propiedades son un elemento que permite mostrar un conjunto de datos o de funcionalidades relacionadas entre sí de manera agrupada. En una misma ventana se podrán mostrar los contenidos de tantas ventanas como se quiera, separándolas por pestañas, que deberán tener un título y contener informaciones relacionadas. Se puede ver un ejemplo en la figura 15.

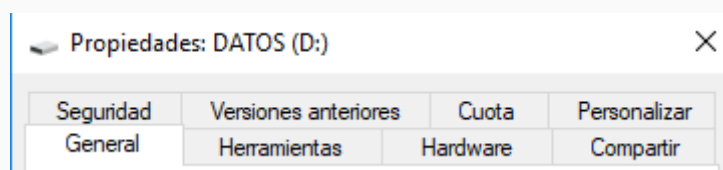


Fig. 15. Ejemplo de pestañas de propiedades

Barras de herramientas

Las barras de herramientas facilitan el acceso a algunas funcionalidades mediante un icono. Todas las funcionalidades ofrecidas a las barras de herramientas se podrán encontrar desarrolladas en los menús, pero no todas las opciones que ofrecen los menús se podrán encontrar en las barras de herramientas. Una vez seleccionadas aquellas funcionalidades que se utilizarán más habitualmente se identificará cada una con un símbolo que represente esa funcionalidad. Las combinaciones de teclas son otro método para poder acceder a estas opciones de los menús.

De esta manera se limitará a una sola acción (un único clic) el acceso a las principales funcionalidades, lo que las hace mucho más sencillas y accesibles. En la figura 16 se puede ver un ejemplo de una barra de herramientas.



Fig. 16. Ejemplo de barra de herramientas de Mozilla Thunderbird

Iconos

Los iconos son imágenes que representan funcionalidades o acciones que se podrán llevar a cabo haciendo un clic encima. Podrán identificar también archivos, carpetas, otras aplicaciones o dispositivos de un sistema informático. Es necesario que los iconos sean muy fácilmente identificables por parte de los usuarios, o, en su defecto, que estén muy bien informados de sus funcionalidades. En caso contrario el usuario deberá memorizar el significado, cosa que no cumpliría uno de los principios de las GUI, el de no obligar a memorizar información al usuario para usarlas correctamente.

Se trata de un elemento muy importante en las interfaces de usuario, ya que hará más rápido, atractivo y sencillo el uso de las aplicaciones más que si las informaciones que se ofrecen al usuario son en formato texto. Además, en el caso de aplicaciones internacionales, los iconos pueden ser los mismos en diferentes culturas y, en cambio, las traducciones pueden ser mal entendidas algunas veces.

Entorno de trabajo

El entorno de trabajo debe considerarse como un elemento más de una interfaz gráfica de usuario. Es un elemento más de las interfaces, como lo son los menús, las barras de herramientas o muchos otros.

Se trata de la ubicación principal de las informaciones con las que se interactúa con el usuario, donde se mostrará el texto, las imágenes o los datos que el usuario deberá pedir mediante los menús.

En la figura 17 se puede ver un ejemplo de un entorno de trabajo con el sistema operativo Windows 10.



Figura 17. Ejemplo de un entorno de trabajo de Windows 10

Tipografías

El texto que se utilizará en la comunicación con los usuarios dispondrá de varias opciones tipográficas para ser mostrado. La elección cuidadosa de estas tipografías también ofrecerá un mensaje específico al usuario y es un elemento más a tener en cuenta en el diseño y desarrollo de las interfaces.

Se conoce como tipografía digital las fuentes desarrolladas exclusivamente para pantallas, para usarlos en interfaces gráficas de aplicaciones informáticas. De este modo, Microsoft tiene un sistema propio de tipo desarrollado, tanto para su sistema operativo como para las aplicaciones que funcionarán encima. Apple también ha dedicado un esfuerzo importante a este tema, y desarrolló las primeras fuentes exclusivas para pantallas de sistemas informáticos.

Otras características a tener en cuenta de las tipografías digitales serán la intensidad y los colores utilizados, así como los distintos tamaños de letra. Una elección correcta de todas estas variables puede influir mucho en la apreciación por parte del usuario de la interfaz desarrollada.

Controles

Hay muchos tipos de componentes que se pueden utilizar en las interfaces gráficas de usuario. Un tipo de componentes son los controles. Estos proporcionan funciones en la interfaz de usuario que permitirán muchas más posibilidades en las interacciones entre interfaz y usuario. Ejemplos:

- **Botones en forma de radio (radio buttons):** Son elementos que se utilizan en formularios o en menús para dar a seleccionar al usuario una opción entre una lista. Estos botones en forma de radio son botones redondos que podrán ser elegidos por medio de acciones de usuario.
- **Botones de confirmación o cuadro de verificación (check box):** Botones de forma cuadrada muy parecidos a los de radio, también para poder seleccionar opciones de una lista de opciones. La diferencia con los botones en forma de radio será que en los botones de confirmación se podrá

seleccionar más de una opción entre las mostradas, y en cambio en los botones en forma de radio sólo se puede seleccionar una.

- **Botones de relieve:** Este tipo de botones imita un botón de un dispositivo físico que simula un volumen, y da así la posibilidad de tener varios estados (activado o no, seleccionado o no), además de poder incluir texto con la definición de la funcionalidad que representará.
- **Etiquetas:** Permiten situar un texto en la interfaz. No son interactivos y puede utilizarse para escribir texto en varias líneas.
- **Campos de texto:** cuadros de una sola línea en los que podemos escribir algún dato.
- **Áreas de texto:** cuadros de varias líneas en los que podemos escribir párrafos.
- **Botones:** áreas rectangulares que se pueden pulsar para llevar a cabo alguna acción.
- **Imágenes:** se usan para añadir información gráfica a la interfaz.
- **Password:** es un cuadro de texto en el que los caracteres aparecen ocultos. Se usa para escribir contraseñas que no deben ser vistas por otros usuarios.
- **Listas:** conjunto de datos que se presentan en un cuadro entre los que es posible elegir uno o varios.
- **Listas desplegables:** combinación de cuadro de texto y lista, permites escribir un dato o seleccionarlo de la lista que aparece oculta y puede ser desplegada.

Además de los botones hay muchos otros tipos de componentes, como pueden ser los elementos de entrada de texto o los elementos de información de salida, como la barra de progreso o la barra de estado, o los elementos compuestos, como las barras de tareas o el combo box.



Para saber más

Puedes mirar la [ayuda](#) de Microsoft para ver más tipos de controles con ejemplos y familiarizarte con los elementos con los que iremos trabajando a lo largo del curso.

a) Ejemplo de diseño de interfaz



Ejercicio Resuelto

Vamos a realizar el diseño de nuestra primera interfaz. Para ello vamos a hacer el [mockup](#) utilizando una extensión de Google que permite arrastrar controles a un documento de Google casi tal y cómo lo haríamos en una herramienta de desarrollo, pero sólo con el objetivo de familiarizarnos con los controles.

Así, nuestro primer proyecto será un formulario para rellenar una pequeña encuesta. Al final debe quedarte un boceto de interfaz semejante al de la imagen.

([encuestaGrande.jpg](#))

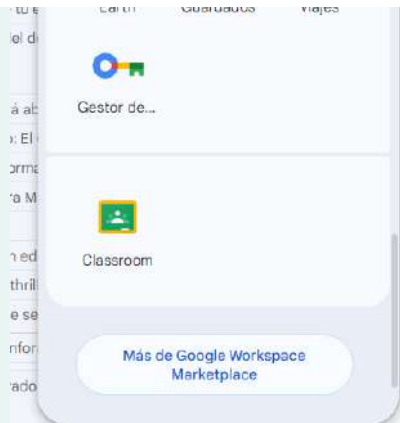
encuesta

La interfaz que debes crear se emplea para hacer una pequeña encuesta en la que se pregunta al usuario algunos datos personales, como la profesión, edad, o número de hermanos, y otros datos relacionados con sus gustos sobre los deportes o en qué medida le gusta al usuario ir de compras, ver la televisión o ir al cine. Verás que, finalmente, obtenemos un boceto de diseño que luego podremos recrear en MS Visual Studio 2022.

Mostrar retroalimentación

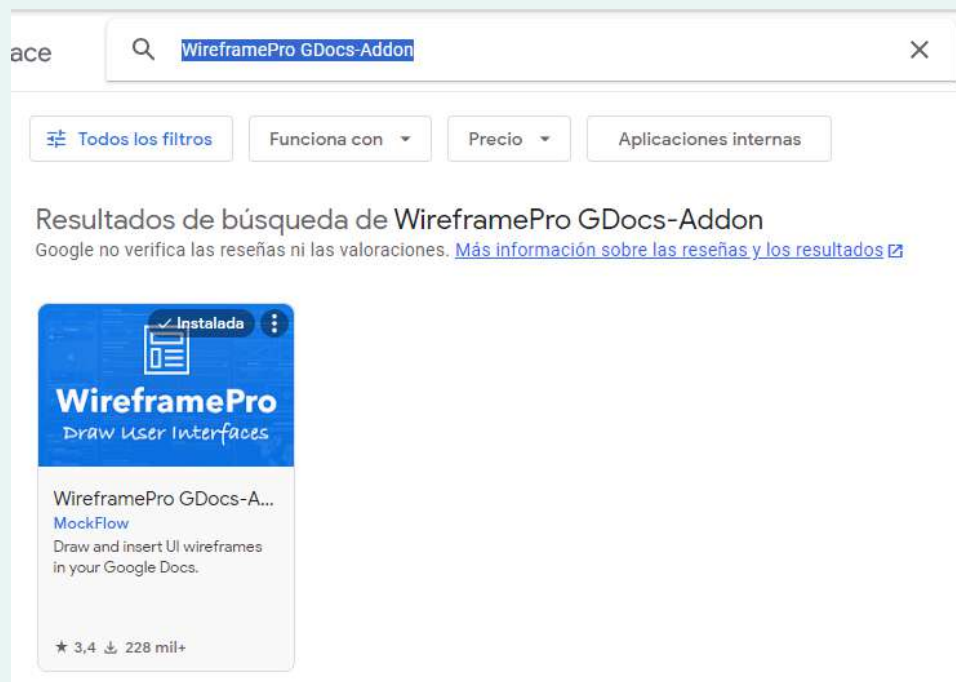
El primer paso es buscar en las extensiones de Google, que se localizan desplegando los puntos de opciones desde nuestro correo de Google desplazándonos hasta el final:



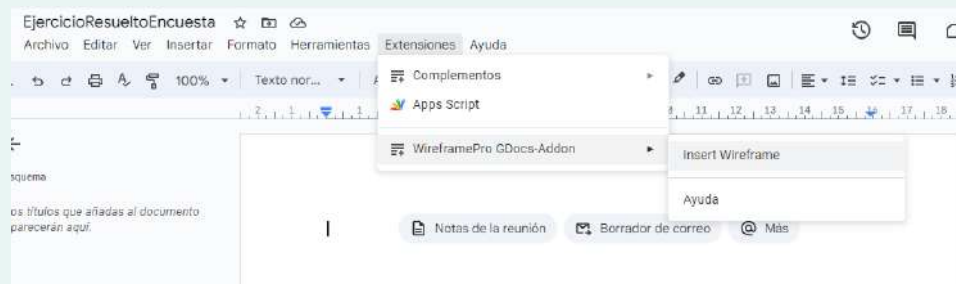


Buscamos: *WireframePro GDocs-Addon*

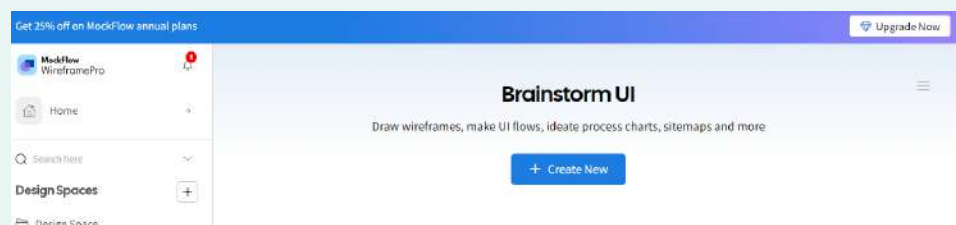
Cuando la tengamos instalada aparecerá así:



A partir de aquí, creando un documento de texto en nuestro drive, podremos acceder a esta extensión que nos permite realizar un boceto del diseño de la aplicación que queremos:

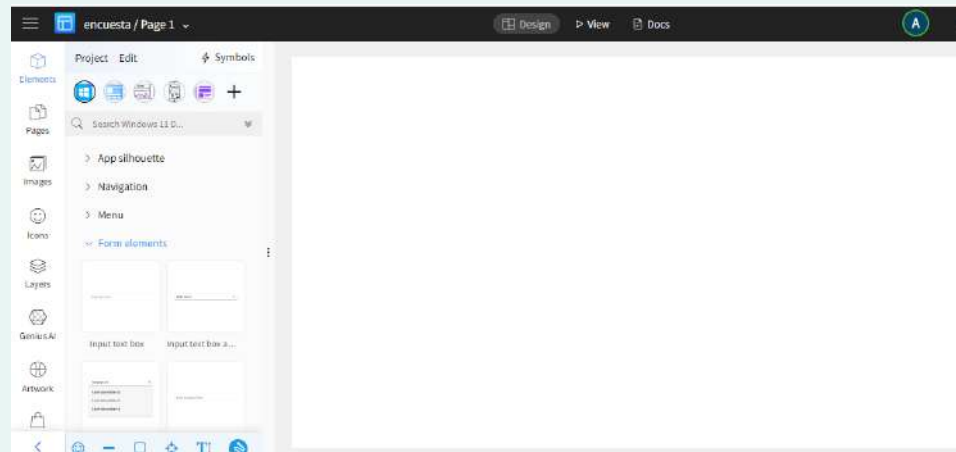


Una vez en el editor del WireframePro, ya podremos iniciar un nuevo elemento y comenzar a seleccionar los componentes necesarios:



Para ello, seleccionaremos Desktop app y Windows 11 Desktop.

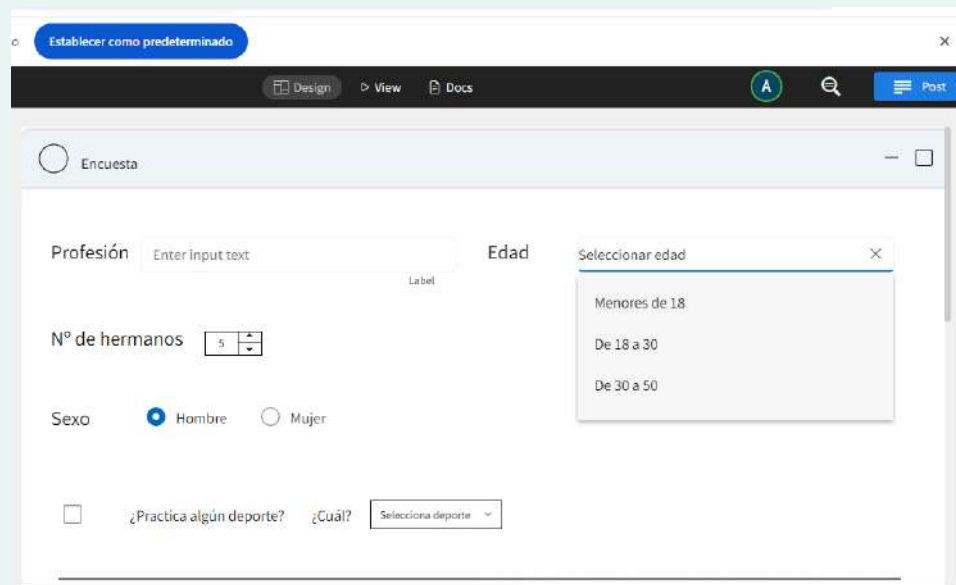
A la izquierda encontraremos un menú desde donde arrastrar los componentes que necesitamos.



Para generar esta interfaz necesitarás los siguientes componentes:

- etiquetas,
- un campo de texto para la profesión,
- un Spinner para el número de hermanos,
- una lista desplegable para la edad, y los deportes
- un grupo de botones de radio para el sexo,
- una casilla de verificación para preguntar al usuario si le gusta el deporte,
- también un separador y
- para las aficiones tres deslizadores.

Cuando tengamos el diseño finalizado, lo publicamos. ¡No olvides ir guardando durante el desarrollo del proyecto!





Para saber más

Por si necesitas más ayuda para manejarte con la extensión de mockup que te recomiendo usar, [aquí](#) encontrarás un vídeo de cómo empezar.



Autoevaluación

Rellena los huecos.

Para diseñar una interfaz tengo que pensar qué usar dependiendo de lo que necesito. Utilizaré una o label para poner texto literal. Los se utilizan para ofrecer la posibilidad al usuario de introducir texto. Y así, con todos los elementos que nos permiten diseñar una interfaz.

Enviar

Para diseñar una interfaz tengo que pensar qué controles usar dependiendo de lo que necesito. Utilizaré una etiqueta o label para poner texto literal. Los cuadros de texto se utilizan para ofrecer la posibilidad al usuario de introducir texto. Y así, con todos los elementos que nos permiten diseñar una interfaz.

2.5. Herramientas propietarias y herramientas libres de edición de interfaces

El desarrollo de una interfaz gráfica de usuario dependerá mucho del lenguaje de programación que se utilice y de su entorno de desarrollo.

Será muy diferente (y se necesitarán herramientas de software diferentes) desarrollar una aplicación informática con un [lenguaje de programación](#)  de **tercera generación**, en el que se trabajará con bibliotecas para poder tener acceso a un entorno gráfico, o trabajar con un lenguaje de **cuarta generación**, con componentes gráficos y de acceso a datos incorporados.

Otra diferencia importante será desarrollar una aplicación para un entorno Web o desarrollarla por un entorno de ventanas (Winforms). Los componentes, los lenguajes, la metodología de trabajo y las posibilidades serán diferentes en los dos casos.

Las herramientas de creación y edición de interfaces son un tipo de software que ayudará al programador a desarrollar interfaces que sigan las indicaciones establecidas en la fase de diseño a partir del análisis de las necesidades. Deberán tener en cuenta todas las fases de desarrollo de un proyecto informático.

Este tipo de software deberá tener en cuenta muchas disciplinas, ya que la creación de una interfaz de usuario engloba conceptos de diseño gráfico, de lingüística, de ergonomía, de sociología, de programación y tecnología informática y de psicología cognitiva, entre otros.

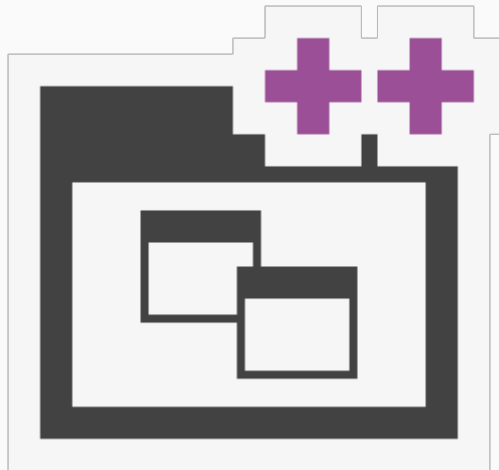
Hay muchas herramientas diferentes en el mercado en función del lenguaje de programación que se utilice, en función del entorno de desarrollo o de si la herramienta es propietaria o es libre. Para conseguir desarrollar de manera sencilla las interfaces, las herramientas incorporan diversos elementos preprogramados que facilitan mucho la creación de interfaces gráficas complejas de una manera sencilla. Así será muy sencillo añadir a una interfaz un menú o un icono, tan sencillo como desplazar este elemento al panel principal donde se desarrolla la interfaz, y modificar posteriormente sus propiedades. Estos elementos pueden ser para fomentar la iteración con el usuario (ayudas, actualización de datos, acciones sobre cursores...) o elementos para configurar los (iconos, ventanas, controladores de dispositivos, menús...). Además, se pueden gestionar los posibles errores por parte del usuario en utilizar la interfaz, procesar excepciones, gestionar y controlar sus accesos y gestionar los dispositivos de entrada.

Otras características a tener en cuenta de las herramientas para la creación y edición de interfaces gráficas es que éstas son tipos de herramientas del tipo **WYSIWYG**, es decir, se podrán observar de manera directa los cambios en el diseño y desarrollo de las interfaces. Lo que hacen es lo que verá el usuario final de las aplicaciones, y se pueden ver los resultados de forma inmediata.

A continuación se enumeran algunas de las muchas herramientas de desarrollo de interfaces existentes:

- **Microsoft Visual Studio.** Se trata de un entorno de desarrollo integrado desarrollado por Microsoft para sistemas operativos Windows. Soporta los lenguajes de programación siguientes: Visual C++, Visual C#, Visual J#, ASP.NET y Visual Basic .NET.
- **Eclipse:** Desarrollado inicialmente por IBM. Actualmente desarrollado por una fundación sin ánimo de lucro, lo que hace la herramienta gratuita y de código abierto. Soporta, entre otros, los lenguajes Java, C++, Perl y PHP. Es multiplataforma.
- **Netbeans:** Desarrollado inicialmente por Sun, que actualmente patrocina los proyectos de desarrollo. Herramienta de código abierto y gratuito. Soporta, entre otros, los lenguajes Java, JSP, C++ y PHP. Es multiplataforma.
- **Monodevelop:** Entorno de desarrollo gratuito y de código abierto. Actualmente patrocinado por Novell. Diseñado para C#, también soporta Java o Boo. Es multiplataforma.
- **SharpDevelop:** Entorno de desarrollo de código abierto. Para los lenguajes de programación C#, Visual Basic .NET y Boo. Para sistemas operativos Windows. Es para desarrolladores que no quieren o no pueden utilizar Microsoft Visual Studio.

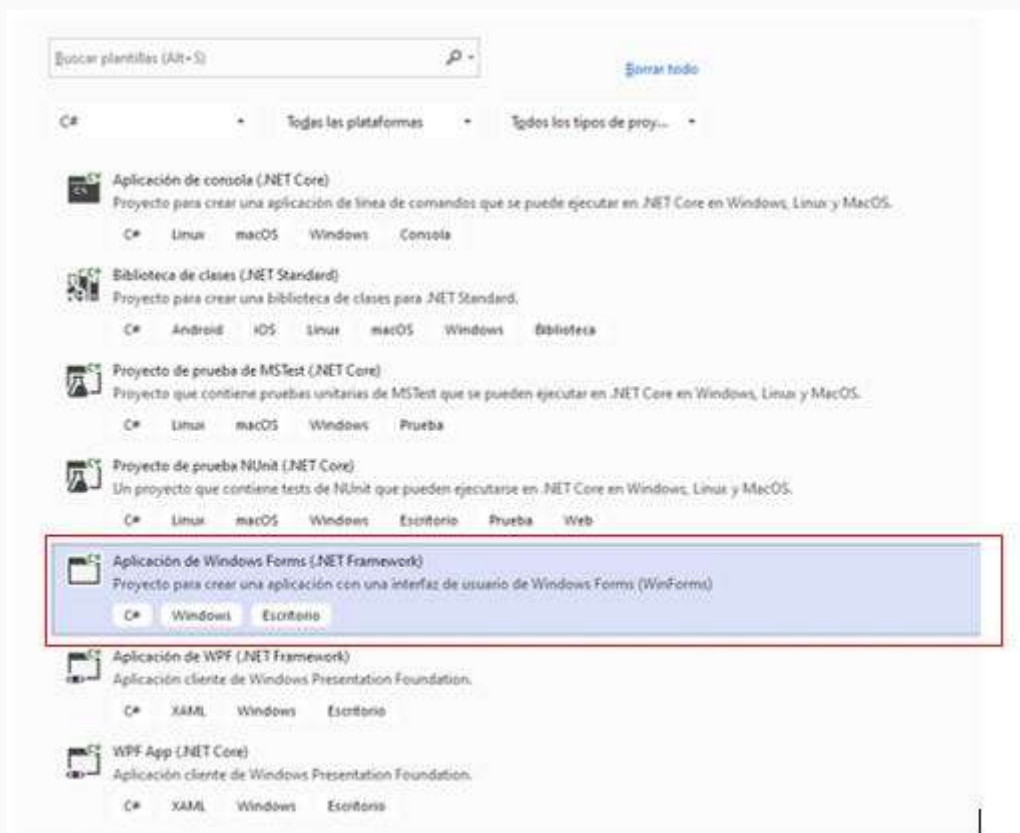
3. EL IDE DE VISUAL STUDIO



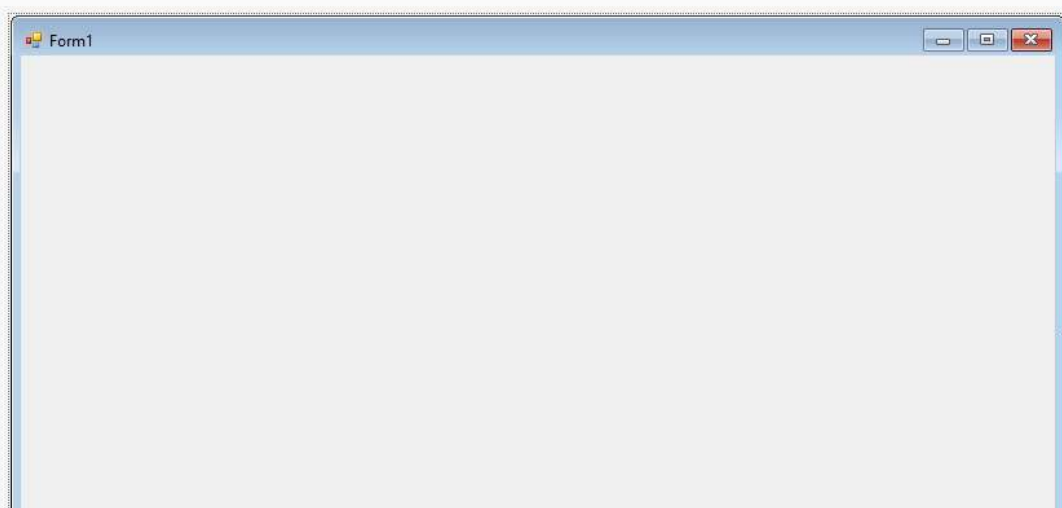
3.1. El IDE o Entorno Integrado de Desarrollo.

Lo primero que tenemos que aprender antes de meternos de lleno con la programación es aprender a manejarnos con el entorno de desarrollo (IDE) de Visual Studio .NET. Como comentamos en el primer tema, Visual Studio es ya un único entorno para todo lo referente al desarrollo con Microsoft y su .NET. Por tanto, si además de usar el C# .NET queremos pasar a otros lenguajes como el Visual Basic o ASP.NET, no tendremos que utilizar otro programa. Incluso podremos trabajar con varios proyectos en el mismo IDE, aunque sean de lenguajes diferentes.

Lo primero que vamos a hacer es cargar el entorno de desarrollo del Visual Studio .NET, así que ejecutamos el programa y creamos un nuevo proyecto de tipo “Aplicación de Windows Forms (.NET Framework)”:

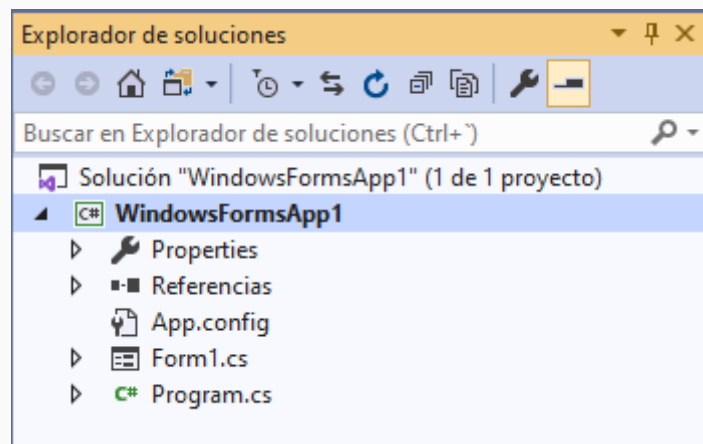


En el proyecto, se habrá creado un formulario que se mostrará en la pantalla central de forma automática.

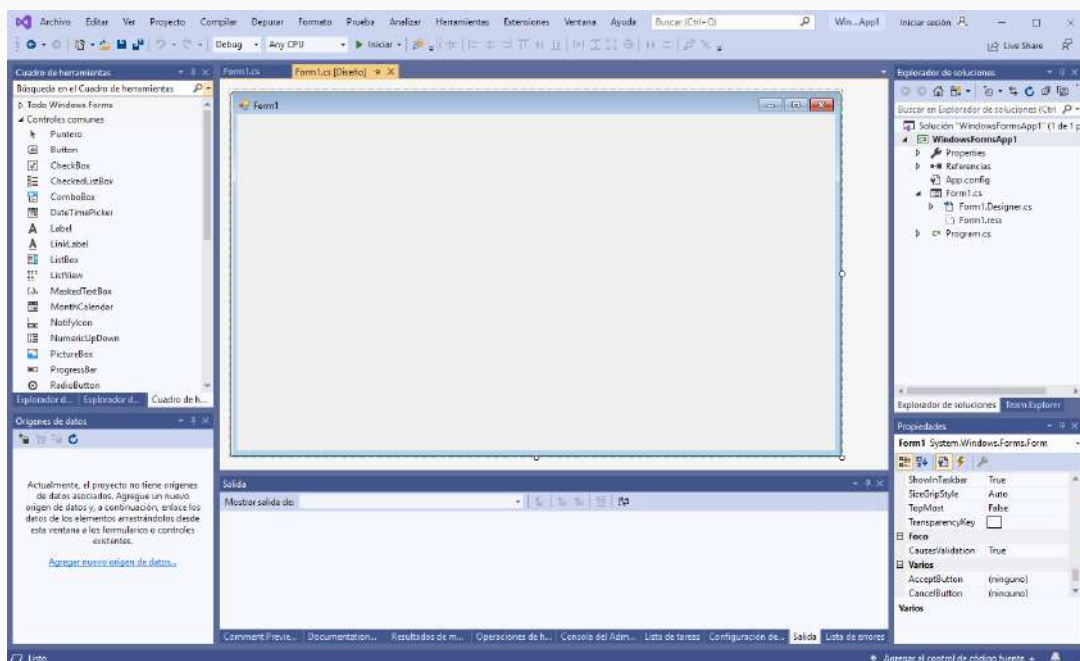




En el lado derecho de la pantalla, verás que hay un "panel" o ventana en la que se indica el proyecto actual y se muestran los ficheros que lo componen. Ese panel es el **Explorador de Soluciones**:



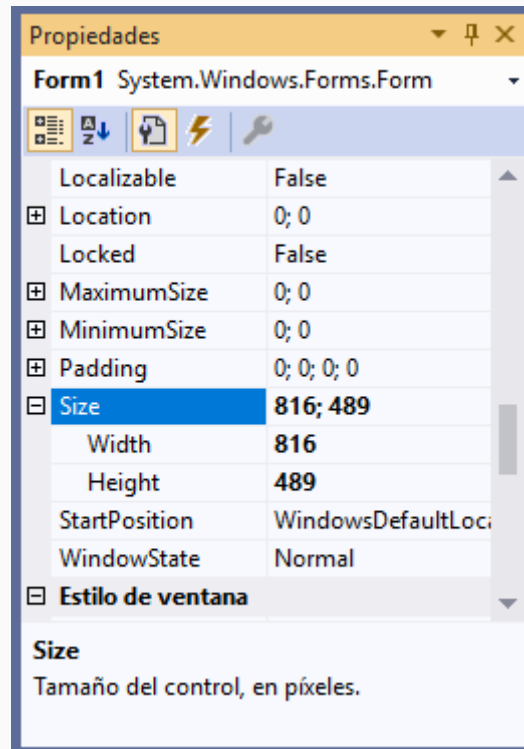
Una vez cargado el formulario, la pantalla tendrá un aspecto similar a este:



Donde distinguimos en la parte izquierda el cuadro de herramientas, en el centro el formulario principal y en la parte derecha las ventanas de propiedades y el explorador de soluciones.

3.1.1. Cambiar propiedades del IDE

El tamaño del formulario que se ha insertado por defecto es de 300 × 300 píxeles. Vamos a cambiarlo a 900 × 600.



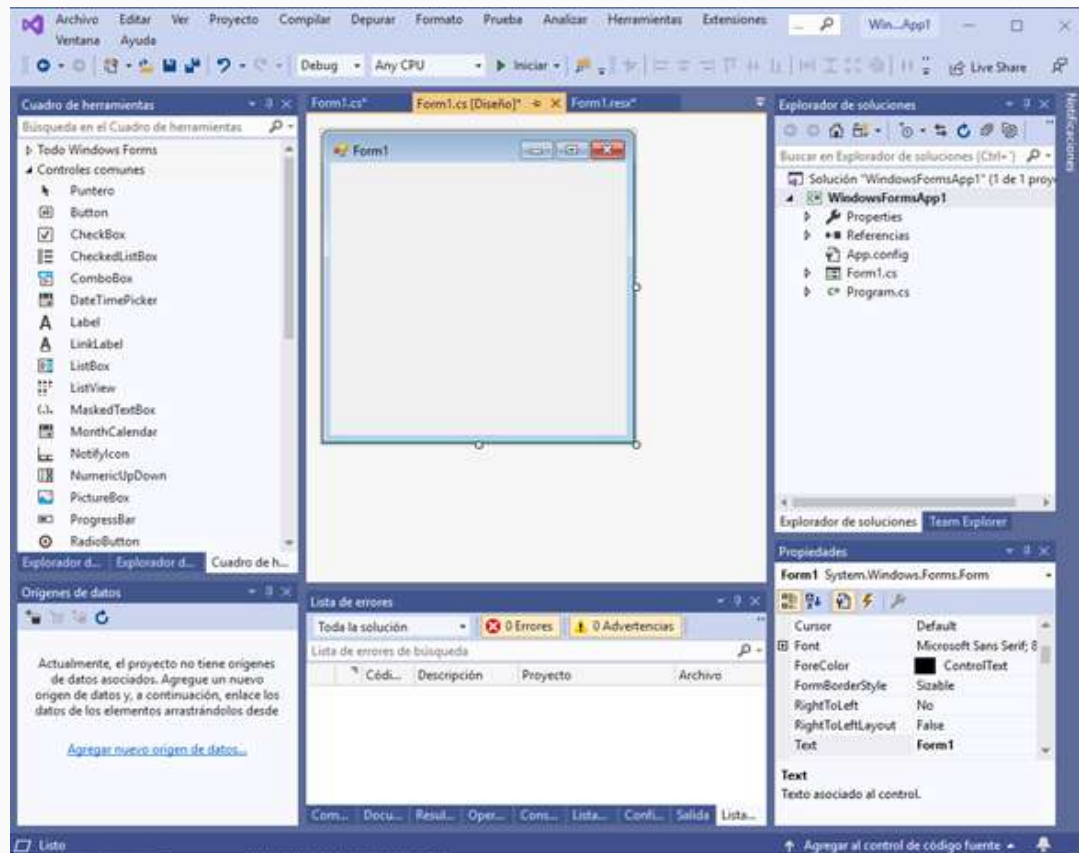
Seleccionamos el formulario haciendo clic en él y vamos a ver sus propiedades. En el panel de la derecha, debajo del explorador de soluciones, está la ventana de propiedades del elemento que actualmente esté seleccionado, en nuestro caso son las propiedades del Form1. Vamos a buscar el elemento **Size** para poder cambiar el tamaño del mismo. Se muestran dos valores separados por punto y coma, pero también hay una cruz a la izquierda de la palabra Size. Si pulsas en esa cruz, se mostrarán los tamaños a lo que se refiere esos valores separados por punto y coma. Posiciónate en ellos y asígnale los valores 900 y 600, respectivamente. De esta forma, tendremos un formulario más grande, en el que será más fácil posicionar los controles que queramos añadir al formulario en cuestión. Además hemos visto debajo de las propiedades que cuando seleccionamos una nos aparece una descripción de ella: Size, determina el tamaño del control en píxeles. Esto será de gran ayuda para más adelante. Asimismo nos encontramos con algo importante: las propiedades. En este caso los formularios tienen todas las propiedades que aparecen en esta ventana y que hacen referencia a su aspecto y comportamiento. En nuestro caso hemos modificado la propiedad Size para cambiar el tamaño del formulario.

Fíjate que después de haber cambiado esos valores, los mismos se muestran en "negrita", indicándonos de esta forma que son valores que nosotros hemos asignado, distintos a los que tiene por defecto. Si el tamaño del formulario no es el adecuado para tu gusto puedes cambiarlo y dejarlo como estaba, lo importante es que ya hemos cambiado una propiedad del formulario.

3.1.2. El inicio

Repitamos otra vez el inicio de una aplicación para ver los elementos de esta primera pantalla:

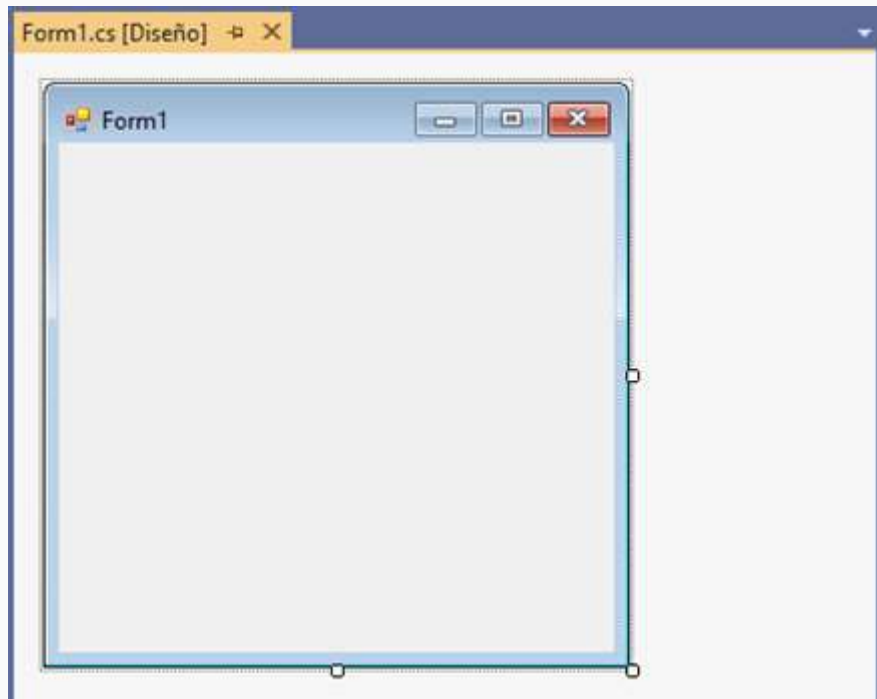
Veamos qué herramientas tenemos en pantalla:



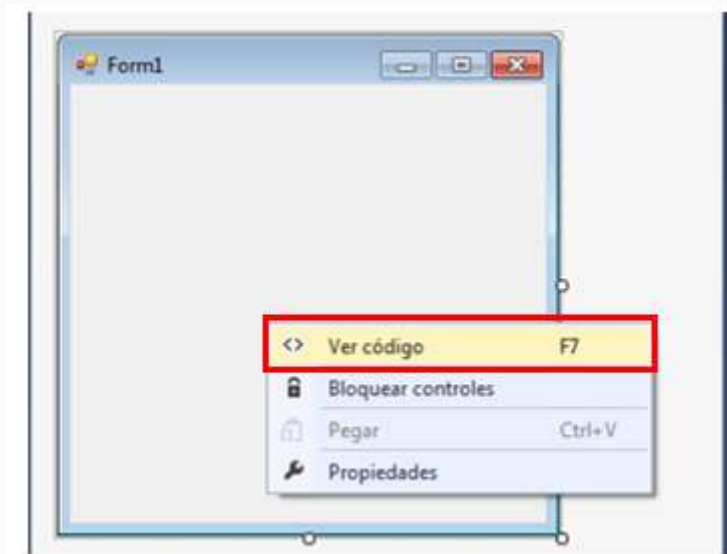
- Parte izquierda. Inicialmente no teníamos desplegado ningún panel, pero vimos cómo dejar visible el cuadro de herramientas haciendo clic en la solapa vertical. Dejaremos visible siempre este panel haciendo clic en la chincheta que aparece en la parte superior del cuadro.
- Cuadro de herramientas. Disponemos de varias secciones con distintas funciones para cada una de ellas. De momento dejamos las predeterminadas. Si las exploramos veremos los grupos de control que tenemos en esta versión: controles comunes, controles contenedores, de menús... La segunda pestaña nos mostrará los orígenes de datos para conectarnos a bases de datos.
- Parte central: formulario de edición. Aquí nos podremos mover con las pestañas superiores por los distintos formularios que tengamos abiertos.
- Parte derecha: son ventanas muy importantes y que tendremos que tener siempre visibles. Tenemos el explorador de soluciones y debajo, la imprescindible ventana de propiedades.
- Parte inferior: por defecto aparece la ventana de lista de errores pero también se muestran bastantes más que nos serán de gran ayuda a lo largo del proyecto. Sobre todo durante la ejecución de los programas y para poder ver información de depuración.

Cada una de estas partes la veremos con detalle a continuación.

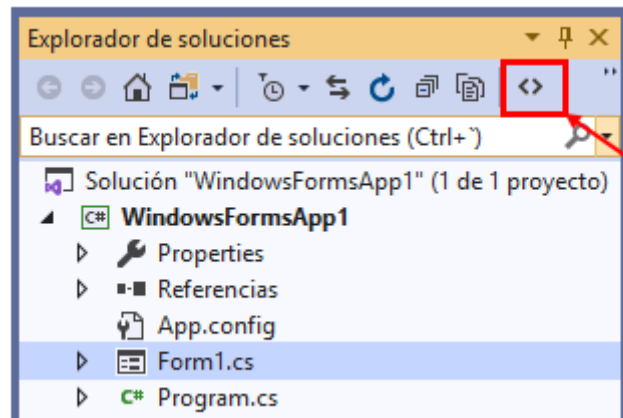
3.1.3. Ventana principal de trabajo



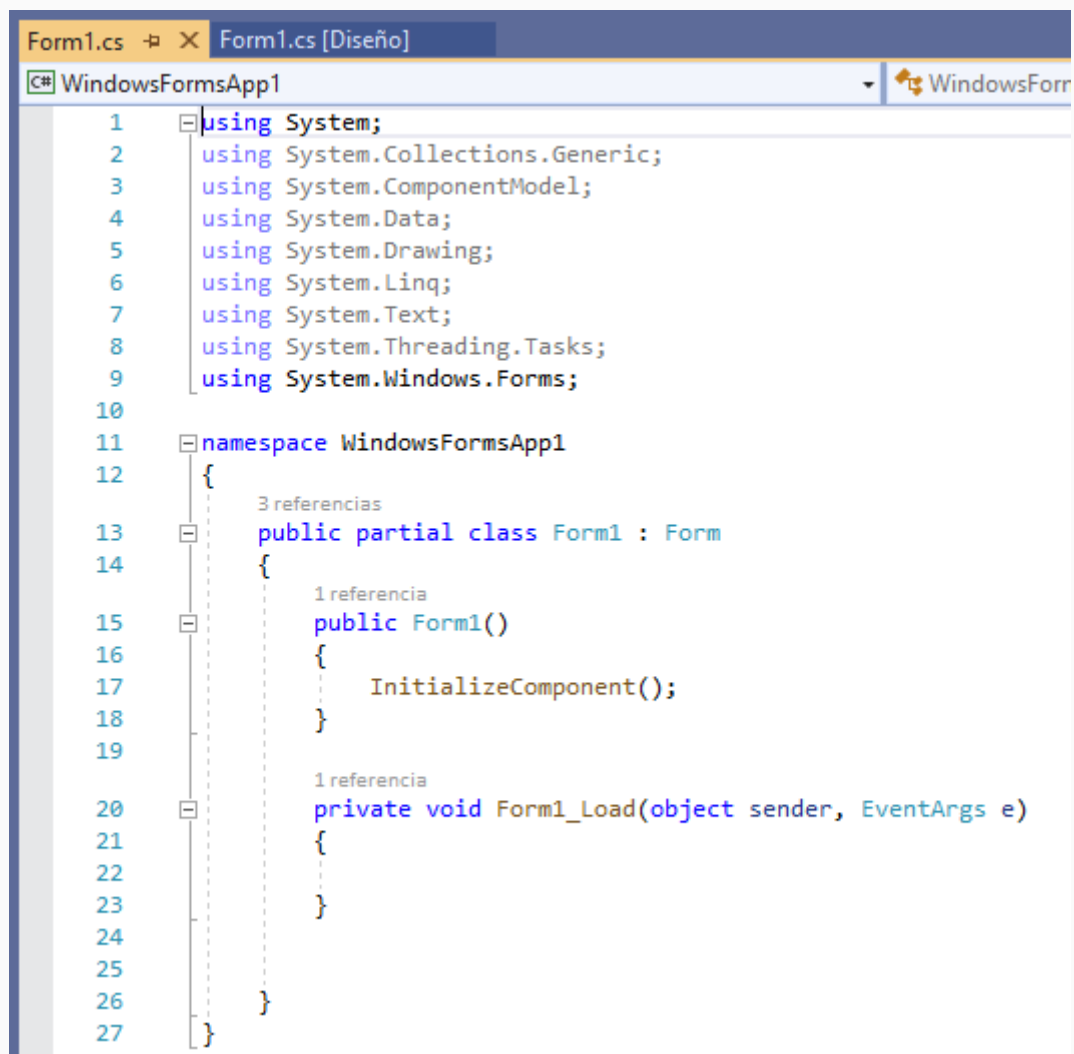
Es la ventana que más área de trabajo utiliza y la que habitualmente utilizaremos para introducir nuestro código. Para pasar a ver el código de este formulario podemos hacer dos cosas. Una, en el formulario elegimos "Ver código" haciendo clic con el botón derecho del ratón:



O pulsando en el Explorador de soluciones de la derecha el botón que indica "Ver código":



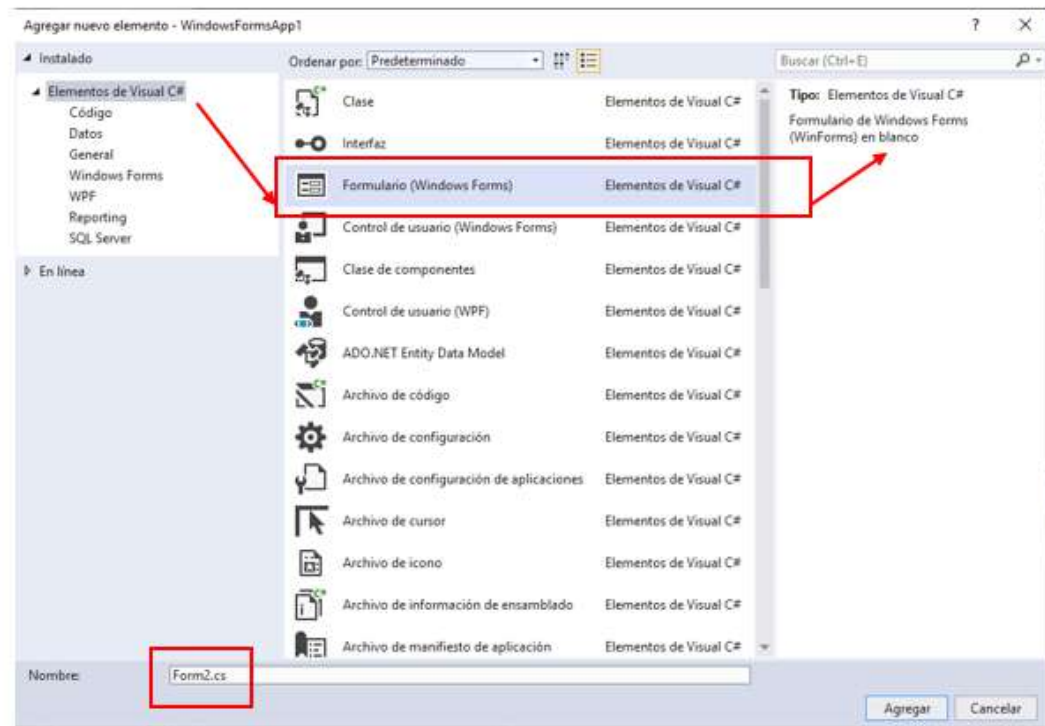
Para llegar a la ventana de código previamente debemos tener marcado el formulario del que queremos ver el código



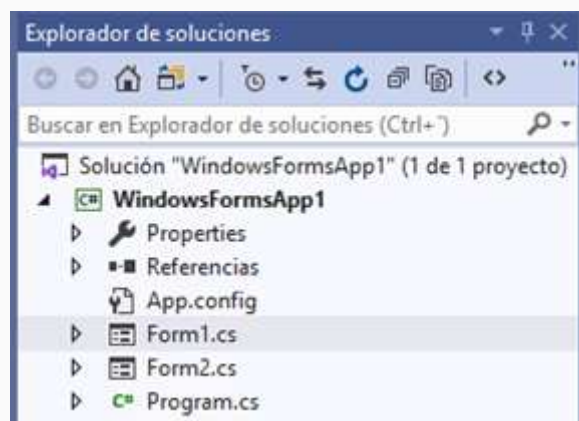
Vamos a añadir un nuevo formulario para ver cómo navegar entre ellos. En el menú "Proyecto" seleccionamos la opción "Agregar Windows Forms" o "Agregar nuevo elemento".



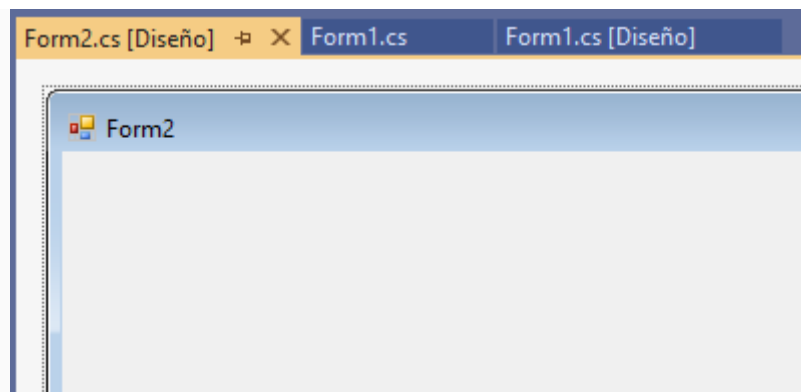
Indicamos que queremos un formulario:



Al pulsar en "Agregar" lo veremos inmediatamente en el explorador de soluciones:



Podemos ver cómo ahora hay una pestaña más con este segundo formulario.



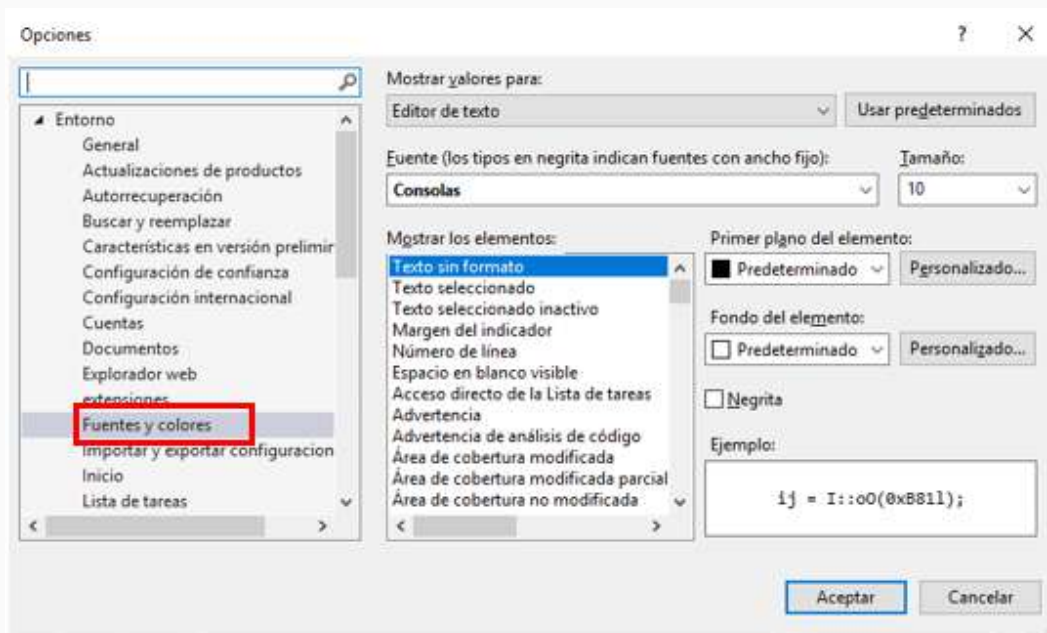
Podemos cambiar con las teclas Control+Tabulador o lógicamente con el ratón. Para cerrar alguna de ellas pulsaremos las teclas Control + F4 o en la "x" de la derecha.

En esta ventana escribiremos el código del programa en forma de declaraciones, procedimientos, funciones, módulos,...

Veamos algunas posibilidades para personalizar esta ventana.

Ajustar el color y el tipo de letra

Para cambiar el tipo de letra y algunos detalles más del estilo de la fuente seleccionaremos "Opciones" del menú "Herramientas" y luego en la parte del Entorno seleccionamos Fuentes y colores:

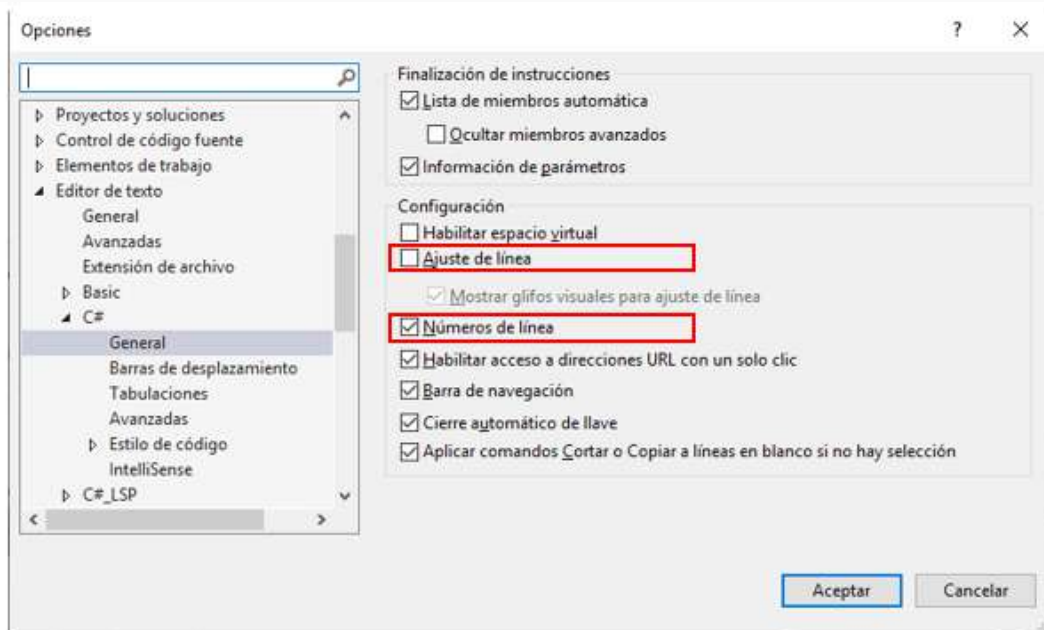


Aquí se pueden modificar cualquiera de los estilos utilizados en el editor, no sólo para la edición, sino, para el comportamiento en general.

Nota: Se recomienda que el tipo de letra utilizado sea de paso proporcional como la "Courier" que es la normalmente utilizada. Esta fuente de paso fijo deja el mismo espacio para la letra "i" que para la "m" con lo que las columnas quedan todas alineadas. De no utilizarlo el manejo del código con los sangrados y tabulaciones nos dejará unos listados muy incómodos.

Mostrar números de líneas

Por defecto esta opción está activada. Para desactivarla utiliza la opción C#, General de la sección "Editor de texto" de la pantalla anterior:

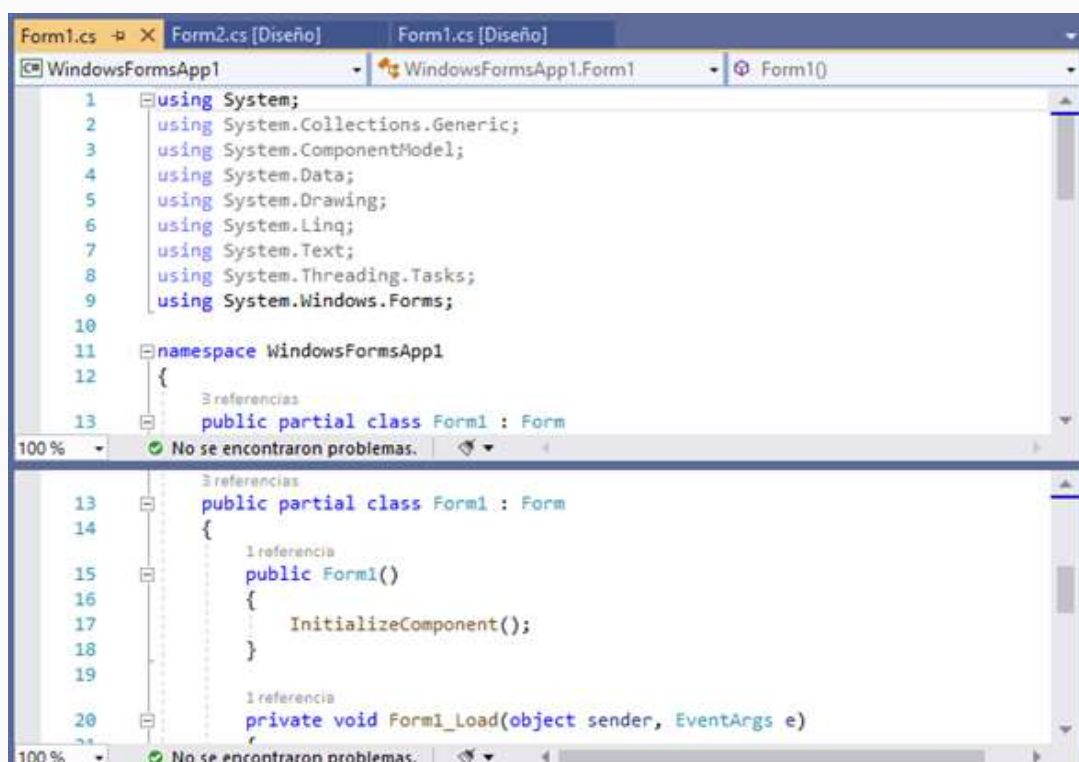


Gracias a esta opción podremos utilizar atajos como los que ofrece Excel o Project de ir hasta una línea en concreto utilizando la opción *Ir a* del menú *Edición*.

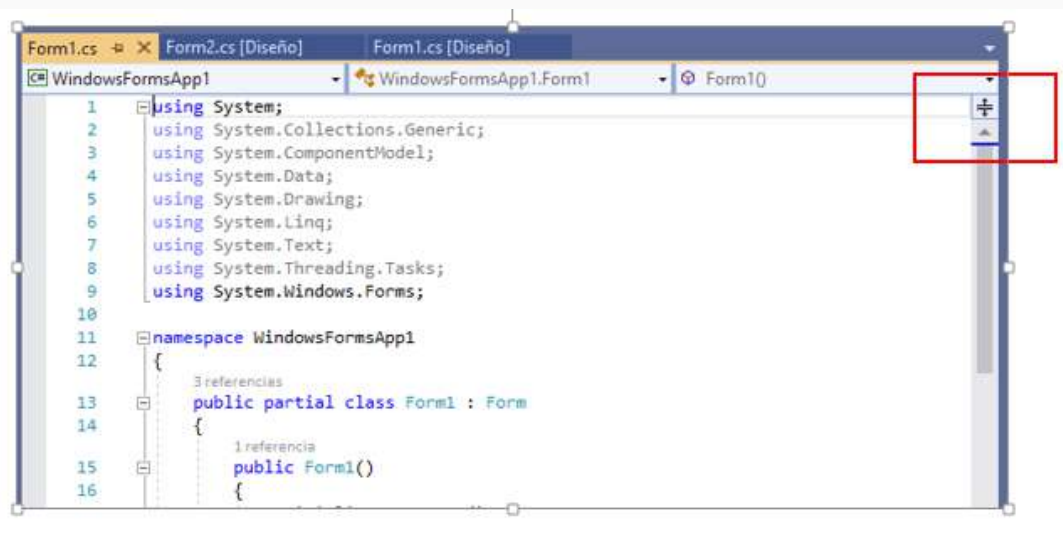
Dividir la ventana

Al igual que en otros programa de Office podemos dividir la ventana de código. Es especialmente útil si queremos visualizar permanentemente un área de código y queremos ver otra a la vez para, por ejemplo, comparar secciones de código, copiar y pegar,...

Para esto utilizaremos la opción *Dividir* del menú *Ventana*:

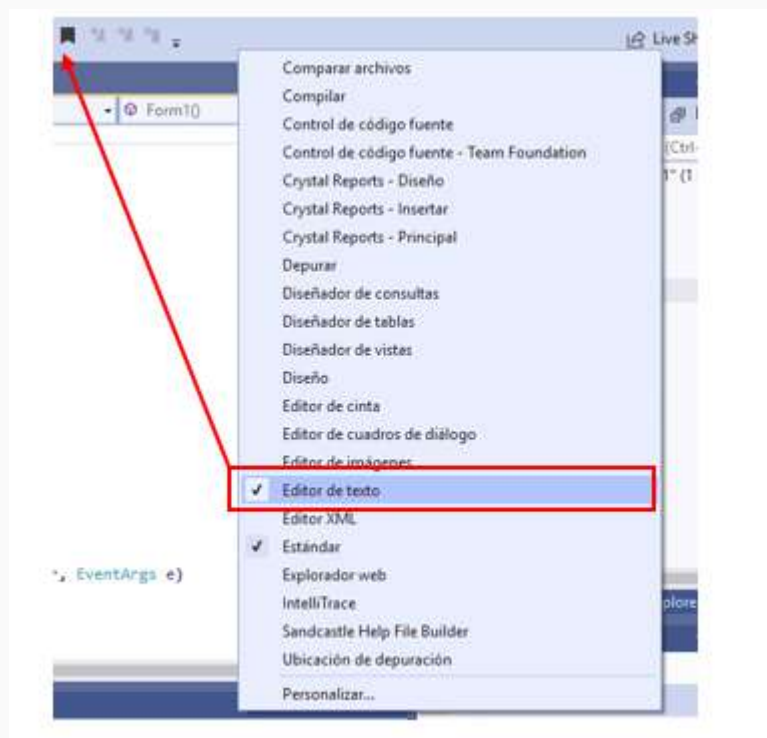


También podemos hacer clic en el divisor de ventana que se encuentra justo encima de la barra de desplazamiento:



Marcadores

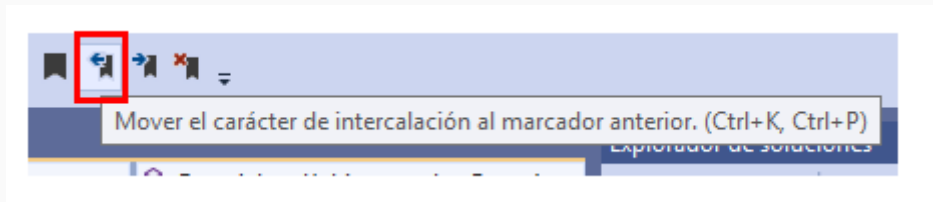
Los marcadores son señales que introducimos en las líneas de código. Con esto podremos volver rápidamente a la línea en la que colocamos el marcador. Es especialmente útil en listados grandes para ver una determinada parte y volver donde estábamos editando rápidamente. Necesitaremos la barra de botones "Editor de texto". En caso de no mostrarse la obtenemos pulsando con el botón derecho en las barras de botones que tengamos desplegadas:



Los iconos indicados con la flecha  son los que controlan estos marcadores, que se representan de esta forma:



Si tenemos más de un marcador nos podemos mover entre ellos desde los iconos de la barra.



Poner comentarios en el código

Poner comentarios en el código es una buena práctica que debemos utilizar. Cuando estemos escribiendo nuestro programa, sabemos lo que estamos haciendo y estaremos plasmando todas las ideas que tenemos en la cabeza en el código. Pero seguramente si abrimos este código dentro de un mes tendremos muchos problemas para recordar ciertos detalles sobre el funcionamiento de algunos algoritmos. Unos comentarios en los sitios precisos dando explicación a unas rutinas, fragmentos u otros serán de un valor inestimable para nosotros o compañeros que deban editar el código. Por ejemplo:

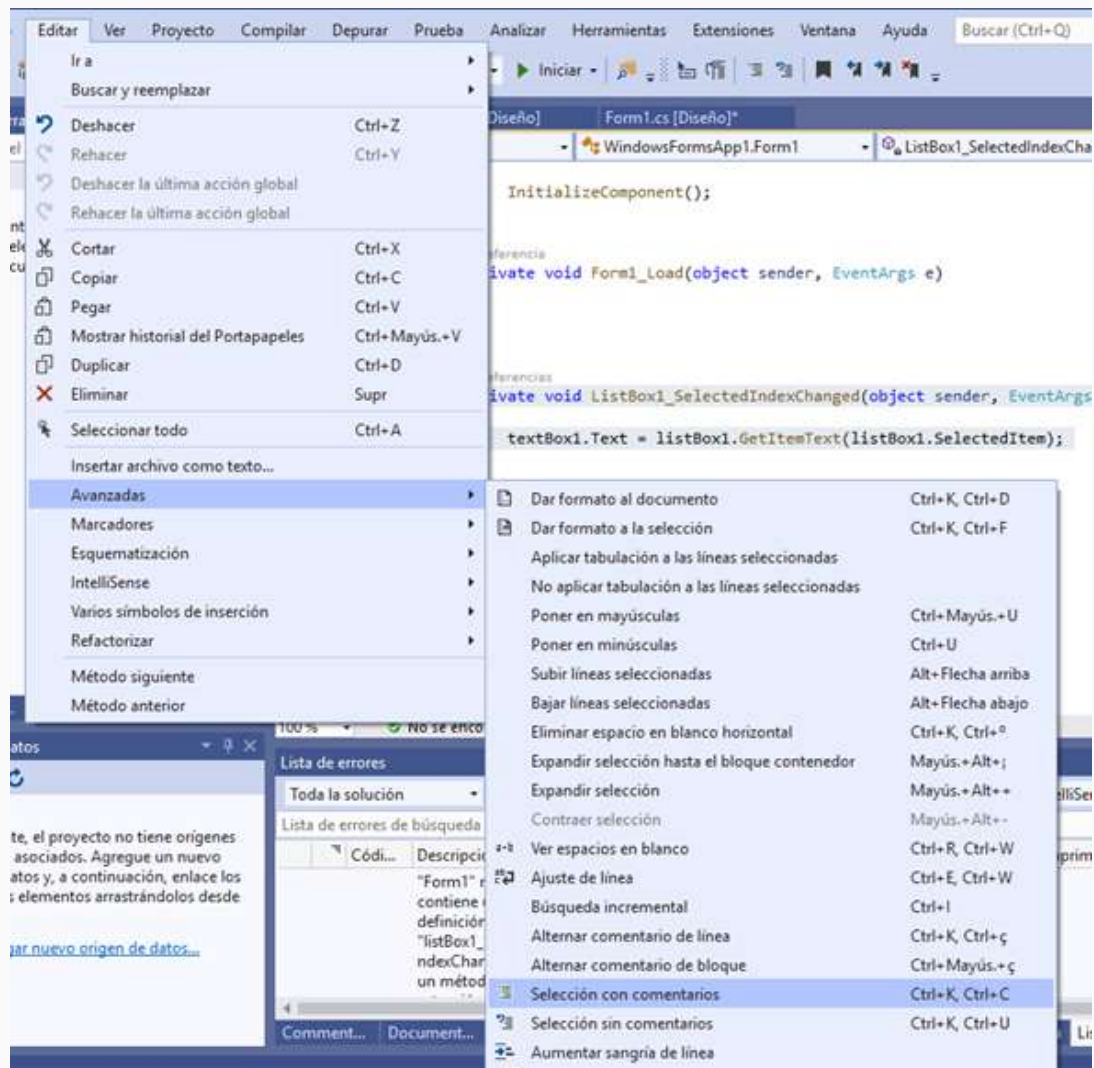
```
private void ListBox1_SelectedIndexChanged(object sender, EventArgs e)
{
    // Cuando pulsamos en un elemento de la lista
    // obtenemos su contenido con el método GetItemText
    textBox1.Text = listBox1.GetItemText(listBox1.SelectedItem);
}
```

Los comentarios, que en el IDE aparecen en color verde, nos ayudan a explicar las partes del código.

También nos pasará que tengamos que retocar el código de otra persona. Si su código está detallado nos alegraremos de que tenga un buen estilo de programación comentando debidamente el código para poder revisarse con detalle. Otras veces simplemente queremos que no se ejecute un fragmento de código y queremos que se quede momentáneamente como comentarios.

Además de poner los comentarios necesarios a medida que escribimos el código (incluyendo dos simples barras // antes del comentario) podemos hacer que el editor nos marque una zona como comentario, es decir, sin valor para C# que lo ignorará. Para esto, marcamos las líneas que queramos y utilizamos la

opción del menú "Selección con comentarios" de la sección "Avanzadas" en el menú "Editar".



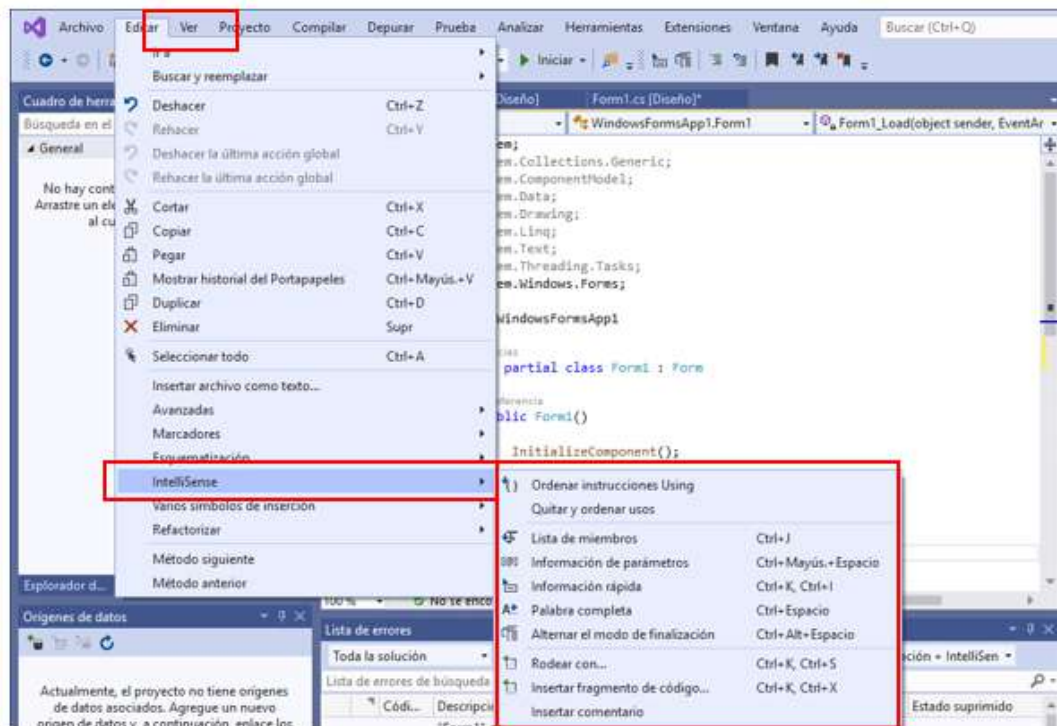
También disponemos de los iconos para realizar esto:



Intellisense

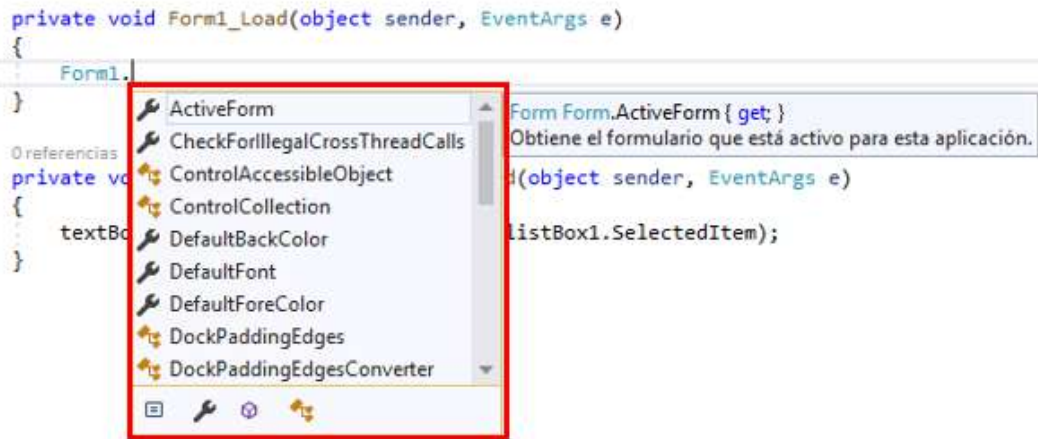
Para terminar con esta parte del editor vamos a ver una tecnología que nos ayudará a escribir el código y, en muchas ocasiones, nos servirá como comprobación de la sintaxis del código que estamos escribiendo.

En el menú "Editar" podemos ver las partes de que se compone este "Intellisense":



Veamos a qué se refiere cada una de ellas:

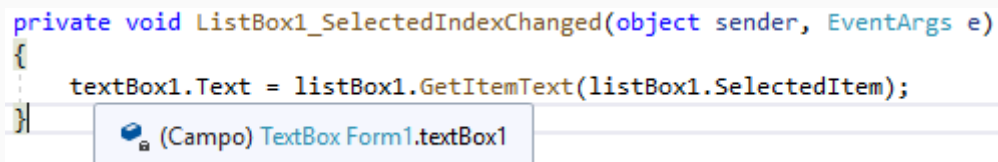
- Ordenar instrucciones Using. Ordena las instrucciones *Using* alfabéticamente en la parte superior del archivo ya que de este modo resulta más fácil encontrar una directiva using.
- Quitar y ordenar usos. Ordena y quita instrucciones *Using* sin cambiar el comportamiento del código fuente. Compacta el código fuente eliminando las instrucciones *Using* que no se usan; además, las ordena, lo que hace que sean más fáciles de leer.
- Lista de miembros. Después de escribir un carácter desencadenador como un punto (.), aparece una lista de los miembros válidos de un tipo (o espacio de nombres). Si sigues escribiendo caracteres, la lista se filtrará y solo incluirá los miembros que empiecen por esos caracteres o donde el principio de cualquier palabra del nombre empiece por esos caracteres. IntelliSense también efectúa búsquedas de coincidencias "camel case", por lo que puedes escribir la primera letra de cada palabra con camel case del nombre del miembro para ver las coincidencias. Después de seleccionar un elemento, puedes insertarlo en el código presionando la tecla TAB o insertando un espacio. Si seleccionas un elemento y escribes un punto, el elemento aparece seguido del punto, con lo que se muestra otra lista de miembros. Cuando seleccionas un elemento, obtendrás la información rápida del mismo antes de insertarlo.



- Información de parámetros. Cuando llamemos a procedimientos y funciones (subprogramas o fragmentos que realizan una operación o proceso) éstos necesitan normalmente unos parámetros para poder trabajar correctamente. Pues bien, en el momento que escribamos la llamada a este procedimiento o función nos irá indicando las variables o parámetros que debemos escribir y de qué tipo. En este ejemplo se llama a un método que necesita al menos un parámetro. Al escribir la llamada nos lo indica en la ayuda:



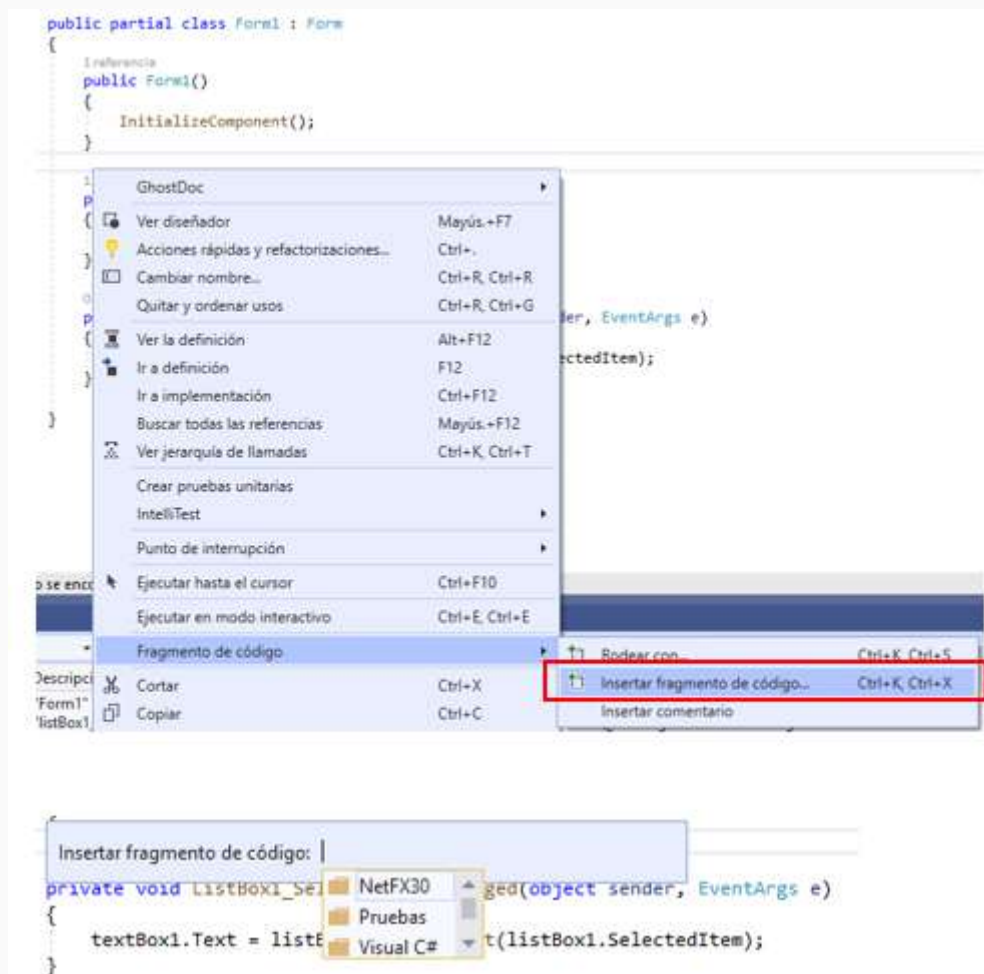
- Información rápida. Nos muestra una viñeta con información.



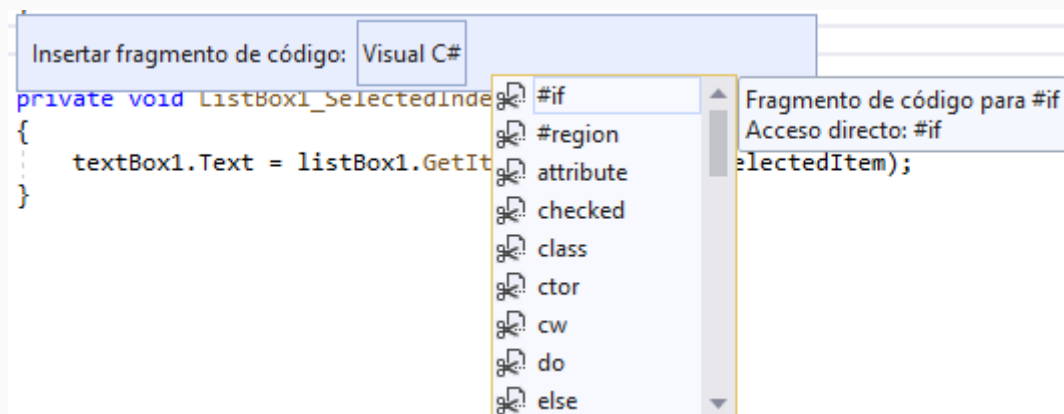
- Palabra completa. Escribe el resto del nombre de una variable, un comando o una función después de que se hayan escrito suficientes caracteres como para identificar de forma exclusiva esos elementos. Para invocar la función de palabra completa, elegimos **Edición > IntelliSense > Palabra completa** o presiona **Ctrl+Barra espaciadora** o selecciona el botón **Palabra completa** de la barra de herramientas del editor.
- Alternar el modo de finalización. Alterna entre los modos de finalización estándar y sugerencias. Cuando comenzamos a escribir el nombre de un objeto, IntelliSense muestra todos los miembros válidos en una lista desplazable. Al escribirlos, se resalta el símbolo adecuado. También se puede cambiar al modo de sugerencias, en el que solo se inserta en el código el texto que se escribe. Por ejemplo, si escribes un identificador que no está en la lista y presionas la tecla **TAB** en modo de finalización, la entrada reemplaza al identificador escrito.
- Rodear con. Rodea con la expresión elegida el fragmento de código

señalado.

- **Insertar fragmento de código.** Con esta opción podemos añadir código genérico que nos servirá de plantilla para seguir escribiendo. Pulsaremos con el botón derecho.



Por ejemplo, seleccionamos Visual C#. Al indicarlo nos propone estas situaciones:



Al seleccionarlás nos introducirá el código correspondiente. Personalmente es más costoso esto que consultar la ayuda, pero es una ayuda más.

- **Insertar comentario.** Inserta comentarios XML sobre un elemento de código. También se pueden insertar comentarios escribiendo /// en C#.

3.1.4. Ventanas adicionales

Además de la ventana principal que hemos visto antes tenemos una serie de ventanas suplementarias que nos ayudarán en nuestro trabajo. Vamos a ver cómo trabajar con ellas para adaptarlas a nuestro gusto.

Las ventanas adicionales están por defecto ocultas en los laterales de nuestro IDE, indicando con una ficha que están ocultas pero disponibles y que se desplegarán al pulsar con el ratón. Esto es verdaderamente útil porque las utilizaremos en algunas ocasiones pero no desearemos que nos ocupen permanentemente un área de pantalla. Veamos este ejemplo:

Vemos que está activa la ventana del "Cuadro de Herramientas" y que debajo tenemos disponible la ventana de "Explorador de objetos de SQL Server" y de "Explorador de servidores". Basta con pulsar con el ratón sobre esta pestaña para que se active la ventana. Si en el área lateral de la izquierda pulsamos con el botón derecho nos aparecerá una lista con todas las ventanas disponibles.

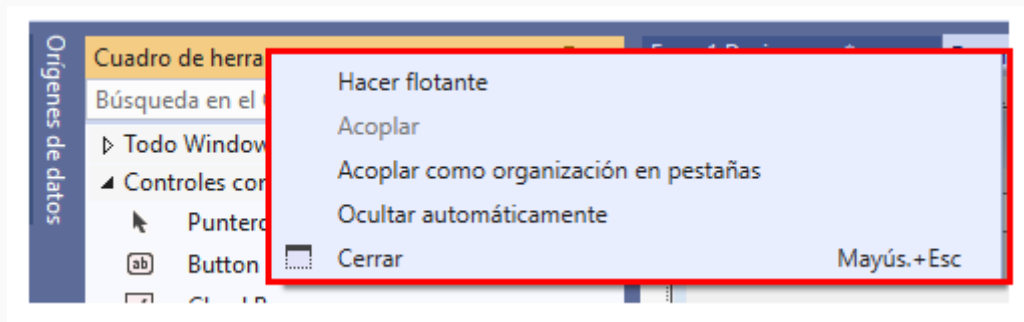


Normalmente estas ventanas disponen de cuatro posibilidades de visualización:

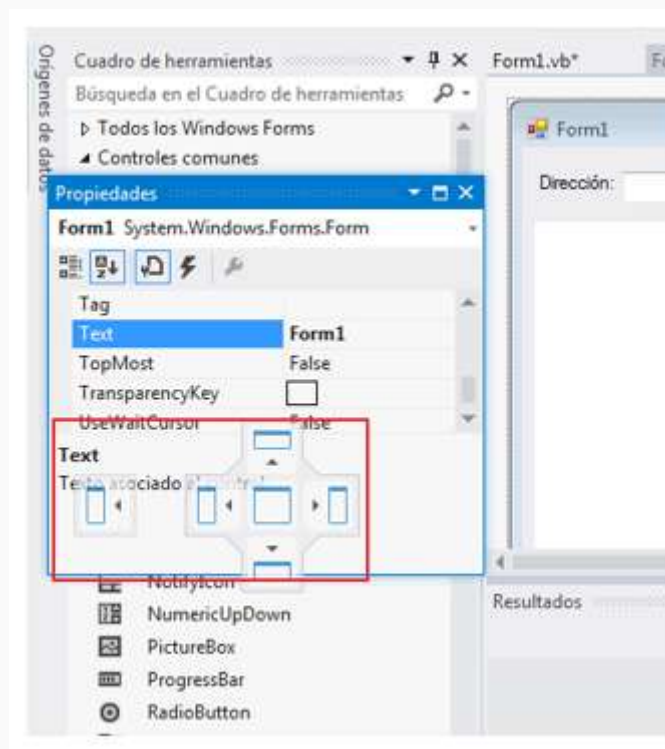
- Se acopla con las ventanas contiguas repartiendo el espacio. Por ejemplo, es habitual tener acopladas en una misma columna la ventana del explorador de soluciones y la ventana de Team Explorer.
- La ventana no se muestra.

- Está visible pero no se acopla en ningún sitio. Permanece en primer plano en nuestro IDE.
- Ocultar automáticamente. Es la posición predeterminada y se activa pulsando el ratón en la pestaña.

En ocasiones queremos que una ventana permanezca visible permanentemente. Para esto, una vez la tengamos visible pulsamos en la chincheta que aparece en la parte superior derecha . Para ver todas las opciones pulsaremos con el botón derecho encima del título y elegiremos el estilo que queramos:

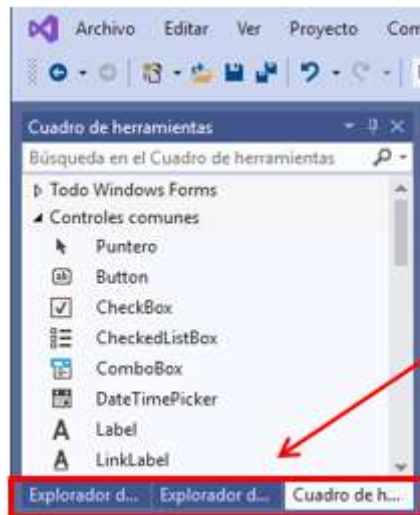


Quizás el más complicado de configurar es el "acoplable" ya que se debe arrastrar con cuidado el título de la ventana para que la acople a la posición que deseemos. Para conseguir que se acoplen varias ventanas arrastraremos una ventana hacia la zona de trabajo de otra y soltaremos el ratón. Si intentamos arrastrar por ejemplo la de propiedades, al moverse aparecerá lo siguiente:



Que son las posiciones donde se acoplará la ventana. En el ejemplo, vemos las posibles direcciones donde quedaría acoplada.

La última posibilidad, y casi la más práctica, es acoplar varias ventanas organizándolas en fichas. De esta forma sólo se ve una y las otras se desplegarán al ir pulsando en los títulos.



Para conseguir acoplarla hemos arrastrado la ventana al cuadro de herramientas que estaba abierto. Si nos fijamos en la parte inferior, están disponibles las ventanas del Explorador de objetos de SQL Server y del Explorador de servidores.

Si eliminamos accidentalmente alguna ventana, cosa muy probable cuando estamos aprendiendo a manejar el IDE, siempre podemos ir a la opción del menú:

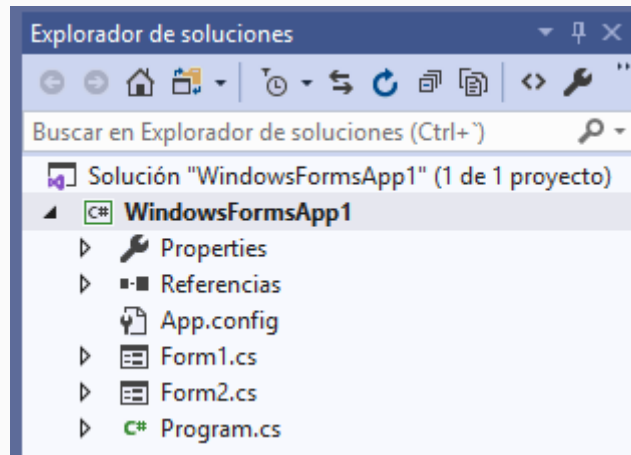


Hay más ventanas que podemos ver en la opción "Otras ventanas" del menú anterior.

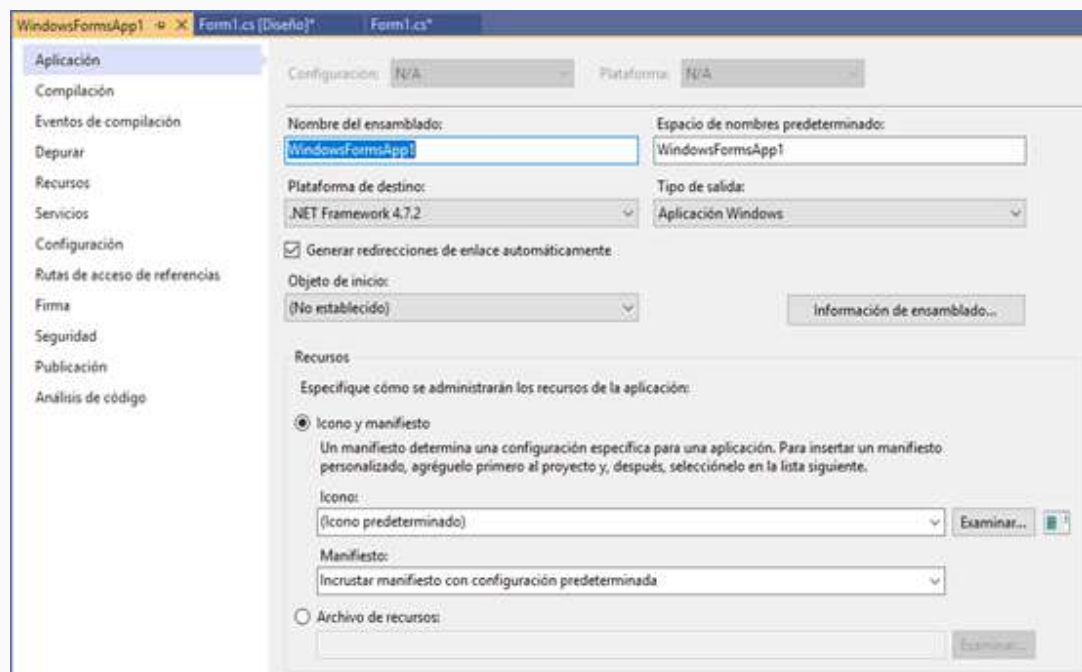
3.1.5. Explorador de soluciones

El explorador de soluciones es una herramienta indispensable. Contiene todos los ficheros de que consta nuestra solución. Permite navegar entre las distintas carpetas y archivos que componen el código fuente de una solución. Una solución puede contener varios proyectos, que se corresponden con varias aplicaciones y/o bibliotecas de clases.

Veamos la pantalla del ejemplo anterior:



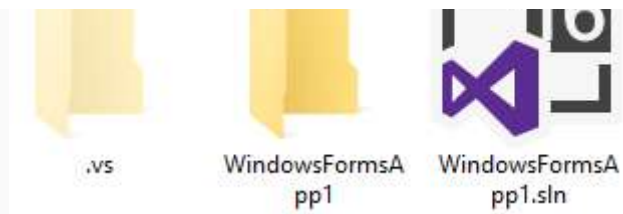
Vemos primero el nombre de la solución y debajo el nombre del proyecto; debajo de él un fichero que pone "Properties" que al hacer doble clic sobre él veremos qué tiene:



Como ves, aparece "Nombre del ensamblado" en lugar de aplicación. Recuerda lo que vimos sobre .NET en el primer tema donde lo que generamos se le llama ahora "ensamblados" aunque lógicamente se trata de una aplicación, pero se utiliza por seguir la nomenclatura de este entorno.

Si vemos la carpeta donde guardamos un proyecto podemos ver estos iconos:





Como ves, hay uno que tiene de extensión ".sln" que es el que define la solución, que en este caso se compone de un proyecto de una sola aplicación de "Windows Forms", o de formularios. En ocasiones será necesario crear una solución con varios proyectos distintos dentro de ella, pero de momento, se compondrán, como hemos hecho hasta ahora, de un solo proyecto.

Volviendo a la ventana de propiedades de nuestro proyecto podemos ver desde el nombre de la aplicación o ensamblado que va a generar hasta el icono que le pondrá al programa cuando lo terminemos. Siguiendo con el explorador de soluciones vemos que aparece el icono del formulario "Form1". Si recuerdas, en la carpeta aparecen varios ficheros más, pero se ocultan para tener el área más limpia. Aparece como título el nombre genérico de "WindowsApplicationxx".

Fichero "App.config"

En el explorador de soluciones aparece un fichero llamado "App.config". Este fichero es similar a los que utiliza el mundo de ASP.NET. Se trata de un fichero XML de configuración en el que podemos definir parámetros de configuración para la aplicación.

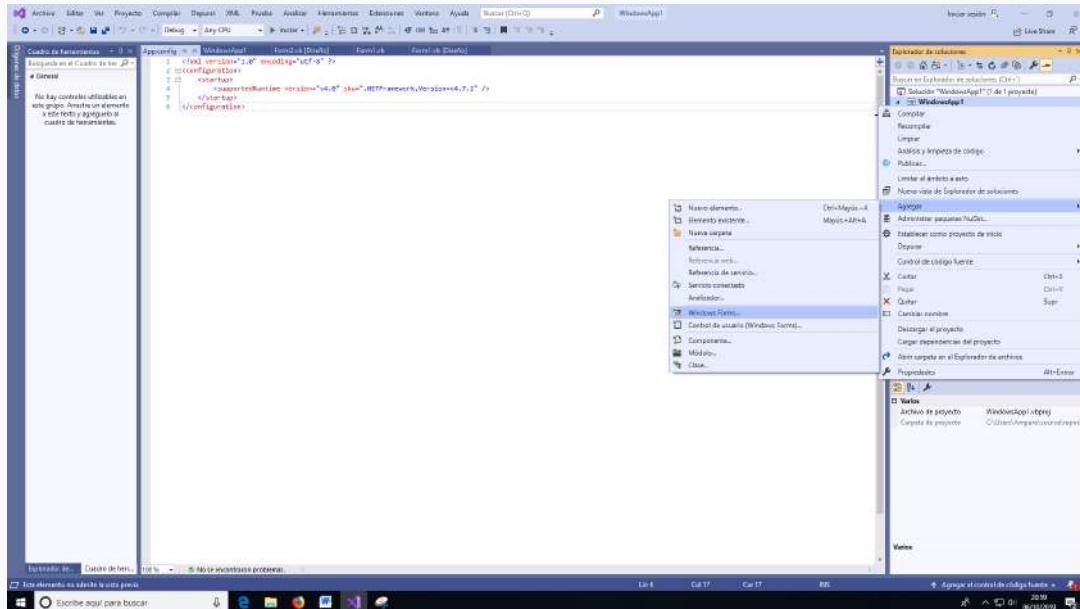
Por ejemplo, cuando definamos una conexión a una base de datos, tendremos que definir una cadena de conexión donde le digamos el servidor y usuario de conexión. En condiciones normales esto lo tendríamos que definir dentro de los formularios. Con este fichero se pretende no escribir ninguna configuración dentro del código C#. Así que definiremos dentro de él los parámetros generales del programa.

Utilizaremos este fichero cuando tratemos con bases de datos.

3.1.6. Añadir más elementos a un proyecto

A lo largo de nuestro proyecto tendremos que añadir más elementos, en su mayoría nuevos formularios. Para esto elegiremos la opción "Agregar Windows Forms" o "Agregar nuevo elemento" del menú Proyecto.

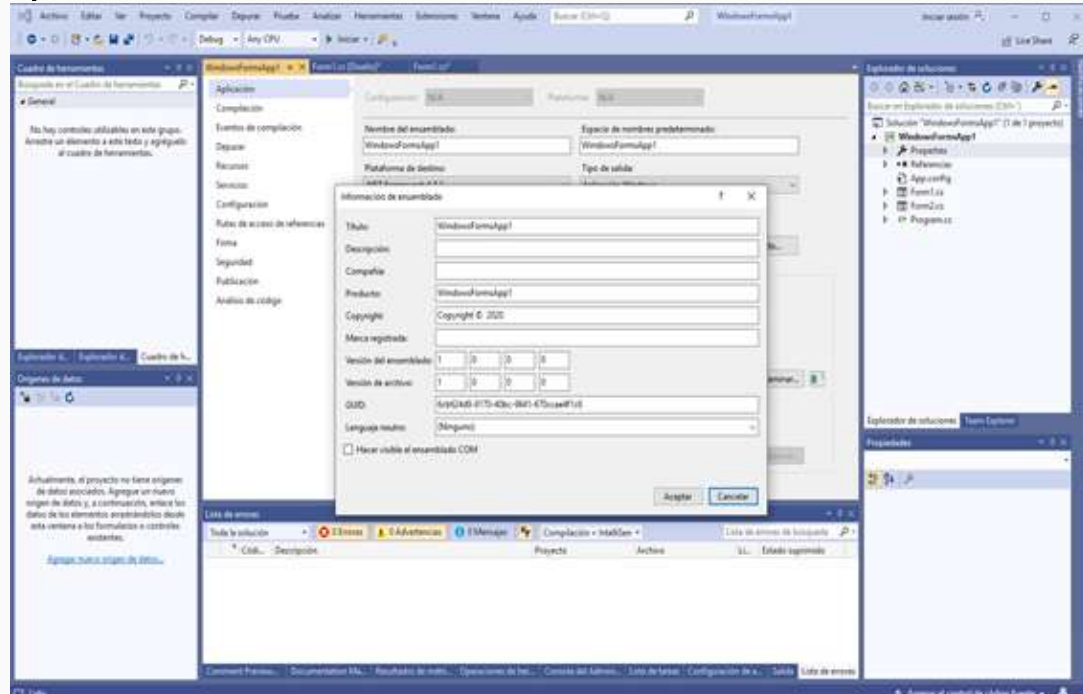
Estas operaciones también se pueden realizar con el botón derecho sobre el nombre del proyecto en el Explorador de soluciones.



3.1.7. Propiedades de la solución y del proyecto

En algunas ocasiones necesitaremos cambiar propiedades del proyecto o de la solución. Para acceder a esta ventana pulsaremos con el botón derecho en la ventana del explorador de soluciones y luego en el título del proyecto o en el título de la solución:

Aplicación



Como podemos ver, podemos indicarle el nombre del programa (ensamblado), el espacio de nombres, tipo de aplicación e icono. En la pantalla destacada "Información de ensamblado" podemos ver detalles que luego podrá ver el usuario como la versión o autor.

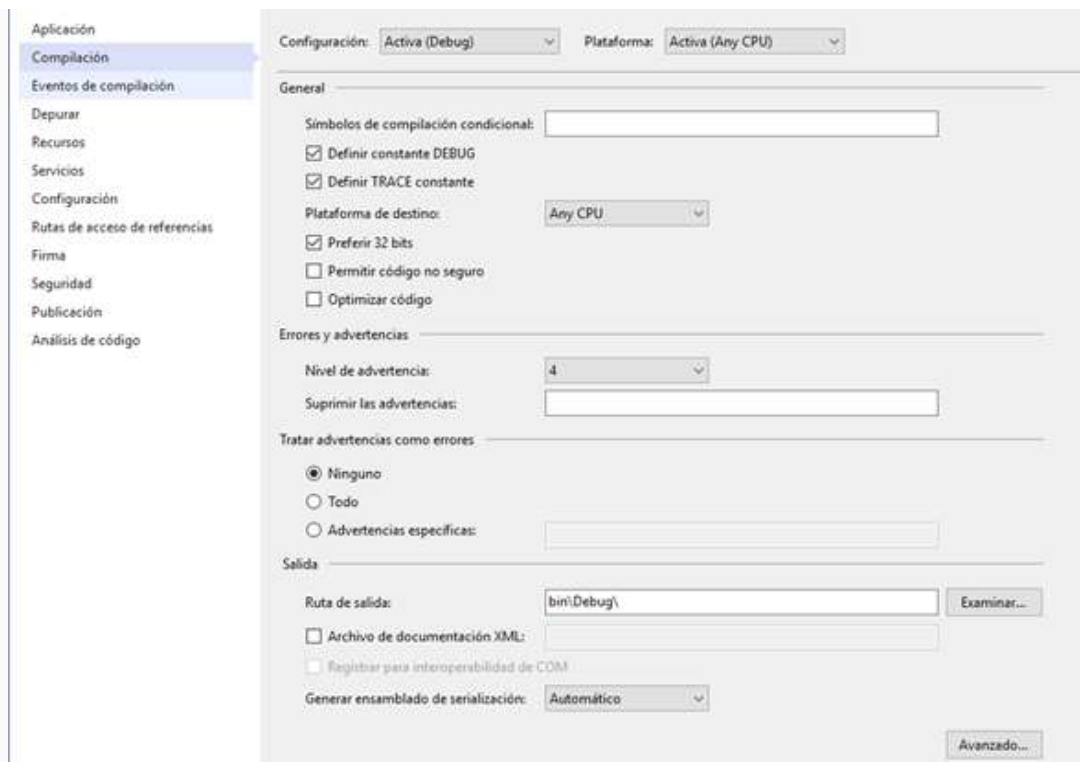
"Objeto de inicio" define el punto de entrada que se llamará cuando se cargue la aplicación. Normalmente, esto se establece en el formulario principal de la aplicación o en el procedimiento Main que debe ejecutarse cuando se inicia la aplicación. Dado que las bibliotecas de clases no tienen un punto de entrada, su única opción para esta propiedad es **(Sin establecer)**.

```
// De esta forma se selecciona el formulario inicial
```

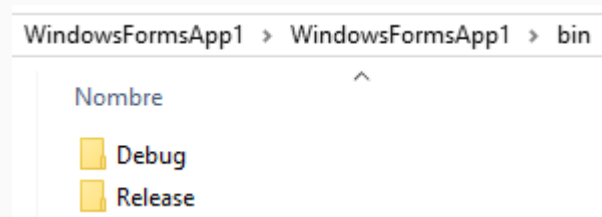
```
Application.Run(new Form1());
```

Compilación

En la sección "Compilación" podemos ver estas opciones:



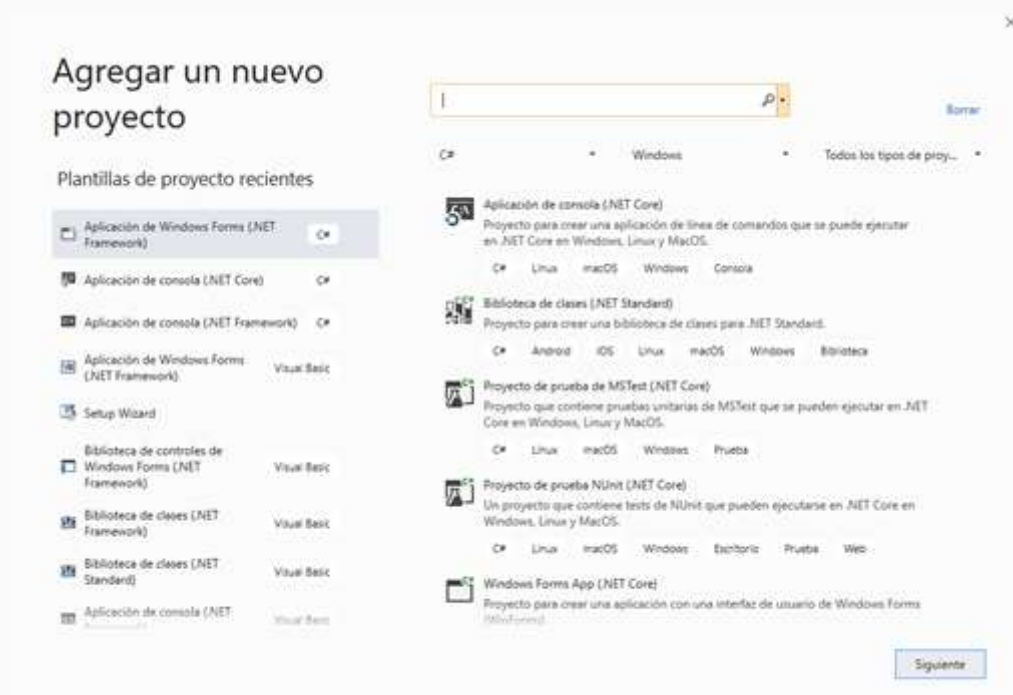
Es la forma en que el compilador generará nuestro programa. La carpeta que aparece en primer lugar, es la carpeta donde nos va a generar la aplicación:



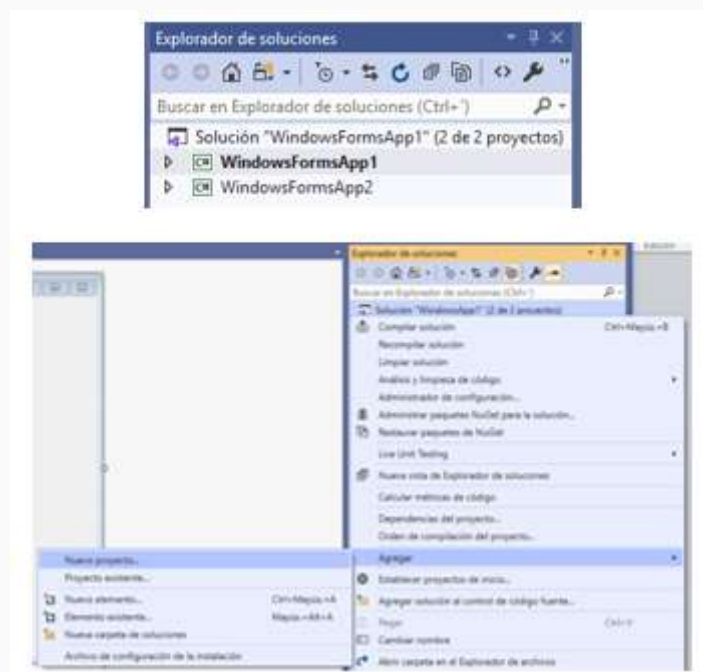
Es decir, la versión final estará en la carpeta "Release" y la versión provisional de depuración, estará en la carpeta "Debug".

3.1.8. Añadir proyectos a una solución

Antes vimos cómo añadir elementos como formularios y otros a un proyecto. Ahora lo que quiero es añadir otro proyecto a mi solución. Tendremos que guardar el proyecto actual y en el menú *Archivo*, opción *Agregar à Nuevo Proyecto*. En este caso la pantalla será como la inicial:



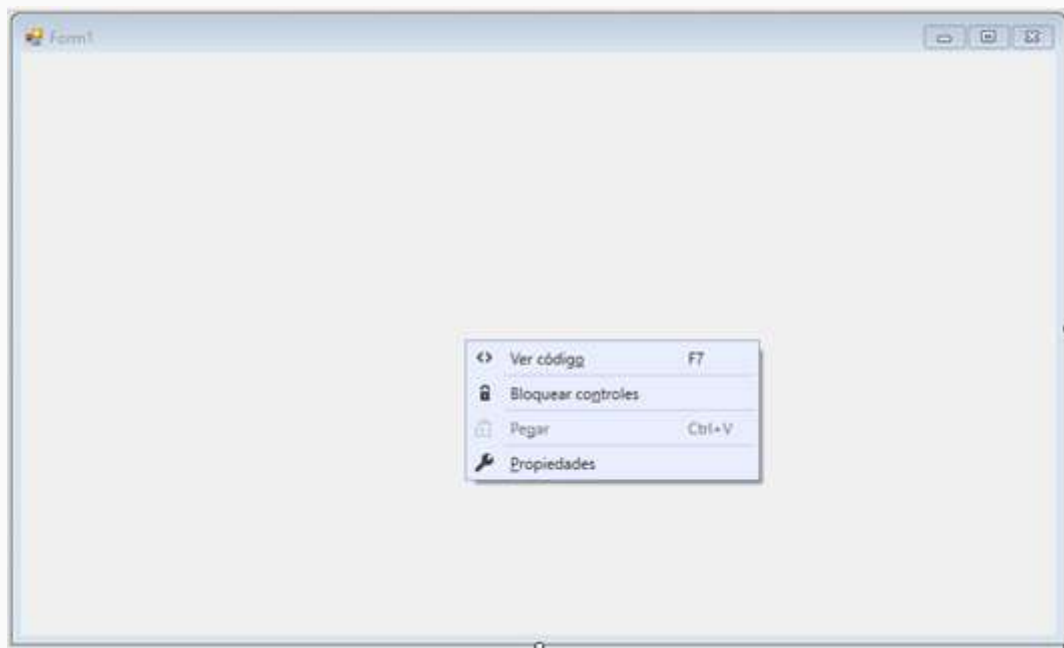
El resultado es el siguiente:



También podemos llegar al mismo resultado del siguiente modo: utilizamos el botón derecho del ratón en el título de la solución del Explorador de soluciones. à *Agregar à Nuevo Proyecto* o *Proyecto Existente*

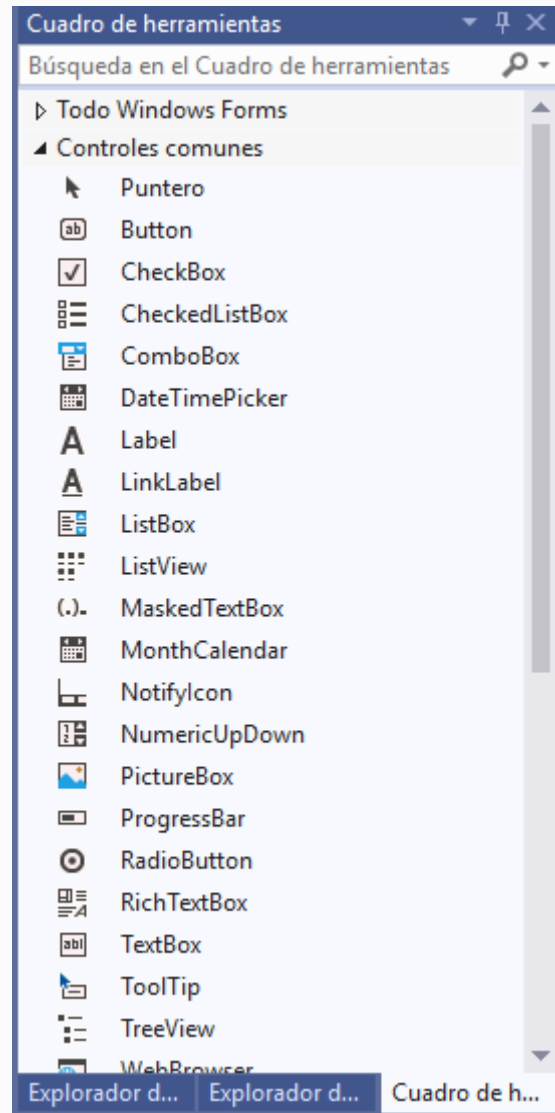
3.1.9. Menú contextual

Cuando pulsamos el botón derecho del ratón se despliega un menú que varía según en la situación en la que estemos: ventana de trabajo, explorador. A este menú se le llama contextual porque depende del contexto en el que nos encontremos. Por ejemplo, en el diseño tendremos este menú:



3.1.10. Insertar controles

Los **controles** son los elementos que insertamos dentro de un formulario y que nos va a permitir interactuar entre el usuario y el código. Controles son: botones, cuadros de texto, etiquetas, cuadros desplegables, cuadrículas de datos,... En definitiva, todos y cada uno de los elementos que vemos en los formularios de todas las aplicaciones. La lista de controles básicos disponibles la tenemos a la izquierda.



Para añadir controles al formulario utilizaremos esta barra. Por ejemplo, para añadir una etiqueta (Label) y una caja de texto (TextBox), simplemente haremos doble-clic sobre esos elementos de la barra de herramientas y se añadirán al formulario.

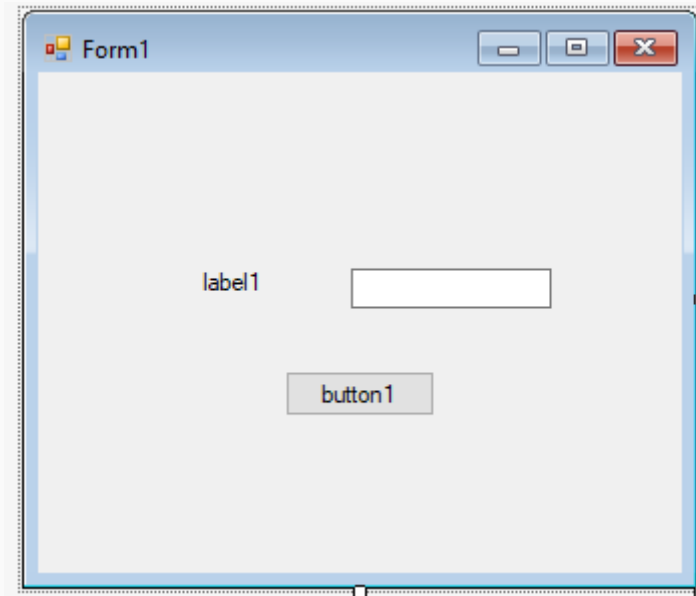
Tenemos varias formas de añadir estos controles:

- Haciendo doble clic sobre el icono del control. Se colocará un control en la parte izquierda superior del formulario.
- Haciendo clic sobre el icono del control y después clic sobre el formulario. El control se insertará en el punto que hemos indicado y se extenderá hacia la derecha y abajo según su tamaño predeterminado.
- La tercera forma es la más utilizada. Hacemos clic sobre el control y luego

en el formulario. Pero esta vez sin soltar el botón izquierdo, vamos dibujando con el ratón la posición y tamaño del control.

- Arrastrando desde la barra de controles.

Para nuestro ejemplo añadiremos un botón (Button) y lo situaremos debajo del cuadro de texto (Textbox). Luego añadimos una etiqueta (Label) para que quede de esta forma:



Como vemos por defecto el IDE pone unos nombres genéricos a los controles: label1, textbox1, button1, es decir, utiliza el tipo de control y lo va numerando: label1, label2, label3,... Todos los controles deben tener un nombre único, así que hasta que se los asignemos, el IDE le pone este genérico. En nuestros proyectos cambiaremos esos nombres de controles por otros nombres más descriptivos, por ejemplo: txtNombre, txtApellidos, txtTelefono en lugar de los puestos por el IDE: textbox1, textbox2, textbox3,...

En cuanto a la convención de nomenclatura en C# seguiremos esta tabla:

C# Coding Standards and Naming Conventions

Object Name	Notation	Length	Plural	Prefix	Suffix	Abbreviation	Char Mask	Underscores
Class name	PascalCase	128	No	No	Yes	No	[A-z][0-9]	No
Constructor name	PascalCase	128	No	No	Yes	No	[A-z][0-9]	No
Method name	PascalCase	128	Yes	No	No	No	[A-z][0-9]	No
Method arguments	camelCase	128	Yes	No	No	Yes	[A-z][0-9]	No
Local variables	camelCase	50	Yes	No	No	Yes	[A-z][0-9]	No
Constants name	PascalCase	50	No	No	No	No	[A-z][0-9]	No
Field name	camelCase	50	Yes	No	No	Yes	[A-z][0-9]	Yes
Properties name	PascalCase	50	Yes	No	No	Yes	[A-z][0-9]	No
Delegate name	PascalCase	128	No	No	Yes	Yes	[A-z]	No
Enum type name	PascalCase	128	Yes	No	No	No	[A-z]	No

Para más información y ejemplos:

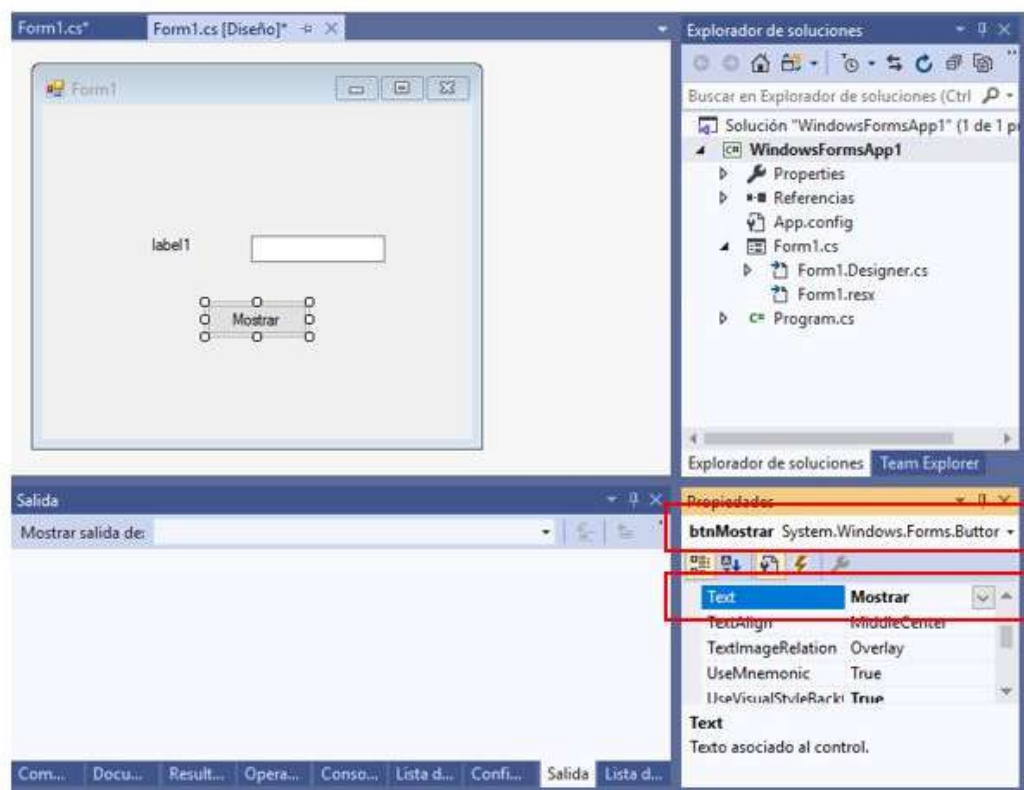
<https://github.com/ktaranov/naming-convention/blob/master/C%23%20Coding%20Standards%20and%20Naming%20Conventions.md>

En nuestro caso, utilizaremos un prefijo para cada uno de los controles más su nombre. El prefijo viene indicado en la siguiente tabla. En el ejemplo: *lblNombre*, *txtNombre*, *btnMostrar*.

Control	Prefijo
Label	lbl
TextBox	txt
DataGrid	dtg
Button	btn
ImageButton	imb
Hyperlink	hlk
DropDownList	ddl
ListBox	lst
DataList	dtl
Repeater	rep
Checkbox	chk
CheckBoxList	cbl
RadioButton	rdo
RadioButtonList	rbl

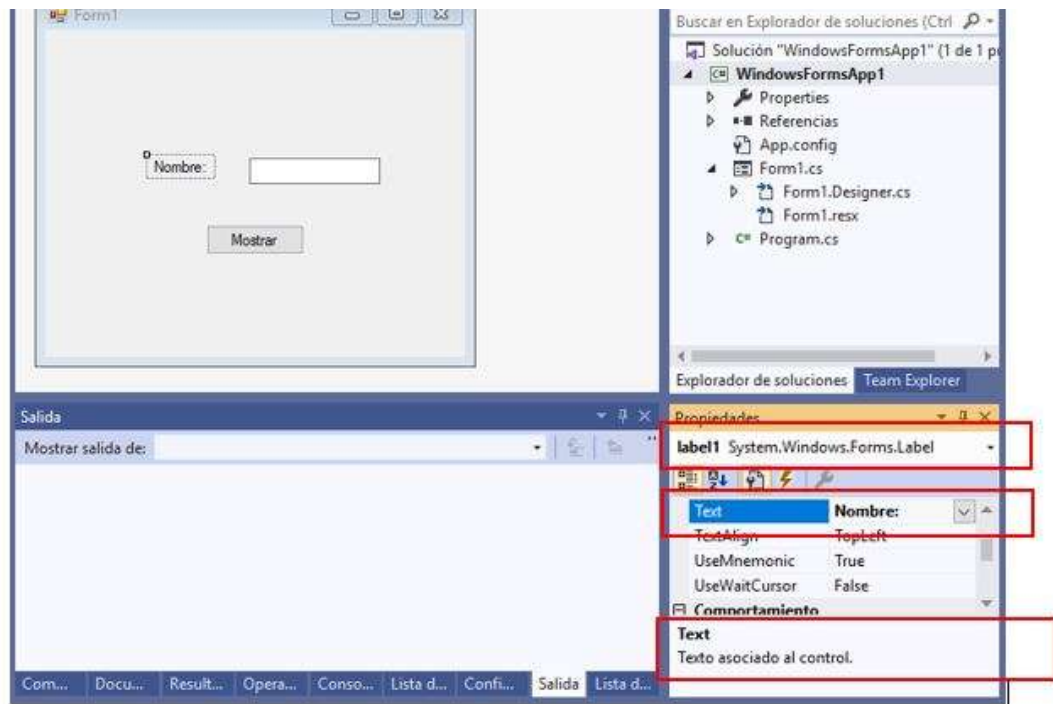
Image	img
Panel	pnl
Placeholder	phd
Table	tbl
Validators	val

Ahora vamos a cambiar el texto que contiene el botón "btnMostrar". Para cambiarle este texto hay que utilizar la ventana de propiedades, en esta ocasión el elemento que nos interesa de esa ventana de propiedades es **Text**, escribimos en esta propiedad la palabra "Mostrar" y cuando pulses Intro o el tabulador veremos que el texto del botón se ha actualizado:

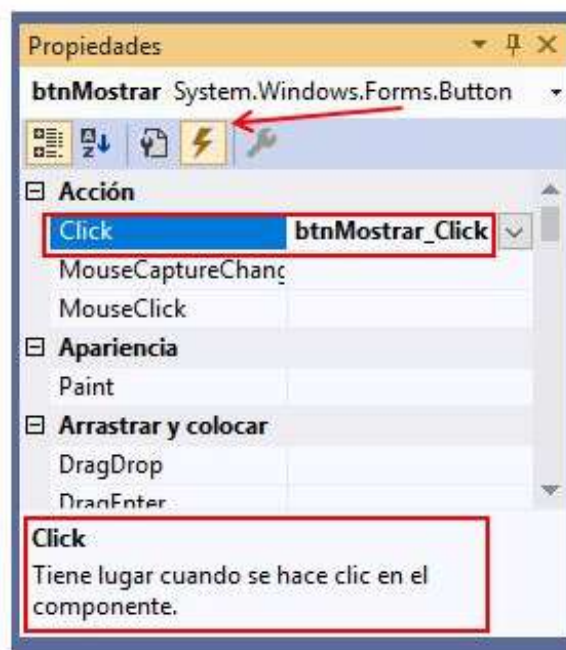


Hacemos lo mismo con la etiqueta. Recuerda que hay que seleccionarla primero haciendo clic para que se muestren las propiedades de la etiqueta, escribimos **"Nombre:"** y pulsamos intro o el tabulador:





Debemos fijarnos siempre en la parte superior de la ventana para comprobar que pone el nombre del control que estamos editando y de qué tipo es:



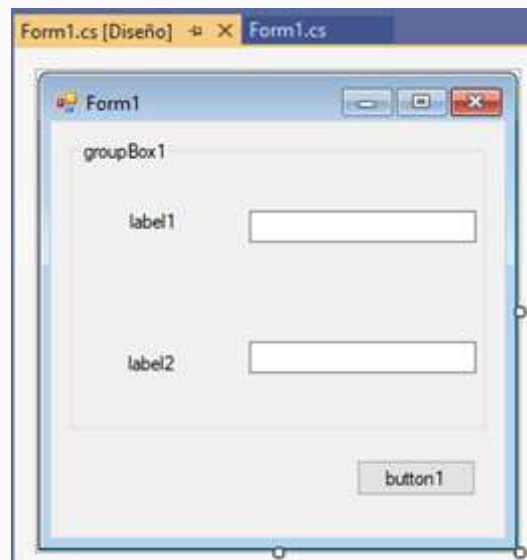
En este caso es una etiqueta (label) que la ha creado a partir de la clase "System.Windows.Forms.Label"

Al principio y para asegurarnos de que vamos escribiendo bien estas propiedades nos fijaremos bien en que tenemos seleccionado el control adecuado. En la pantalla anterior podemos ver en el título superior el nombre del control: 'lblNombre' así sabemos que estamos escribiendo la propiedad en el control adecuado.

3.1.11. Diseñador del formulario

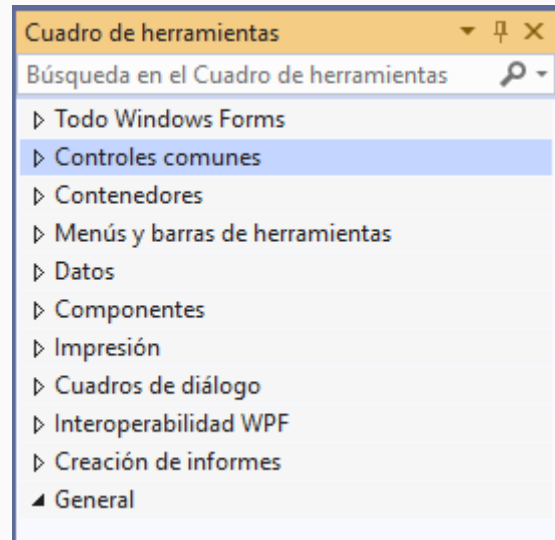
Esta es una de las áreas más entretenidas y difíciles de manejar. Entretenida porque es la que vamos a utilizar para dibujar nuestras pantallas: seleccionar iconos, controles y otros elementos para construir las pantallas con las que interactuarán los usuarios y difícil porque es difícil diseñar una buena interfaz.

En esta pantalla colocaremos pues los controles que vayamos a necesitar en nuestra interfaz pero en ocasiones utilizaremos controles que no ofrecen una funcionalidad con el usuario pero que necesito en el proyecto: estos controles son ocultos para nuestro proyecto. Por ejemplo un menú, un temporizador, un cuadro de diálogo... los tendremos disponibles para utilizar pero no ofrecen interfaz con el usuario:



3.1.12. Cuadro de herramientas

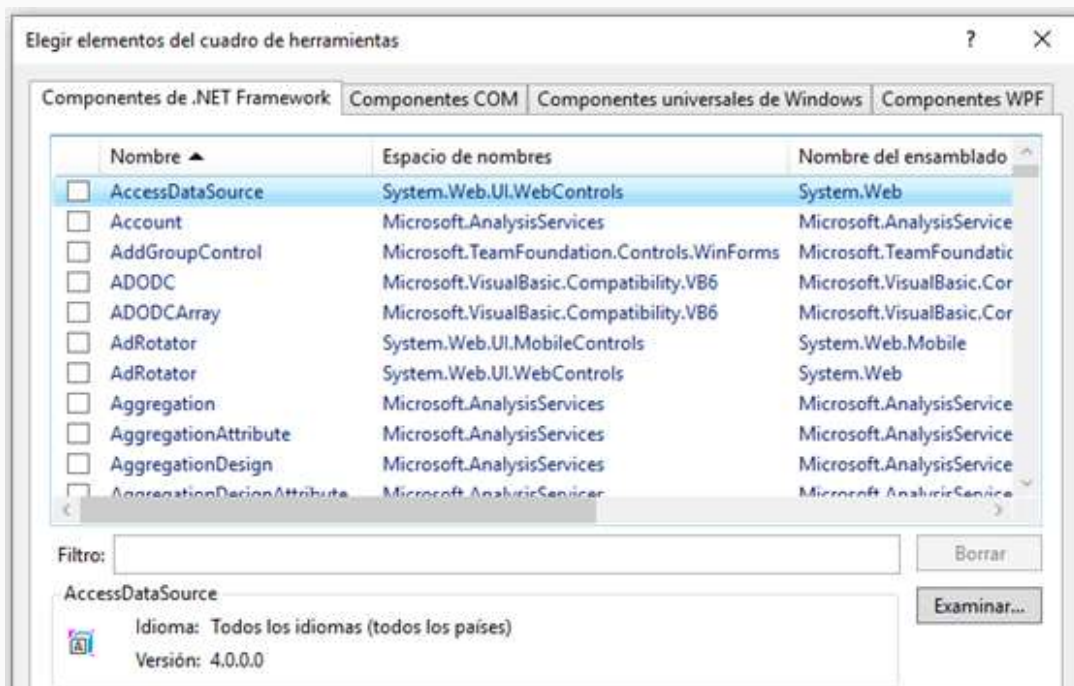
El cuadro de herramientas contiene todos los elementos que podremos incorporar a nuestros proyectos. Como son muchos está dividido en fichas similares a otros programas de Office:



Por defecto, tendremos formularios Windows que son los controles para los formularios. Además de estas fichas tenemos otras disponibles pero que no están visibles. Para que sean visibles pulsaremos con el botón derecho y seleccionamos "Mostrar todo".

Esta pantalla es muy flexible y permite mover de sitio las pestañas simplemente pulsando en el título y arrastrándolo a la posición que queramos.

Por si fuera poco podemos crear nuevas pestañas. Sólo tenemos que seleccionar la opción "Agregar pestaña" que aparece al pulsar con el botón derecho encima de una pestaña. Luego le asignamos un nombre. A continuación pulsamos con el botón derecho sobre ella y seleccionamos "Elegir elementos". Aparecerá la siguiente pantalla que nos permitirá añadir los elementos que queramos.



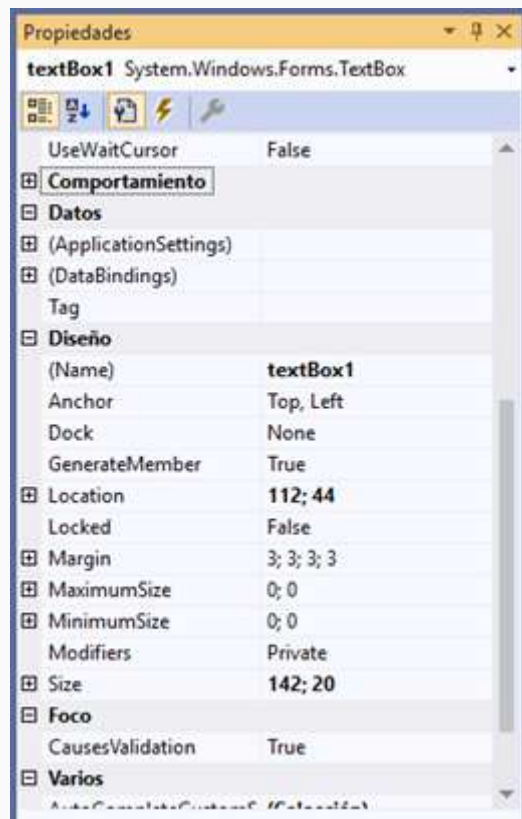
Aceptar

Cancelar

Restablecer

3.1.13. Ventana propiedades

Esta ventana es junto con las dos anteriores las que más vamos utilizar en el diseño ya que aquí es donde daremos nombre a los controles, modificaremos su aspecto y configuraremos su funcionamiento.

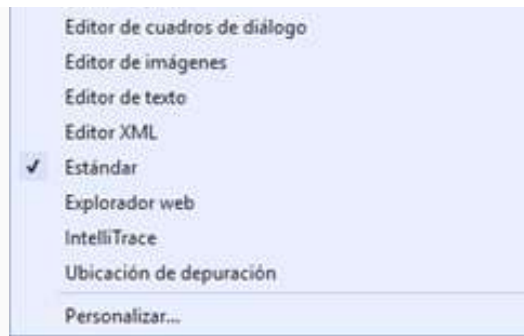


Se divide en varias partes. En la parte superior tenemos el nombre del objeto o control: textBox1 y a la derecha le indica de qué tipo es o su clase. Puedes ver que aparece otra vez el namespace "System.Windows.Forms". Así que ya sabemos que los cuadros de texto como este dependen de este digamos superobjeto que es el que tiene todos los controles de Windows.

A continuación tiene una barra de herramientas que nos permite distribuir las propiedades (en ocasiones serán muchas) en una jerarquía, según el tipo de propiedad que es o en orden alfabético.

Para introducir datos según el tipo de propiedad lo podremos hacer de una forma u otra: un menú desplegable, una selección de fichero, una pantalla adicional con editor de texto, un cuadro de diálogo de selección de color...





Por último, comentar que en la parte donde tenemos el nombre del control en realidad es un cuadro desplegable que contiene todos los controles de nuestro formulario. En ocasiones el control que queremos modificar puede no estar visible, con esta opción podemos buscarlo en la lista para cambiar alguna propiedad.

3.1.14. Barras de herramientas

En Visual Studio disponemos de multitud de barras de herramientas. Su activación y manejo es igual que el resto de las aplicaciones de Office: bien con el botón derecho en una barra de herramientas o seleccionando el menú Ver y luego Barras de Herramientas

Con la última opción, Personalizar, podemos crear y modificar barras de herramientas para poner las opciones que nos interesen. En general veremos las más utilizadas o las que nos hagan falta según lo que estemos manejando.

Igual que en las ventanas anteriores las barras de herramientas también se pueden dejar acopladas o flotantes. Para esto haz clic y arrastra la barra utilizando la barra vertical que tienen justo a la izquierda.

3.1.15. La vista de clases

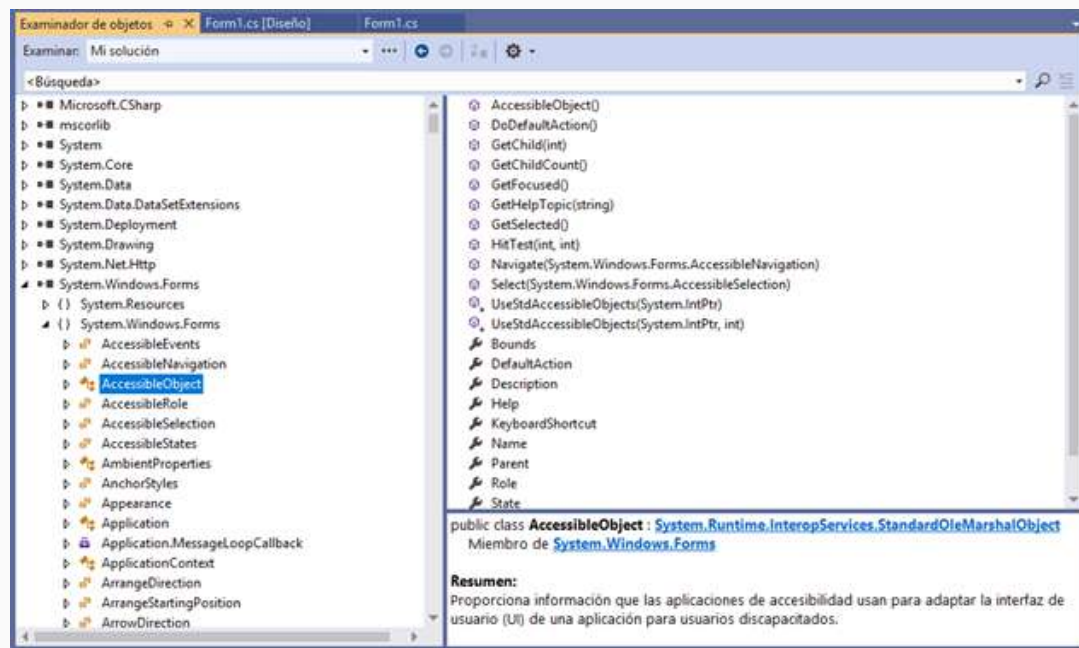
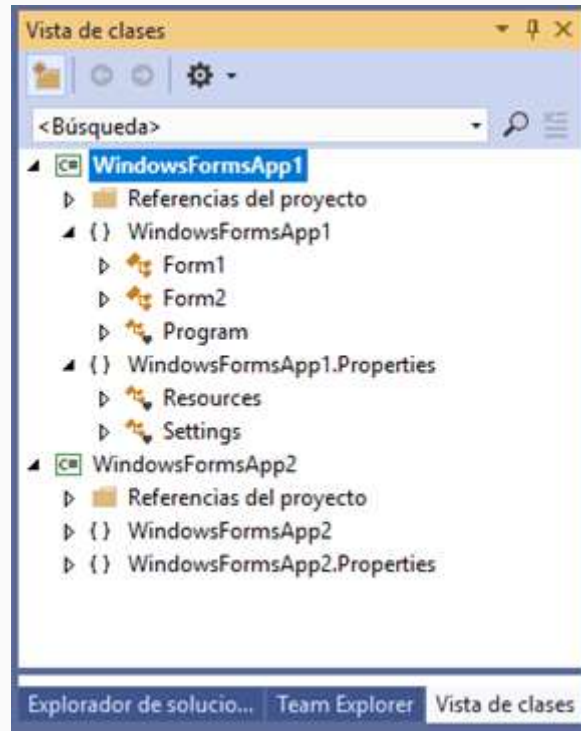
En un lenguaje de programación necesitaremos una herramienta que nos permita ver las clases de un objeto. Para activarla podemos utilizar la opción *Vista de clases* del menú *Ver*:

Está organizada en forma jerárquica, esto es, un nivel con el objeto o elemento más importante y con él un símbolo " " para desplegar el siguiente nivel, luego si tienen más subniveles podrás ir navegando por todos los elementos de la clase.

En el ejemplo de la derecha fijaros que está el nombre de nuestro programa y después dos formularios, que son justo los elementos que tiene el programa. Si expandimos el formulario vemos que están los procedimientos que aparecen los Tipos base de Form.

3.1.16. Explorador de objetos

Esta ventana nos será en muchas ocasiones bastante útil para poder explorar las capacidades de cada objeto. Para mostrarla utilizaremos el menú Ver y luego en "Examinador de objetos":

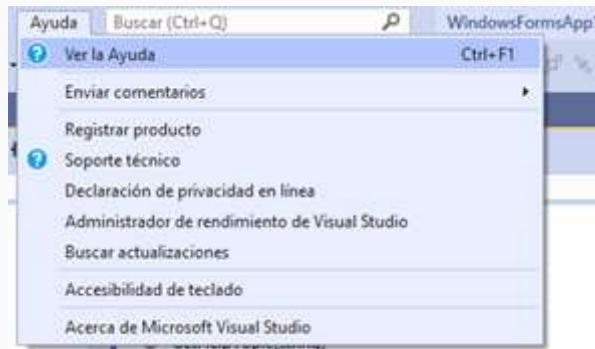


Habrás podido comprobar que se ha acoplado en la ventana principal y se ha añadido como una solapa más.

Así como en el caso anterior podíamos navegar a través de los objetos de nuestro programa, en esta pantalla podemos navegar por todos los objetos de VS.NET. Otra vez aparece el famoso objeto (o mejor dicho espacio de nombres o spacenames) llamado "System.Windows.Forms". En el panel de la derecha muestra la clase o elemento seleccionado y en la parte inferior la declaración.

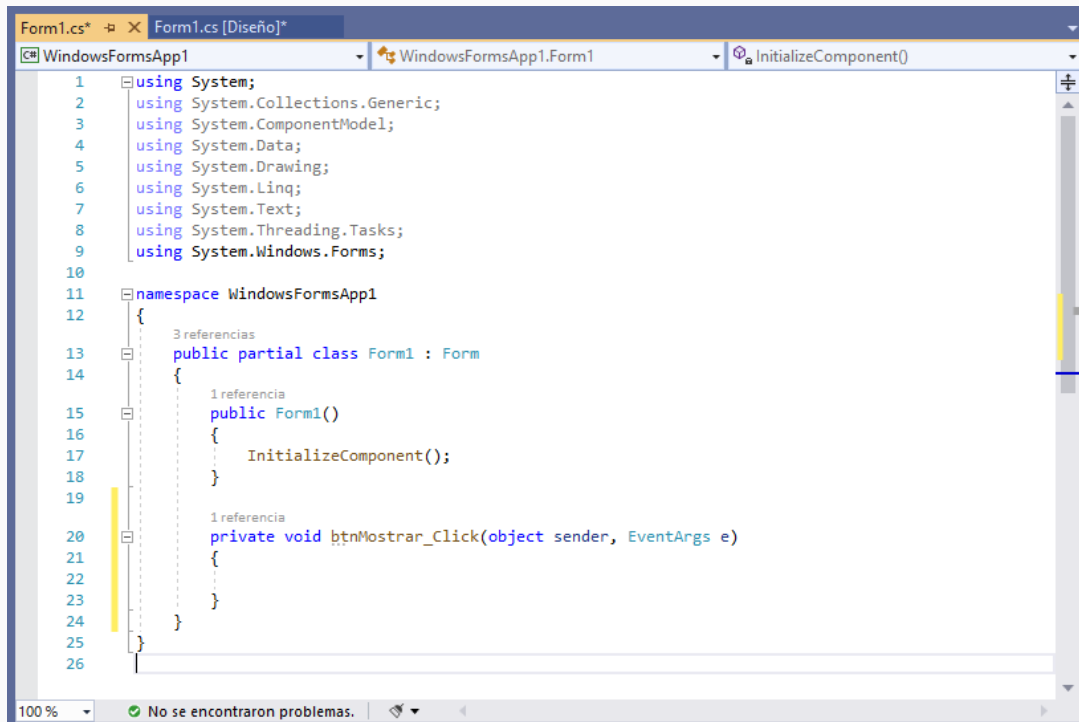
3.1.17. La ayuda

Y por fin otro de los elementos más utilizados en el IDE: la ayuda. Si hay una cosa en que Microsoft tenga especial cuidado es en la ayuda y su traducción. En este caso ha hecho un trabajo magnífico en cuanto a volumen y adaptación a nuestro idioma. Tenemos muchas formas de obtener ayuda.

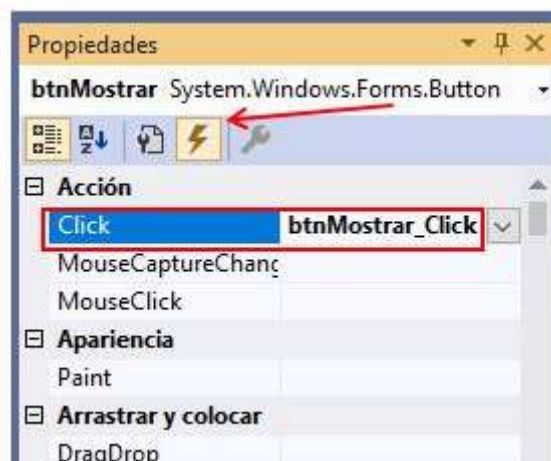


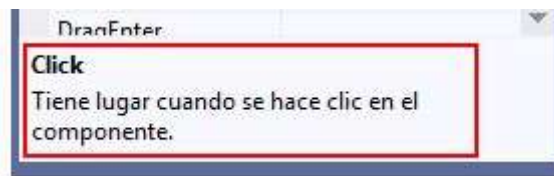
3.1.18. Codificar

Siguiendo con el ejemplo anterior, vamos a escribir código para que se ejecute cada vez que se haga clic en el botón que hemos añadido. Para ello, selecciona el botón Mostrar y hacemos doble clic en él, se mostrará una nueva ventana. En este caso será la ventana de código asociada con el formulario que tenemos en nuestro proyecto. Nos mostrará:

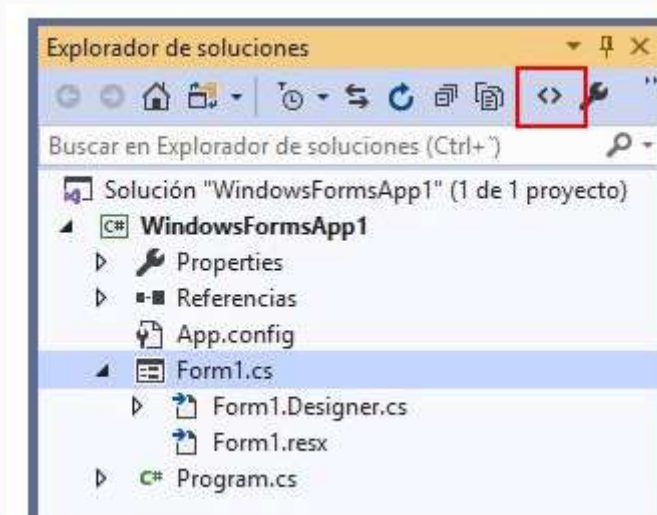


Aquí empieza la programación. Que nosotros pulsemos doble clic en el botón y que aparezca un fragmento de código significa lo siguiente: C# interpreta que quieres poner código para realizar alguna acción cuando se haga clic sobre el botón. Por lo tanto, te muestra ya el "procedimiento" o parte de código que se va a ejecutar cuando suceda esto. A esta forma de trabajar se le llama **programación orientada a eventos**. Es decir, cuando se produzca el evento de pulsar el botón (clic) ejecutas este código. Por lo tanto, vemos que los controles además de tener propiedades (que modifican su aspecto) también atienden a una serie de eventos (clic, doble clic). Los eventos a los que atienden los controles los podemos ver en la ventana de propiedades, seleccionando:

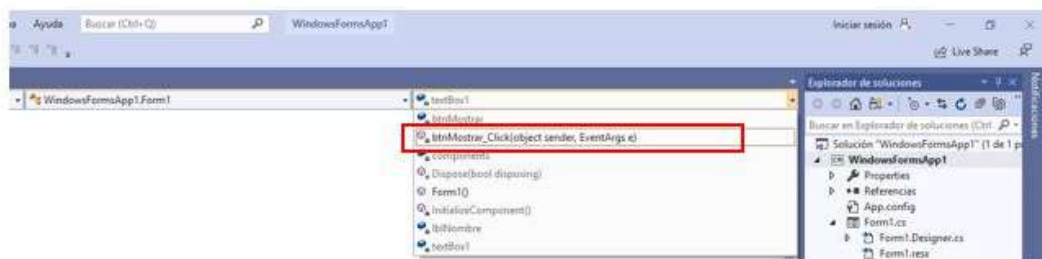




Hay otra forma de acceder a los eventos de los controles y es ésta: nos vamos a la vista del código del formulario pulsando en:

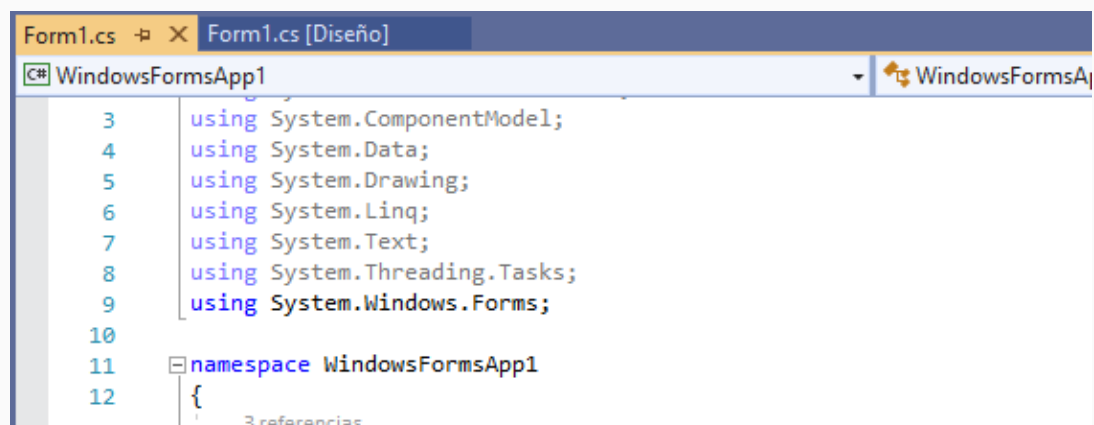


Llegaremos otra vez a la ventana del código de antes. Hacemos clic en el desplegable de arriba a la derecha y veremos la lista con los objetos que hemos añadido en nuestro formulario (botón, etiqueta y cuadro de texto) junto con todos eventos asignados a los mismos:



Esta lista tiene todos los elementos del código C# del archivo Form1.cs. Por tanto, cuando necesitemos acceder al código de un determinado elemento, lo seleccionaremos en la lista y se nos mostrará en el archivo **Form1.cs** o en el archivo de diseño **Form1.Designer.cs**.

Volvamos a la pantalla del código para el evento clic del botón:



```
13 public partial class Form1 : Form
14 {
15     1 referencia
16     public Form1()
17     {
18         InitializeComponent();
19     }
20     1 referencia
21     private void btnMostrar_Click(object sender, EventArgs e)
22     {
23     }
24 }
25
26
```

Esta pantalla nos muestra mucha más información de lo que parece en principio. Nos está indicando en la parte superior que estamos trabajando con "Form1.cs". Debajo nos indica a la derecha que estamos con el evento *click* del control *btnMostrar*, es decir, *btnMostrar_Click*.

Recordamos que lo que queremos hacer ahora, es escribir el código que se ejecutará cuando se haga clic en el botón, lo cual producirá el evento clic asociado con dicho botón. Ese evento se producirá si se hace un clic propiamente dicho, es decir, con el ratón, o bien porque se pulse intro o la barra espaciadora cuando el botón tenga el [enfoque](#) .

La nomenclatura, (forma de llamar a los elementos), para los eventos de C# sigue el siguiente esquema:

[nombre del control] [guión bajo] [nombre del evento]btnMostrar_Click '

y se ejecuta el código cuando se hace clic en el elemento btnMostrar.

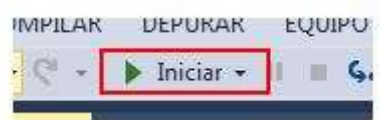
a. Ejemplo Hello World!

Vamos a hacer que se muestre un cuadro de diálogo mostrándonos el nombre que previamente hemos escrito en el cuadro de texto para recrear el clásico ejemplo de programación de saludar, en este caso, al usuario que se haya introducido en el cuadro de texto y cuando se haya pulsado el botón "Mostrar".

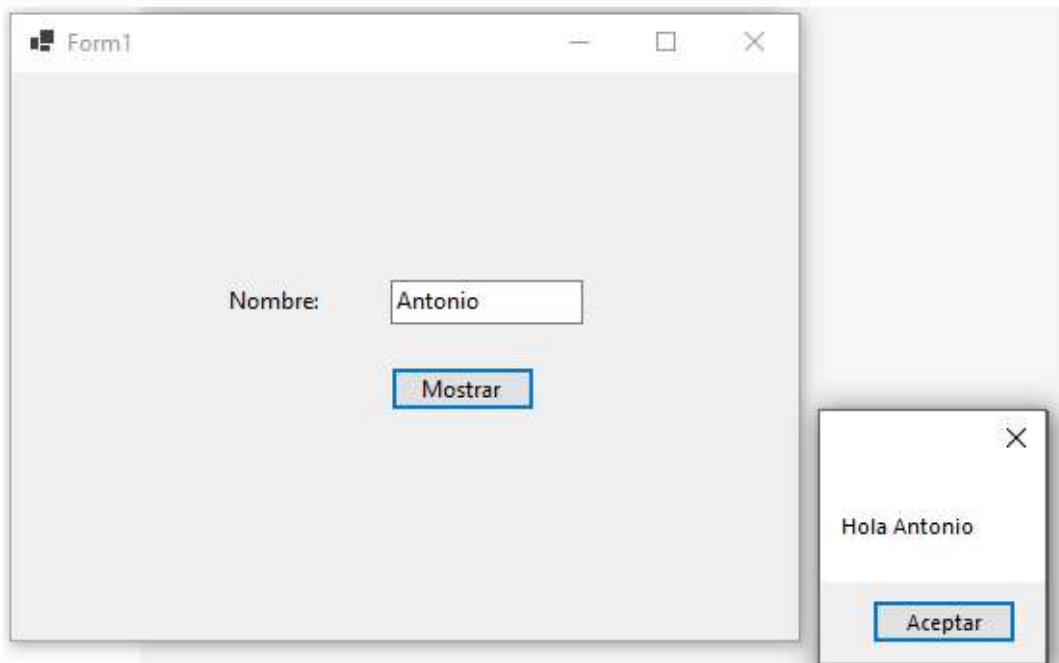
Escribimos el siguiente código en el hueco dejado por C# entre las líneas que hay entre private void... { y }

```
private void btnMostrar_Click(object sender, EventArgs e)
{
    MessageBox.Show("Hola " + txtNombre.Text);
}
```

Para poder ver el resultado, necesitamos compilar y ejecutar el proyecto. Para ello, pulsamos F5 para que se ejecute el código que hemos escrito o pulsamos en el botón "play" que está en la barra de botones.



Cuando aparezca el formulario, escribiremos algo en el cuadro de texto y pulsaremos en el botón "Mostrar". Veremos que se muestra un cuadro de diálogo con el texto "Hola" y a continuación lo que hayamos escrito en el cuadro de texto (TextBox):



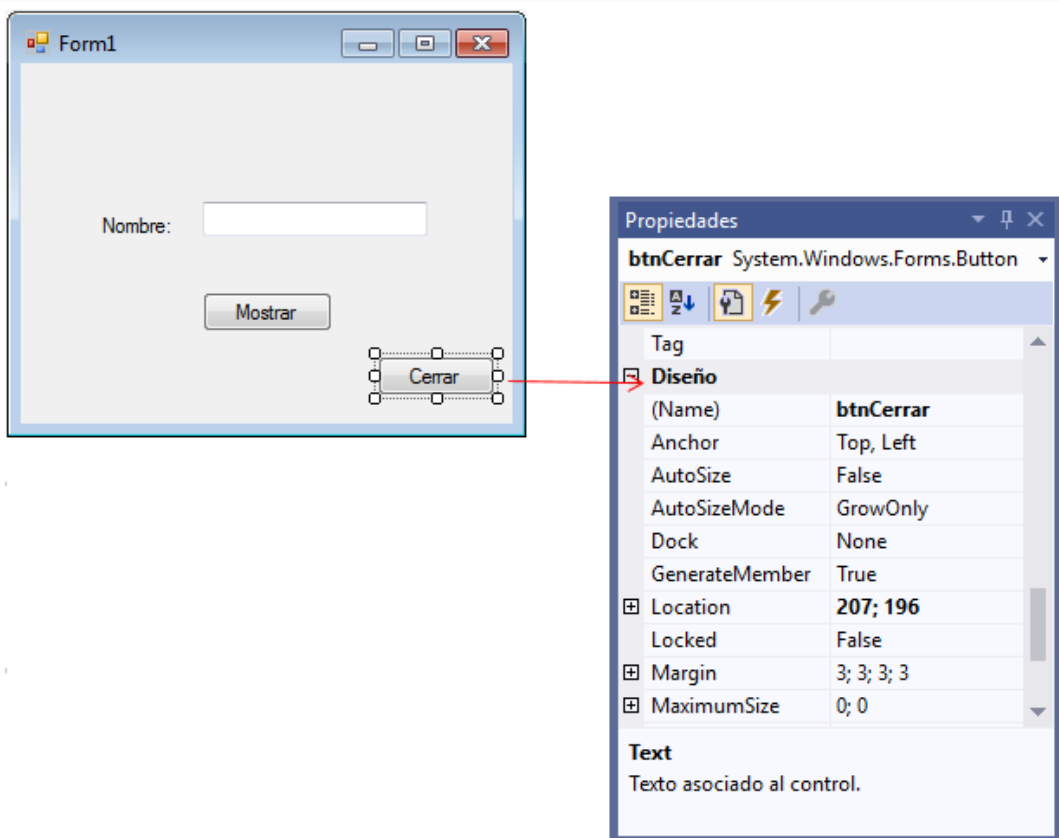
Ya tenemos nuestra primera aplicación Windows funcionando creada con C#.NET. Para terminar pulsamos en la "x" del formulario principal o en el botón de terminar de la barra de tareas.



Ahora vamos a añadir otro botón que llamaremos *btnCerrar* con el texto "Cerrar". (el nombre es por "btn" de botón y Cerrar de la acción: *btnCerrar*). Para esto primero vamos al IDE para añadir otro botón. Ahora cambiaremos dos propiedades: Name para ponerle *btnCerrar* y Text para ponerle Cerrar:

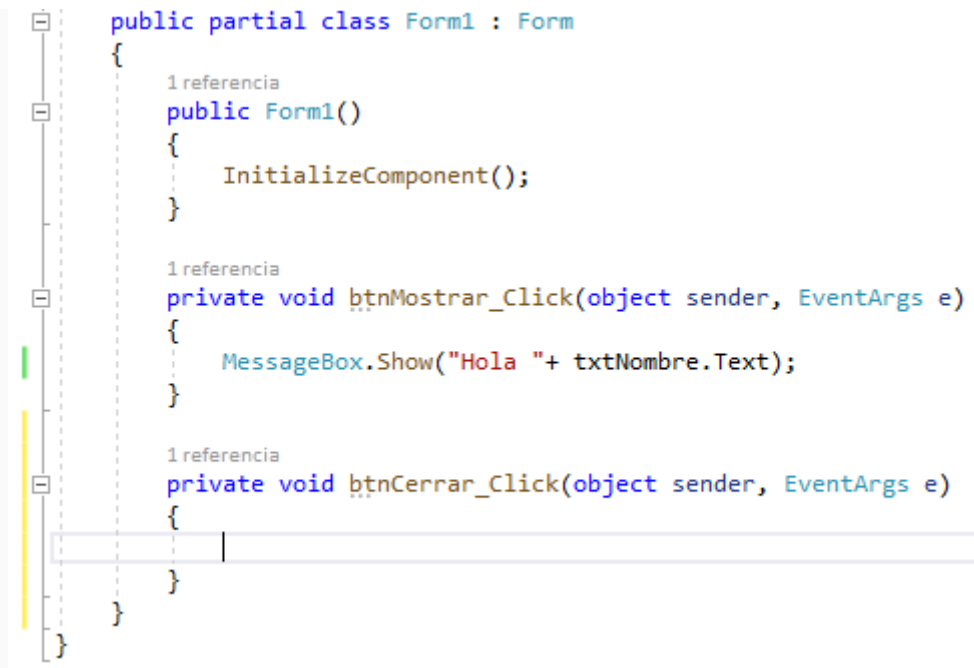
Nota: Si estamos con el editor de código y queremos pasar al diseñador del formulario haremos doble clic en el formulario en la parte derecha en el "Explorador de soluciones".

Seguimos con el ejemplo, pintamos un nuevo botón y cambiamos las dos propiedades comentadas anteriormente:



Obviamente al ponerle esta vez un nombre al control, en lugar de llamarse *button2* se llamará como le hemos indicado *btnCerrar* y así lo veremos en el código. Pulsamos doble clic en él para escribir el código que queremos que ejecute:

```
namespace WindowsFormsApp1
{
    // 3 referencias
```



y escribimos la instrucción

`this.Close()`

Pulsamos F5 para ejecutar el programa y la diferencia respecto al programa anterior es que ahora al pulsar este botón el programa termina, cierra el formulario y nos devuelve el entorno de desarrollo. De esta forma el programa está un poco más completo.

Ahora veamos con detalle el código que hemos usado en los dos eventos (clic del botón Mostrar y clic del botón Cerrar):

`MessageBox("Hola " + txtNombre.Text)`

MessageBox que es una función o método, (realmente es una clase, como casi todo en .NET), cuya tarea es mostrar en un cuadro de diálogo lo que le indiquemos en el primer **parámetro**, también tiene otros parámetros opcionales, pero por ahora usemos sólo el primero que es obligatorio. Tenemos pues por un lado una instrucción MessageBox que escribe un texto en pantalla y que admite parámetros. Estos parámetros le proporcionan los datos de qué debe hacer, en este caso le proporcionamos un texto para que lo escriba: MessageBox ("Hola"). Los parámetros opcionales sirven para añadirle más funcionalidad o simplemente para que modifique su aspecto... En nuestro ejemplo podría tener un título el cuadro de texto o el mensaje podría ir acompañado de un icono.

En C# .NET todos los procedimientos que reciban parámetros deben usarse con los paréntesis. Por tanto, para indicarle que es lo que queremos que muestre, tendremos que hacerlo dentro de los paréntesis. En este caso, queremos que se muestre la palabra **"Hola"** y lo que haya en la caja de texto.

La palabra *Hola* (seguida de un espacio) es una **constante**, es decir, siempre será la palabra "Hola" seguida de un espacio. Una vez que el programa esté compilado (listo para ejecutarse por otros usuarios), siempre será lo mismo, por eso se llama constante porque no cambia. En este caso una constante alfanumérica o de cadena, por eso va entre comillas, ya que todos los *literales* o cadenas que queramos usar en nuestros proyectos deben ir dentro de comillas

dobles.

Por otro lado **txtNombre.Text** representa el texto que hay en el cuadro de texto y, por tanto, devolverá lo que en él hayamos escrito. Estamos haciendo referencia a la propiedad **Text** de un control llamado **txtNombre**. En nuestro ejemplo pintamos un cuadro de texto y C# .NET le puso el nombre genérico de **textBox1**, si modificamos su propiedad **Text** (recordemos el botón **Cerrar** o la etiqueta por ejemplo) cambiará el contenido de texto que contiene.

Por último, para que esas dos cadenas de caracteres, la constante **Hola** y el contenido de la propiedad **Text** del control **TextBox1**, se puedan unir para usarla como una sola cadena, usaremos el signo **+** (más) que sirve para concatenar cadenas de caracteres y hacer que C# entienda que es una sola.

Por tanto, si la propiedad **Text** del control **txtNombre** contenía la cadena **Nombre**, al unir las dos cadenas resultará una nueva con el siguiente contenido: **"Hola Nombre"** que no es ni más ni menos que la "suma" de las dos cadenas que teníamos.

En el método del evento **Clic** del botón **Cerrar** hemos escrito: **this.Close()**. **This** representa al objeto o clase **Form1** (el formulario) y el método **Close** lo que hace es cerrar el formulario, igual que cuando pulsamos en el botón **cerrar** del formulario. Otra vez hemos utilizado una propiedad (mejor dicho método) de un elemento, por lo tanto, la sintaxis es ya sencilla: **elemento.acción**

b. Lista de tareas

A medida que escribamos una aplicación nos saldrán pequeños trabajos o tareas que debemos escribir o apuntar y quedarán pendientes. Para esto Visual Studio incluye una sección llamada *Lista de tareas* que permite definir trabajos pendientes de realizar en el código de la aplicación, asignándoles una prioridad.

Podemos verla en Ver → Lista de tareas.

Se usa la **Lista de tareas** para hacer un seguimiento de los comentarios de código que usan tokens como TODO y HACK (o tokens personalizados) y administrar los accesos directos que llevarán directamente a una ubicación predefinida del código. Haremos clic en el elemento de la lista para ir a su ubicación en el código fuente.

Cuando la ventana **Lista de tareas** está abierta, aparece en la parte inferior de la ventana de la aplicación.

Para cambiar el criterio de ordenación de la lista, seleccionamos el encabezado de cualquier columna. Para refinar los resultados de la búsqueda, presiona la tecla **Mayúsculas** y elige un segundo encabezado de columna. Como alternativa, elige **Ordenar por** en el menú contextual y elige un encabezado. Para refinar los resultados de la búsqueda, presiona la tecla **Mayúsculas** y elige un segundo encabezado de columna.

Para mostrar u ocultar columnas, elige **Mostrar columnas** en el menú contextual. Selecciona las columnas que quieres mostrar u ocultar.

Para cambiar el orden de las columnas, arrastra cualquier encabezado de columna a la ubicación que quieras.

También aparece un comentario en el código precedido de un marcador de comentario y un token predefinido en la ventana **Lista de tareas**. Por ejemplo, el siguiente comentario de C# tiene tres partes distintas:

- El marcador de comentario (//)
- El token, por ejemplo (TODO)
- El comentario (el resto del texto)

C#

```
// TODO: Load state from previously suspended application
```

Dado que TODO es un token predefinido, este comentario aparece como una tarea TODO en la lista.

Los tokens predeterminados solo están disponibles para los lenguajes C/C++, C# y VB.

De manera predeterminada, Visual Studio incluye los tokens siguientes: HACK, TODO, UNDONE y UnresolvedMergeConflict. No distinguen mayúsculas de minúsculas. También se pueden crear tokens propios personalizados.

Para crear un token personalizado:

1. En el menú **Herramientas à Opciones**.
2. Abre la carpeta **Entorno** y, a continuación, elige **Lista de tareas**.
3. En el cuadro de texto **Nombre**, escribe el nombre del token, por ejemplo, **BUG**.
4. En la lista desplegable **Prioridad**, elige una prioridad predeterminada para el nuevo token.
5. Haz clic en **Agregar**.

El botón **Agregar** se habilita una vez que especifica un nombre. Debes escribir un nombre antes de hacer clic en **Agregar**.

3.2. TRABAJAR CON FORMULARIOS (System.Windows.Forms.Form)

Windows Forms es la nueva plataforma de desarrollo de aplicaciones para Microsoft Windows, basada en .NET Framework. Este marco de trabajo proporciona un conjunto de clases claro orientado a objetos y ampliable, que permite desarrollar complejas aplicaciones para Windows.

Un formulario es una representación de una ventana. La mayoría de los formularios se utilizan para mostrar controles con información que el usuario puede ver o manipular. Un formulario como hemos dicho en otras ocasiones es un objeto con propiedades que definen su aspecto, métodos que definen su comportamiento y eventos que definen su interacción con el usuario.

Para entender todo esto, vamos a seguir practicando con el entorno de desarrollo integrado. Nos vamos a centrar en diferentes aspectos de los formularios como pueden ser sus propiedades. Por ejemplo, vamos a hacer que los controles que tengamos en un formulario se adapten de forma automática al nuevo tamaño de la ventana así como a los distintos tamaños de fuentes.

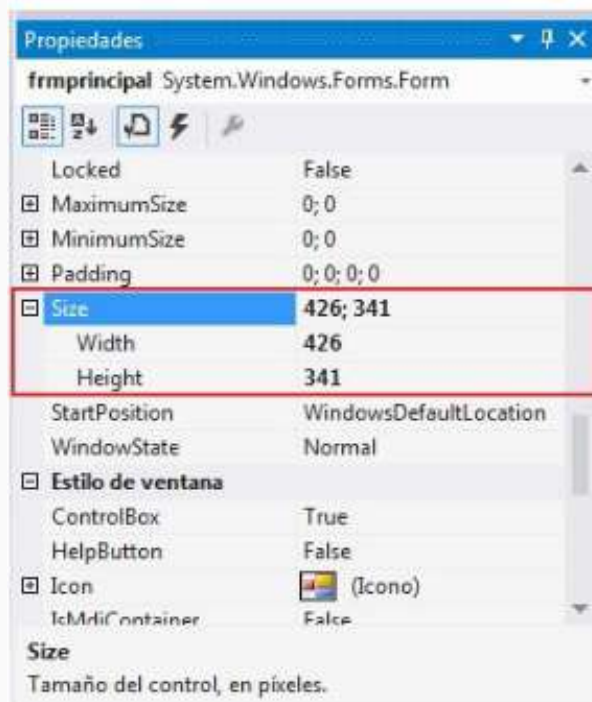
No todo el mundo usa la misma resolución de pantalla que nosotros en la edición del programa ni los mismos tamaños de letras. Ya habréis visto en algunas aplicaciones cómo al maximizar la ventana se descolocan los controles o simplemente se quedan en una esquina de la ventana. Veremos cómo hacer esto y cómo trabajar con otras propiedades como pueden ser la ubicación inicial, personalizar con imágenes de fondo, nombre y otras propiedades como por ejemplo, cuál es el formulario de inicio cuando tenemos más de un formulario en una aplicación.

También aprenderemos las diferencias entre formulario modal y no modal y los métodos y eventos propios de los formularios.

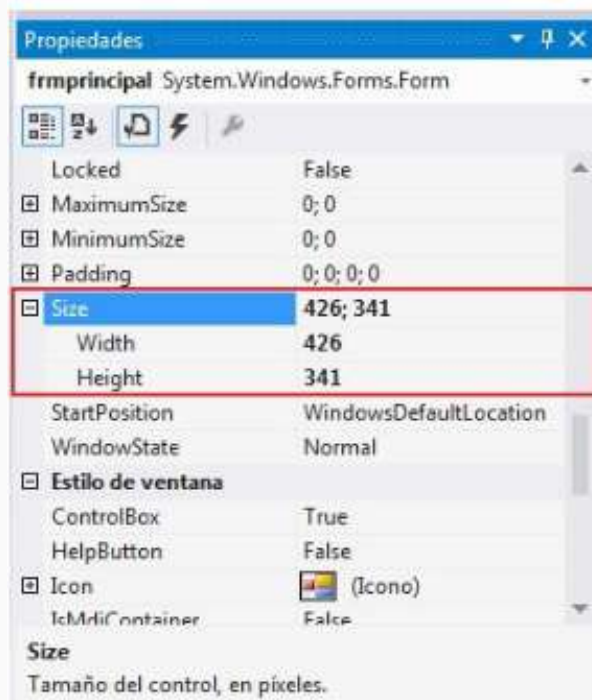
¡Comencemos!

3.2.1. Cambiar el tamaño a un formulario

Cambiar el tamaño a un formulario es muy sencillo, sólo debemos arrastrar con el ratón los bordes de la derecha inferior y ángulo:



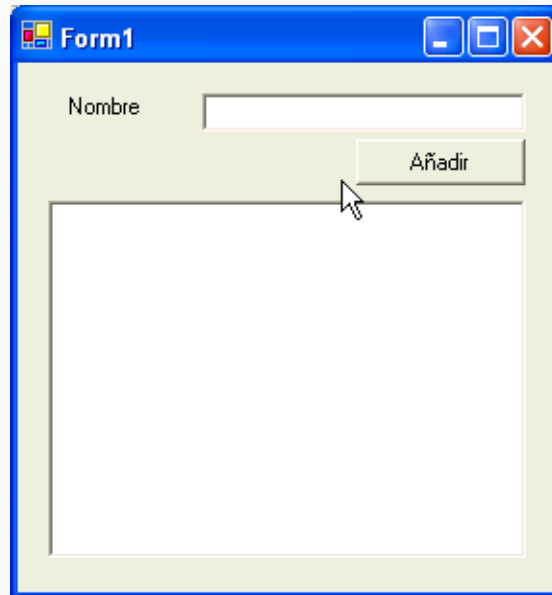
Como todas las propiedades, los valores del tamaño y posición se pueden controlar en la ventana propiedades. Podemos modificar directamente sus valores con la sección **"Size"**



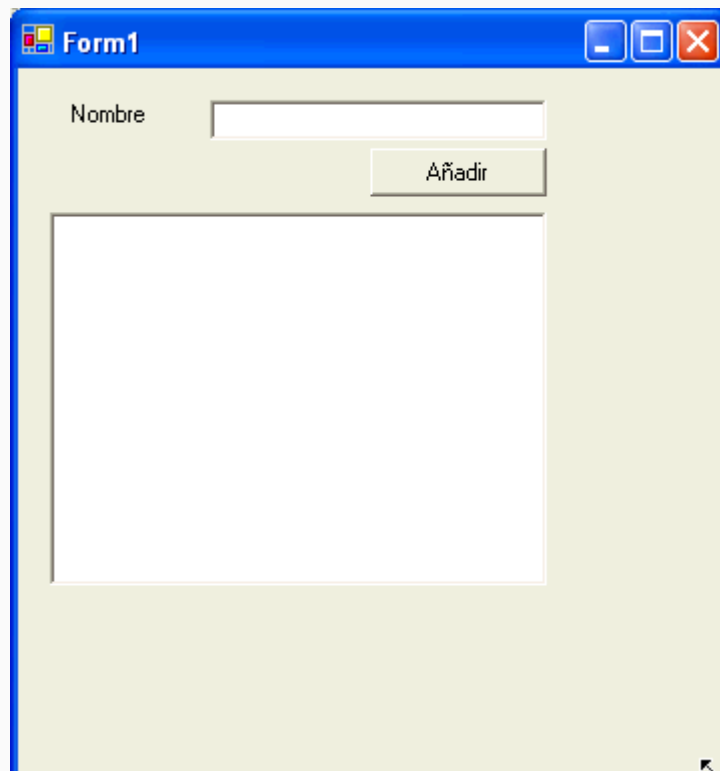
a. Redimensionar los controles en un formulario

Veamos un caso típico: el tamaño de una ventana de Windows, (que al fin y al cabo es un formulario), se puede hacer redimensionable, es decir, que el usuario pueda cambiar de tamaño. Lo normal es que los controles que dicho formulario contiene se adapten al nuevo tamaño de la ventana, con la idea de que no queden huecos vacíos al cambiar el tamaño de la ventana.

Por ejemplo, si tenemos esta ventana (o formulario):



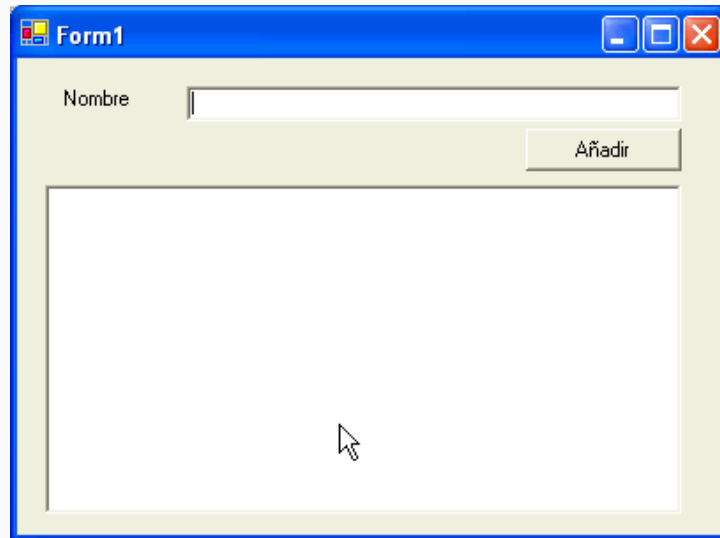
y la agrandamos, por ejemplo, para que tenga este otro aspecto:



Comprobaremos que la ventana se ha agrandado, pero los controles que hay en ella siguen teniendo el mismo tamaño y la misma posición que en la ventana

anterior, es decir, que cuando se diseñó el formulario.

Pues bien, la idea de lo que queremos hacer aquí es que al cambiar el tamaño de la ventana se ajusten los controles al nuevo tamaño, para que tuviesen este otro aspecto:



Para que esto sea posible de forma automática, hay que hacer unas cuantas asignaciones a los controles, de forma que podamos indicarle qué tienen que hacer cuando el tamaño de la ventana varíe. En este ejemplo, lo correcto sería que:

- La caja de texto superior se agrandase hacia el lado derecho.
- El botón Añadir se moviese hacia el extremo derecho del formulario.
- La lista se ajustara al ancho y también al alto de la ventana.

Todo esto lo podemos hacer en tiempo de diseño, es decir, cuando tenemos el formulario en el entorno integrado o bien lo podemos codificar dentro del propio formulario. Dónde hacerlo queda a tu criterio. Veamos cómo hacerlo en los dos casos y queda a tu elección cuál de los dos métodos quieres utilizar.

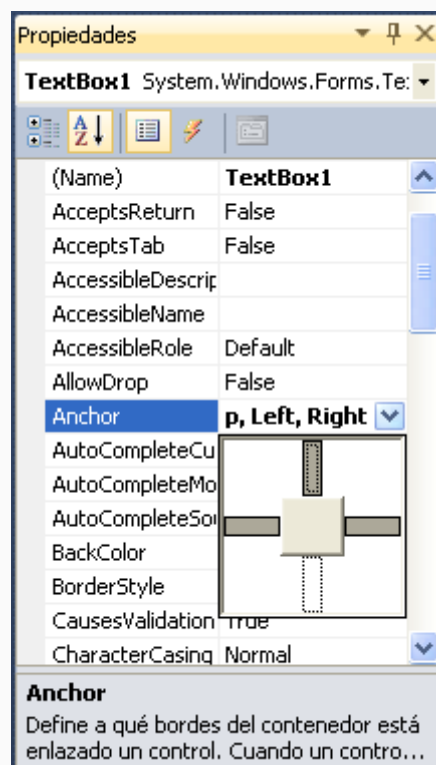
Antes de empezar vamos a crear un nuevo proyecto. Creamos un proyecto del tipo Windows, (aplicación Windows o Windows Application), y le ponemos como nombre WinApp3.

Añadimos una etiqueta (Label), un cuadro de texto (TextBox), un botón (Button) y un cuadro de lista (ListBox). Dejamos los nombres que el IDE ha puesto, salvo para el botón, el cual se llamará btnAdd. Colócalos como hemos visto en la figura de arriba en el primer caso:



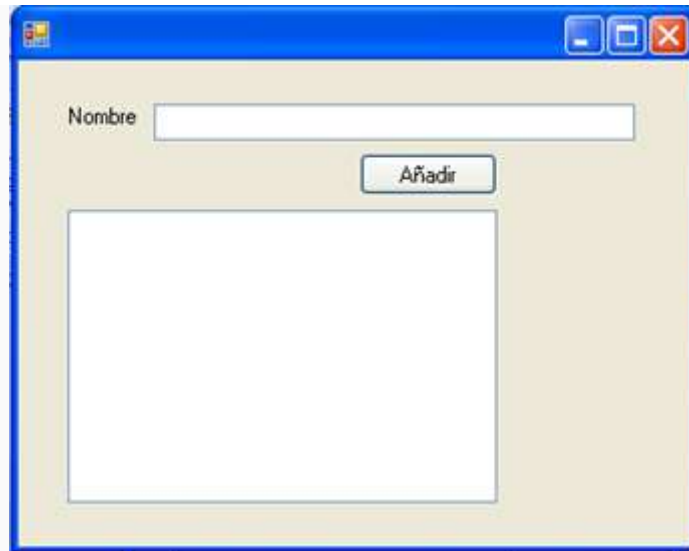
Ejemplo 1. Utilizando el IDE (en diseño)

Comenzamos: selecciona el cuadro de texto (textBox1) y en la ventana de propiedades, selecciona **Anchor**, verás que por defecto estarán los valores Top y Left, esto quiere decir que la caja de textos estará "anclada" arriba y a la izquierda, pero ahora vamos a seleccionar también la derecha. Cuando pulses en la lista desplegable verás que se muestra una imagen con cuatro líneas:



Inicialmente sólo estaban marcadas la superior y la izquierda con color más oscuro. Pulsa en la línea de la derecha, para que se ponga gris, de esta forma estaremos indicándole que también se "ancla" a la derecha.

Vamos a comprobar que esto funciona... Pulsa en el botón de ejecución o en la tecla F5, cuando tengas el formulario amplía el tamaño de la ventana, verás lo siguiente:

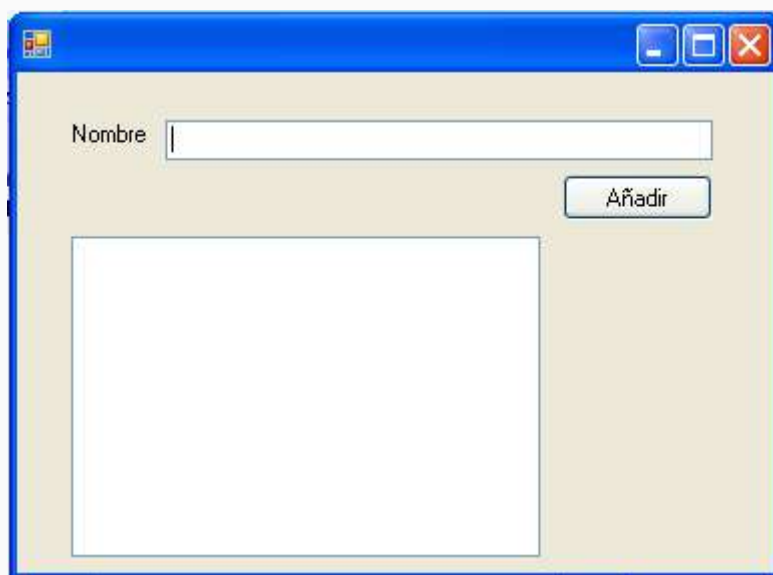
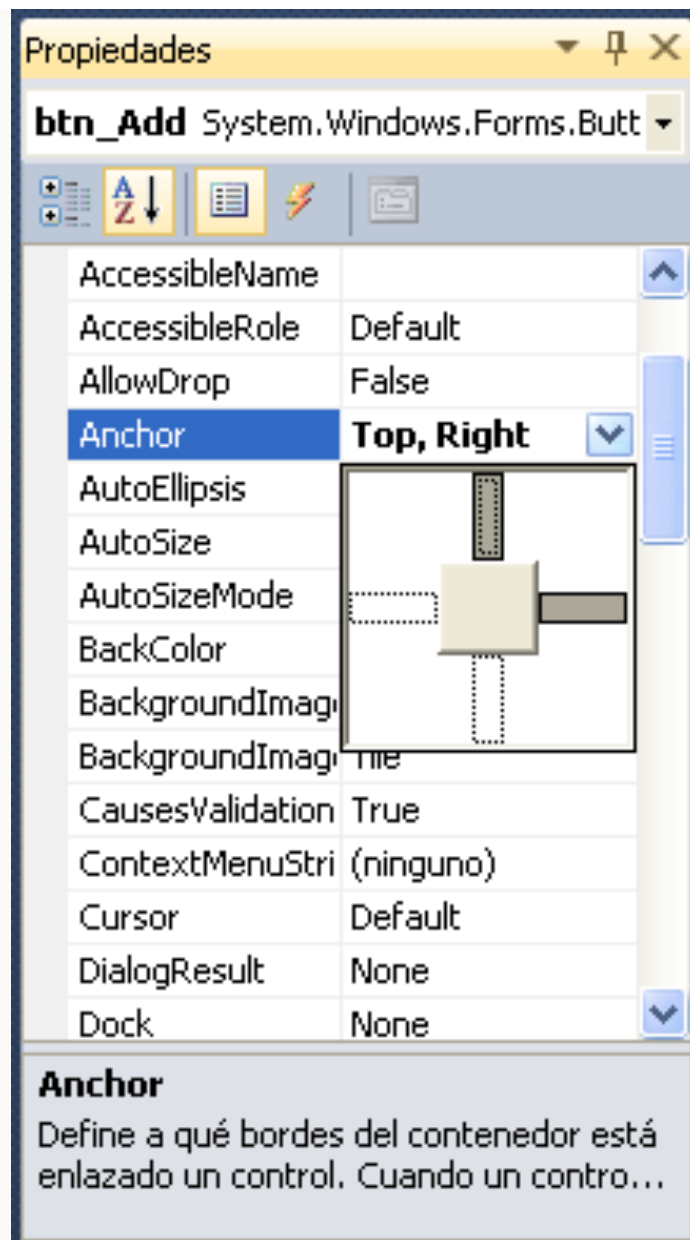


Perfecto, el cuadro de texto se ha adaptado al nuevo tamaño. Ahora encojemos la ventana e incluso haz que no se vea el botón. Podremos comprobar que el cuadro de texto sigue estando proporcionalmente igual de separado de los extremos superior, derecho e izquierdo y se adapta al tamaño de la ventana.

Incluso si intentamos hacer la ventana muy pequeña, el ancho se quedará justo en la parte izquierda de la caja de texto, con el alto podemos hacer que casi desaparezca, (salvo el título de la ventana, la barra de arriba, que se mantiene).

Ahora vamos a "**anclar**" el botón. Selecciona el botón y en la ventana de propiedades selecciona la propiedad **Anchor**. En este caso, lo que nos interesa es que el botón se desplace a la derecha, pero que no se haga más grande. Para ello, debes seleccionar las líneas de la derecha y la de arriba.

Es decir: ánclate en la parte de arriba y en la derecha, de forma que si cambiamos el tamaño del formulario, el botón se desplazará a la derecha o a la izquierda, pero no cambiará de tamaño, como le ocurre al cuadro de texto:



Si pulsamos F5 veremos que el botón llegará casi a tocar el lado izquierdo del formulario y allí se quedará, no permitiendo que se haga más pequeño.

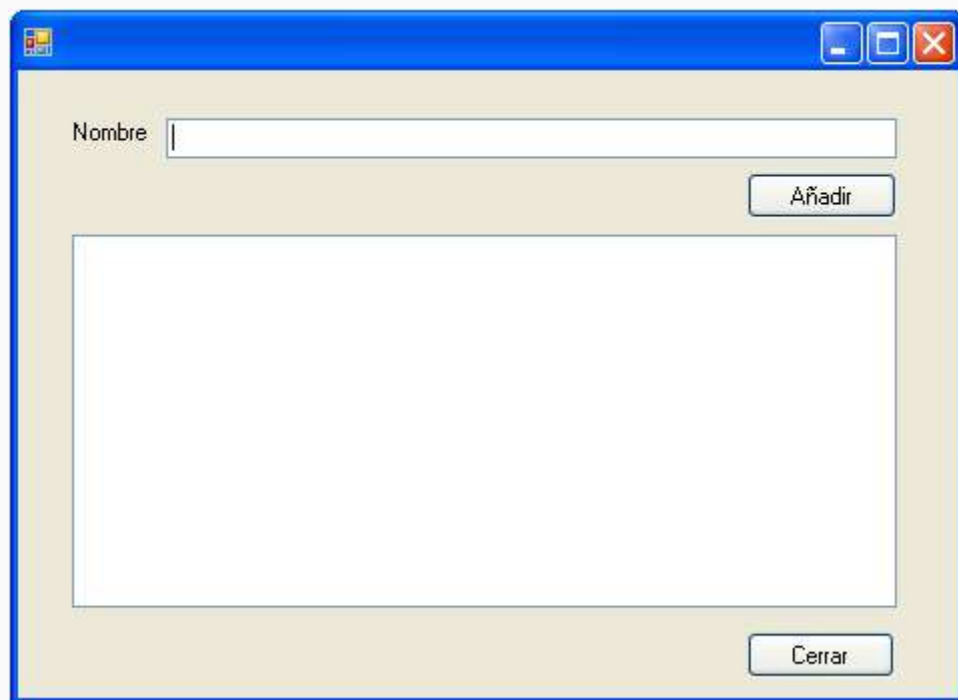
Por último vamos a anclar el cuadro de lista... en este caso debe hacerse grande tanto hacia la derecha como hacia la izquierda e incluso cuando se estira el

formulario desde la parte inferior, pero en la parte superior debe mantenerse en la misma posición. Simplemente debemos marcar las cuatro posiciones y hará que se ajuste siempre en la parte inferior y derecha.

Vamos a complicar un poco más la cosa y vamos a añadirle otro botón. En este caso, dicho botón estará en la parte inferior derecha del formulario, será el botón cerrar y al pulsarlo hay que cerrar el formulario.

Utilizaremos el método *close* del formulario en el evento clic del botón. Es decir, *this.Close* en el evento Clic de botón, que vamos a llamar btn_Cerrar. Como decíamos antes, este botón se debería anclar en la parte inferior derecha, por tanto, los valores que hay que asignar en *Anchor* son precisamente esos: Bottom y Right (abajo y derecha).

Como habrás notado, con el Label1 no hay que hacer nada, ya que por defecto el tamaño se ajusta por arriba y a la izquierda, por tanto, se quedará en la misma posición. Son los valores por defecto de la propiedad *Anchor*, por eso no es necesario asignarle nada.



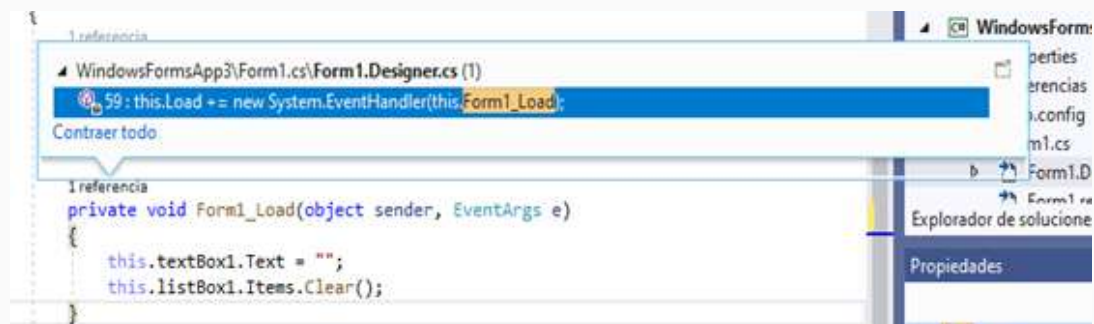
Ejemplo 2. Por código (en tiempo de ejecución)

Volvamos a nuestro IDE y hacemos doble clic en el formulario, dentro de él, no en los controles (botones, cuadro de lista, ..) debe ser en el formulario. Se debe mostrar el evento *Form1_Load*:

```
private void Form1_Load(object sender, EventArgs e)
{
}
}
```

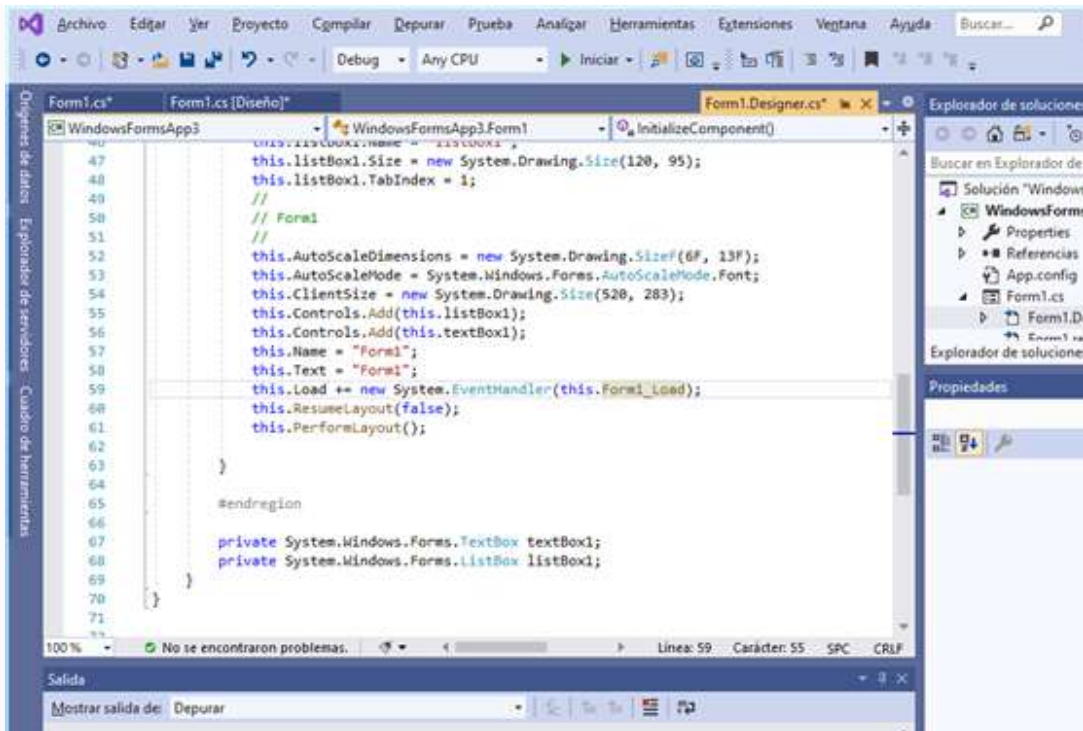
Este código es el que se va a ejecutar cuando se cargue el formulario. Vimos cuando hicimos doble clic en el botón que nos presentaba el código para ese evento. Si lo hacemos en el formulario se presenta el código que ejecutará al cargar (Load) el formulario Form1: **Form1_Load**, además aparecen una serie de parámetros que ya veremos más adelante. Esto se ejecuta pues cuando el formulario "se carga" en la memoria, justo antes de mostrarse, por tanto aquí es un buen sitio para hacer algunas "inicializaciones" o asignaciones que nuestro formulario necesite. De hecho es muy utilizado para que podamos limpiar el contenido de la lista, el del cuadro de texto, etc. e incluso hacer las asignaciones para que los controles se queden "anclados" en la posición que nosotros le indiquemos.

Vamos a ver primero cómo se "declara" este evento, aunque el C# lo hace automáticamente por nosotros, es conveniente verlo para ir aclarando conceptos... que tal vez ahora no necesites, pero en un futuro casi seguro que te hubiese gustado haberlo sabido.



Si pulsamos sobre el texto que nos aparece justo antes del método donde pone 1 "referencia", veremos que nos sale un *bocadillo* indicando un fichero que no hemos creado nosotros. Es el fichero Form1.Designer.cs. En este fichero se realizan las asignaciones para que el compilador de C# .NET sepa qué métodos están enlazados con qué eventos del formulario. Si accedemos a ese fichero veremos la siguiente información:

La línea `this.Load += new System.EventHandler(this.Form1_Load);` establece que al cargar el formulario se asigna como controlador de evento el método `Form1_Load`.



Veamos ahora el código para que los controles se anclen al formulario de forma que se adapten al nuevo tamaño del mismo:

```
private void Form1_Load(object sender, EventArgs e)
{
    this.txtNombre.Text = "";
    this.lstNombres.Items.Clear();
    // Asignar los valores para "anclar" los controles al formulario
    // El TextBox1 se anclará Arriba, Izquierda y Derecha
    this.txtNombre.Anchor = (AnchorStyles.Top | AnchorStyles.Left |
    AnchorStyles.Right);

    // El botón Añadir lo hará Arriba y a la derecha
    this.btnAnadir.Anchor = (AnchorStyles.Top | AnchorStyles.Right);

    // El listbox lo hará en los cuatro vértices
    this.lstNombres.Anchor = (AnchorStyles.Top | AnchorStyles.Left |
    AnchorStyles.Right | AnchorStyles.Bottom);

    // El botón cerrar sólo lo hará a la derecha y abajo
    this.btnCerrar.Anchor = (AnchorStyles.Right | AnchorStyles.Bottom);
}
```

Cuando se inicie el formulario se realizarán las siguientes operaciones:

limpiamos el contenido del cuadro de texto, simplemente asignándole una cadena vacía a la propiedad que almacena el texto que se escribe en pantalla `Text: this.txtNombre.Text = ""`; Muchas veces los objetos se leen al revés para interpretarlos mejor. En este caso sería: asignamos una cadena vacía a la propiedad `Text` del cuadro de texto llamado `txtNombre` del formulario actual `this`.

`IstNombres` es un cuadro de lista, es decir, es una lista de elementos. Inicialmente cuando se ejecute el programa quiero borrar todos los elementos así que los borro llamando al método **Clear** de la colección de elementos, así la dejo en blanco. El resto del código asigna a la propiedad `Anchor` (anclaje) los valores que queremos y que luego condicionarán su funcionamiento. Por ejemplo:

```
this.txtNombre.Anchor = (AnchorStyles.Top | AnchorStyles.Left | AnchorStyles.Right);
```

hará que el cuadro de texto se ancle arriba a la izquierda y a la derecha. Los **Ors** (símbolo `|` *tubería*) que se están utilizando sirven para "sumar" y el resultado sería el mismo que si usáramos el signo de suma, pero la razón de usar **Or** (`|`) es porque lo que queremos hacer es una suma de bits. Realmente da lo mismo usar la suma que `Or` en este caso, pero dejemos el `Or` que es lo apropiado ya que no estamos sumando números sino concatenando propiedades.

El efecto es el mismo que en tiempo de diseño. Pero de esta forma sabemos cómo modificar el comportamiento del anclaje de los controles en la ejecución del programa. Por ejemplo, en algunas ocasiones has visto un formulario que tiene un botón de "más información". Al pulsarlo la ventana o formulario se amplía poniendo más información en pantalla. Se han modificado la colocación y tamaño de algunos de los controles del formulario en tiempo de ejecución

b. Redimensionar el tipo de letra en un formulario

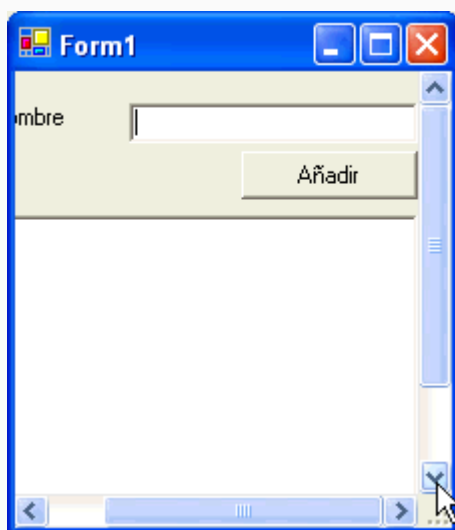
Sigamos ajustando nuestro formulario. Esta vez que los controles se adapten también a otros tamaños de fuentes, no a los que nosotros tenemos en nuestro equipo, ya que hay usuarios que por necesidades tienen que poner tamaños de fuentes más grandes e incluso más pequeñas.

La propiedad que hace eso posible es **AutoScaleMode**. Por defecto, tiene el valor *Font*. El escalado por *Font* es útil si se desea tener un control o un formulario para expandir o reducir según el tamaño de las fuentes en el sistema operativo, y debe usarse cuando no importa el tamaño absoluto del control o formulario. El escalado por *Dpi* es útil cuando se desea cambiar el tamaño de un control o un formulario con respecto a la pantalla. Por ejemplo, puede que se desee usar el ajuste de escala de puntos por pulgada (PPP) en un control que muestre un gráfico u otro gráfico para que siempre ocupe un determinado porcentaje de la pantalla.

Por tanto, los formularios, sin necesidad de que hagamos nada, se auto ajustarán al tamaño de las fuentes.

Otro tema interesante que tienen los formularios es la propiedad **AutoScroll**. Si asignamos el valor *True* (verdadero) a esta propiedad, hacemos que cuando el formulario se haga muy pequeño o muy estrecho, se muestren unas barras de desplazamiento (scrolls) para que pulsando en ellas podamos ver el contenido del mismo.

Por ejemplo, si no hubiésemos "anclado" nuestros controles, al hacer el formulario más estrecho se mostrarían unas barras de desplazamiento para que podamos ver los controles que están contenidos en el formulario como en esta figura:



Sigamos escribiendo algunos ejemplos de código para seguir familiarizándonos con el IDE de .NET. Queremos hacer lo siguiente en nuestro programa:

1. Al pulsar en el botón *Añadir*, se añadirá a la lista lo que hayas escrito en la caja de texto.
2. Al pulsar *Intro* será como si hubieses pulsado en el botón *Añadir*.

3. Al pulsar *Esc* es como si hubieses pulsado en el botón *Cerrar*.
4. Al pulsar en uno de los elementos de la lista, éste se mostrará en la caja de texto.
5. Al seleccionar un elemento de la lista y pulsar la tecla *Supr* (o *Del* si tu teclado está en inglés), dicho elemento se borrará de la lista, pero se quedará en la caja de texto, ya que al seleccionarlo para poder pulsar la tecla suprimir se habrá mostrado...

Veamos cómo hacer esto.

1. Lo de pulsar en Añadir y hacer algo: simplemente codificamos lo que haya que codificar en el evento Clic del botón `btnAnadir`. Para que nos muestre ese evento, sencillamente haremos doble-click en el botón y C# te mostrará el evento en cuestión y añadimos este código:

```
private void btnAnadir_Click(object sender, EventArgs e)
{
    this.lstNombres.Items.Add(this.txtNombre.Text);
}
```

Veamos qué hace este código en ejecución:



Conozcamos que hay en esta instrucción del procedimiento que se va a ejecutar cuando se haga clic (evento clic) del botón. Antes, recordemos el código que vimos en el evento `Form1_Load`, teníamos:

`this.lstNombres.Items.Clear()`

- **this** hace referencia a la clase actual, es decir, al formulario.
- **Items** son los elementos que tiene el objeto *ListBox*
- **Clear** es un método de *Items* que se encarga de limpiar los elementos de la lista, es decir, los borra.

Por tanto, esa línea lo que hace es borrar los elementos del `listbox`.

Ahora lo que necesitamos no es borrarlos, sino añadir nuevos elementos a la lista. Como ya sabemos que **Items** es el contenedor en el que se guardan los elementos de la lista, lo único que tenemos que saber es cómo añadir nuevos elementos a la lista. La respuesta es mediante la utilización del método **Add** que añade elementos a la lista de un listBox: Para añadir elementos a un listBox se usa el método *Add* de *Items*.

En la colección *Items* añadimos el contenido de nuestro cuadro de texto de pruebas "textBox". Pondremos esta instrucción:

```
lstNombres.Items.Add(txtNombre.Text)
```

El resultado es que se añade "add" a la lista de elementos "ítems" el texto "text" que contiene el cuadro de texto "txtNombre", con lo que aparece reflejado inmediatamente en el cuadro de lista.

3.2.2. Ubicación inicial del formulario

Para colocar un formulario en pantalla tenemos que tener en cuenta cómo queremos que se muestre. En ocasiones queremos que aparezca completamente maximizado, como una aplicación normal. Otras veces la intención es que aparezca con el tamaño que lo definimos en el IDE.

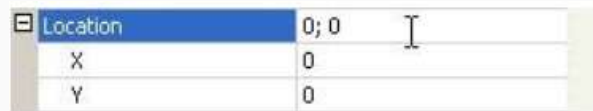
Por ejemplo para mostrar un mensaje, una pantalla de introducción de datos o consulta que no necesitan la pantalla completa.

El comportamiento inicial lo va a decir la propiedad "**StartPosition**":



Esta propiedad permite iniciarse al formulario de cinco formas diferentes:

- **Manual.** Seguirá los valores de tamaño y posición que le indiquemos en la propiedad Location.



- **WindowsDefaultBounds.** El formulario se encuentra colocado en la ubicación predeterminada de Windows y tiene los límites establecidos por Windows de forma predeterminada.
- **CenterParent.** El formulario está centrado en los límites de su formulario principal.

Si queremos tocar el código para cambiar estas propiedades, podemos hacer como en el punto anterior para indicarle en píxeles una determinada posición con las propiedades "**Left**" (para la coordenada X) y "**Top**" (para la coordenada Y), por ejemplo, para colocarlo a 100 píxeles de la parte superior:

```
frmPaleta.Top = 100;
```

Y para colocarlo a 100 píxeles de la izquierda:

```
frmPaleta.Left = 100;
```

Estas propiedades eran las utilizadas en anteriores versiones y que aquí también están disponibles. Pero en .NET se ha añadido el objeto "Location" con las propiedades X e Y. El ejemplo anterior quedaría:

```
frmPaleta.Location = New Point(100, 100);
```

La potencia de este objeto es que podemos cambiar de forma simultánea los dos valores.

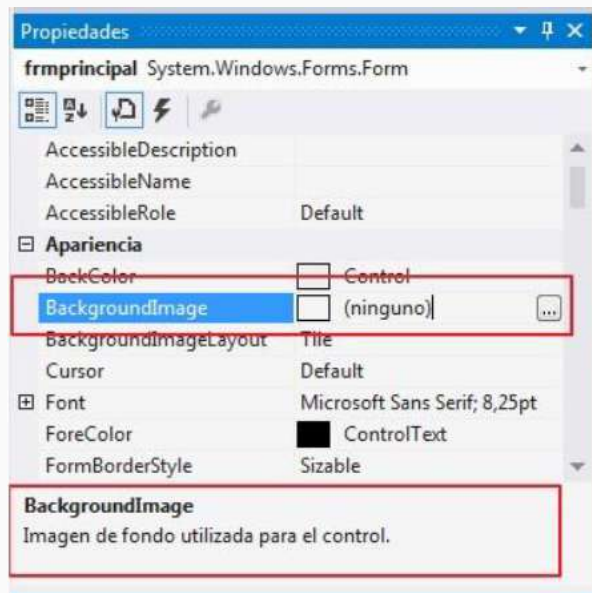
Podemos utilizar también la propiedad "**DesktopLocation**" en lugar de **Location** para ajustar la ubicación del formulario. Esta propiedad determina la ubicación del formulario relativa a la barra de tareas de Windows. Esto es útil si la barra de

tareas no se oculta automáticamente y se ha colocado fija en la parte izquierda o superior de la pantalla que esconde el origen de coordenadas (0,0). Si establecemos la ubicación de esta forma en un formulario a (0,0) nos aseguramos que no ocultará la barra de tareas en ningún caso, por ejemplo:

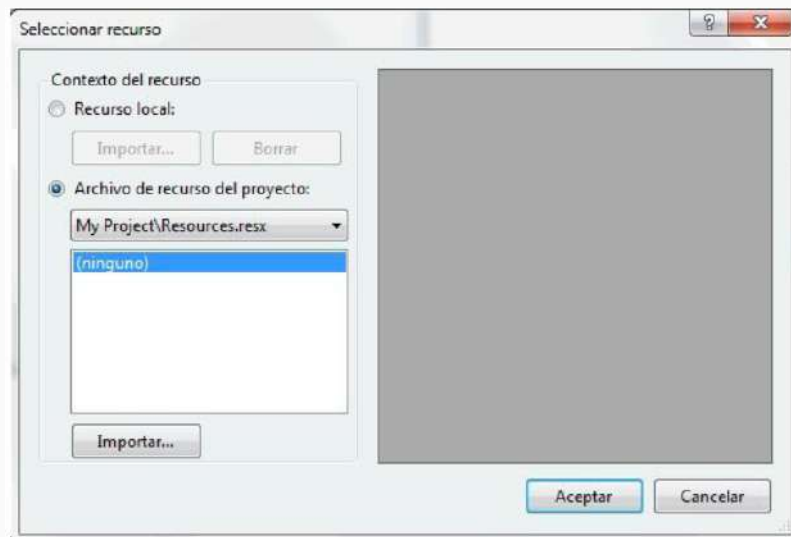
```
frmPaleta.DesktopLocation = New Point(0,0);
```

3.2.3. Poner imagen de fondo, añadir icono

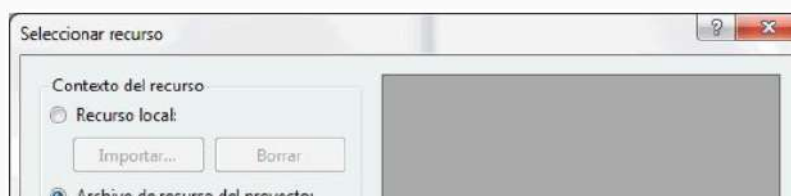
Una propiedad que no utilizaremos muchas veces pero que ayuda a comprender el grado de personalización que podemos llegar a conseguir con estas propiedades, es la de poner una imagen de fondo en el formulario. Puede ser el caso del logotipo de nuestra empresa en la pantalla que sale al comienzo del programa (llamada en ocasiones Splash). Para poner una imagen simplemente buscaremos la propiedad "**BackgroundImage**" y buscaremos la imagen de fondo que queremos:

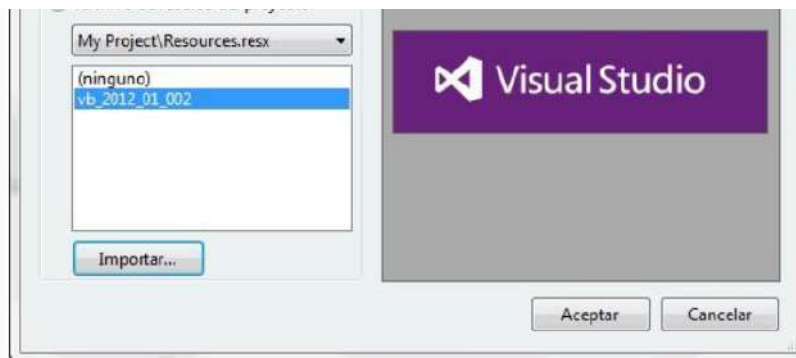


Al hacer clic nos muestra una pantalla que será habitual en varios controles:

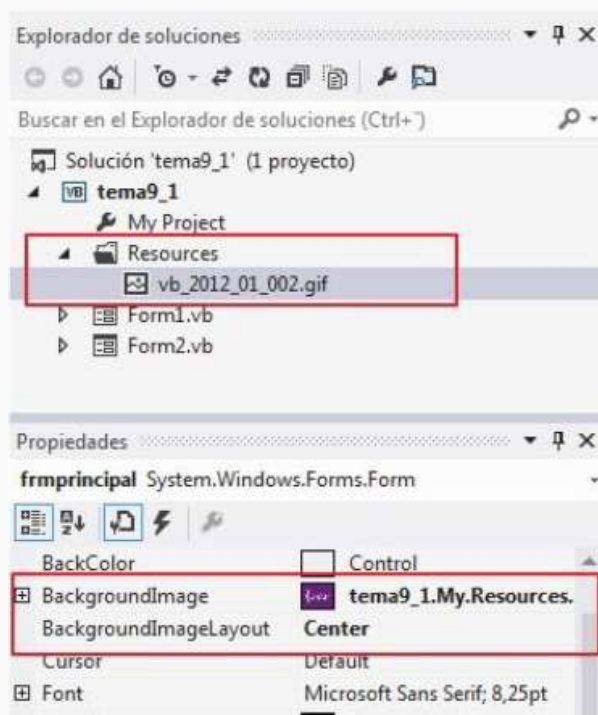


Indicaremos de dónde queremos obtener la imagen. Hay un fichero especial llamado "**My project\Resources.resx**" donde se almacenarán los recursos asociados a nuestro proyecto, como pueden ser iconos y gráficos utilizados en nuestra aplicación. Para este fondo de escritorio pulsaremos "Importar" y le indicamos el gráfico:

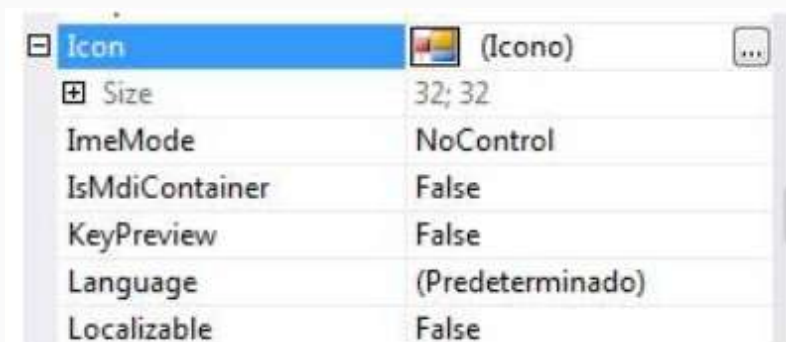




La idea es tener un "contenedor" de imágenes. Si seleccionamos la otra opción sería similar pero no se almacenaría en nuestro fichero de recursos proyecto, sino que la incluiría en la pantalla sin más. Para un uso es válido pero no si queremos invocar en otras partes del proyecto a varias imágenes distintas. Al pulsar en "Aceptar" la imagen queda incluida en nuestro proyecto tal y como te he contado:



En las propiedades nos indicará ahora que hay una imagen. Y además con la propiedad "BackgroundImageLayout" podemos hacer que simplemente quede centrada, en lugar de hacer el mosaico predeterminado:

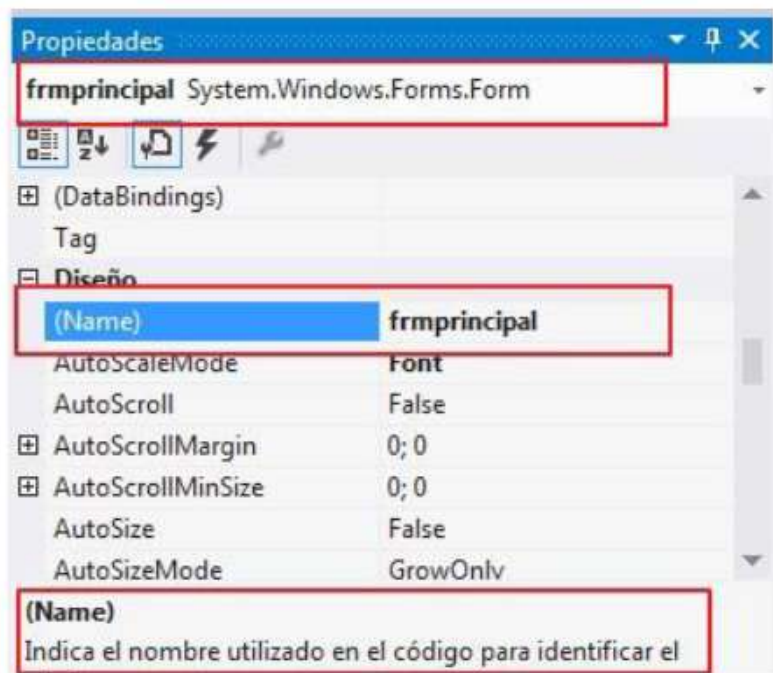


Si se desea se puede poner un icono al formulario. Este icono será la imagen que se muestre en el cuadro de menú del sistema del formulario y cuando el formulario se minimice:

Icon	(Icono)
Size	32; 32
ImeMode	NoControl
IsMdiContainer	False
KeyPreview	False
Language	(Predeterminado)
Localizable	False

3.2.4. Nombre del formulario

El formulario como el resto de los componentes y controles de nuestra interfaz debe tener obligatoriamente un nombre, Visual Studio le pone uno genérico Form1,Form2,... que debemos cambiar para acceder a él en el código más cómodo. Para cambiarle el nombre debemos ponerlo en la propiedad **Name**.



¡Vigila!

En ocasiones nuestro formulario tiene muchos elementos y puede que en nuestra ventana de propiedades nos muestre las propiedades de otro control porque sin querer hemos hecho clic sobre otro. La mejor forma de asegurarse es comprobando en la parte superior de la ventana de propiedades que aparece la clase adecuada, en nuestro ejemplo verás frmPrincipalSystem.Windows.Forms.Form

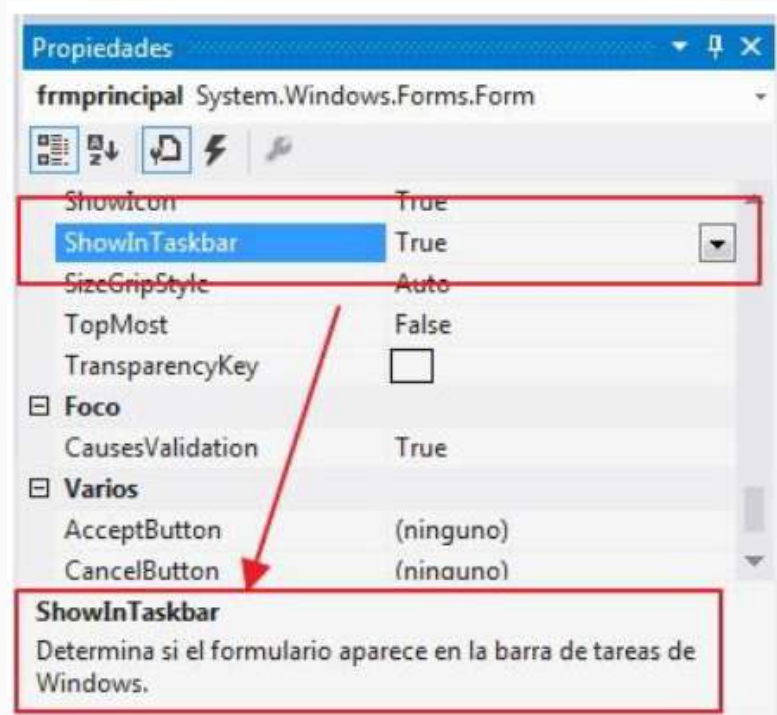
a. Formulario principal

Si vamos a tener varios formularios debemos tener claro cuál va a ser el principal del programa y así lo indicaremos en las propiedades del proyecto. Es importante, porque el que indiquemos aquí será por el que comience el programa.

Para ello vamos a la clase Program.cs y agregamos la siguiente línea de código:

```
Oreferencias
static class Program
{
    /// <summary>
    /// The main entry point for the application.
    /// </summary>
    [STAThread]
    Oreferencias
    static void Main()
    {
        Application.SetHighDpiMode(HighDpiMode.SystemAware);
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new frmPrincipal());
    }
}
```

En la parte central indicamos el formulario de inicio, es decir, el punto de entrada a nuestro programa. Ya tenemos un pequeño avance de las propiedades del formulario, ahora toca practicar con otras para ver su funcionamiento. Para ayudarnos en estas pruebas, recuerda tener activada la ayuda que aparece en la parte inferior de la ventana de propiedades porque nos irá indicando qué hace cada propiedad. Por ejemplo, "**ShowInTaskBar**" hará que el formulario aparezca en la barra de tareas de Windows:



3.2.5. Propiedades y características de los formularios

Veamos ahora una lista de las propiedades más útiles de los formularios:

Propiedad	Descripción
AcceptButton	Representa el botón que se va a usar como botón Aceptar del formulario
AutoScale	Si está activado el formulario ajusta automáticamente la fuente de la pantalla
AutoScroll	Determina si aparecen automáticamente las barras de desplazamiento cuando los controles están situados fuera del área del formulario
BackColor	Color de fondo utilizado para mostrar texto y gráficos en el control
BackgroundImage	Imagen de fondo para el formulario
CancelButton	Obtiene o establece el control de botón que se activará cuando el usuario presione la tecla ESC
CausesValidation	Obtiene o establece un valor que indica si el control hace que se realice una validación de todos los controles que requieren validación cuando reciben el foco.
ContextMenu	Obtiene o establece el menú contextual asociado al control.
ControlBox	Obtiene o establece un valor que indica si se muestra un cuadro de control en la barra de título del formulario.
Cursor	Obtiene o establece el cursor que se muestra cuando el puntero del mouse se sitúa sobre el control.
Enabled	Obtiene o establece un valor que indica si el control puede responder a la interacción del usuario, es decir, si está habilitado.
Font	Obtiene o establece la fuente del texto que muestra el control.
ForeColor	Obtiene o establece el color de primer plano del control.
HelpButton	Obtiene o establece un valor que indica si se muestra un botón de ayuda en el cuadro de título del formulario.
Icon	Obtiene o establece el icono del formulario
IsMdiChild	Obtiene un valor que indica si el formulario es un formulario MDI (interfaz de múltiples documentos) secundario
IsMdiContainer	Obtiene o establece un valor que indica si el formulario es un contenedor para formularios MDI (interfaz de múltiples documentos) secundarios
MaximizeBox	Obtiene o establece un valor que indica si se muestra el botón Maximizar en la barra de título del formulario.
Menu	Obtiene o establece el MainMenu que se muestra en el formulario.
MinimizeBox	Obtiene o establece un valor que indica si se muestra el botón Minimizar en la barra de título del formulario
Opacity	Obtiene o establece el nivel de opacidad del formulario.
ShowInTaskBar	Obtiene o establece un valor que indica si se muestra el formulario en la barra de tareas de Windows.
Size	Tamaño en píxeles
StartPosition	Obtiene o establece la posición inicial del formulario en tiempo de ejecución
TopMost	Obtiene o establece un valor que indica si el formulario debe mostrarse como formulario de nivel superior de la aplicación.
TransparencyKey	Obtiene o establece el color que representará las áreas transparentes del formulario.
WindowState	Obtiene o establece el estado de la ventana del formulario. (normal, minimizado, maximizado)

Los **bordes** del formulario que se asignan con la propiedad "**FormBorderStyle**" condicionan mucho el comportamiento del formulario. Además de proporcionar un aspecto visual distinto, esta propiedad influye en cómo se escribe la barra del título y sus botones. Por otra parte, esta propiedad también afecta a la capacidad que tienen para redimensionar los formularios, podemos eliminar la pestaña inferior derecha que permite su redimensionamiento. En esta tabla tenemos los distintos valores de esta propiedad:

Valor	Descripción
Ninguno	Sin bordes. Utilizado en las pantalla de inicio del programa por ejemplo.
Fixed3D	Borde tridimensional fijo. No se puede redimensionar. Puede incluir un cuadro de control a la izquierda, barra de títulos y botones de maximizar y minimizar en la barra de
FixedDialog	Borde fijo de una sola línea, utilizado para cuadros de diálogo. No se puede redimensionar. Puede incluir un cuadro de control a la izquierda, barra de títulos y botones de maximizar y minimizar en la barra de título
FixedSingle	No se puede redimensionar. Puede incluir un cuadro de control a la izquierda, barra de títulos y botones de maximizar y minimizar en la barra de título.
FixedToolWindow	Borde de ventana de herramienta de tamaño fijo. Una ventana de herramientas no aparece en la barra de tareas ni en la ventana que aparece cuando el usuario presiona ALT+TAB. Muestra un botón de cerrar y el título con letra más pequeña.
Sizable	Es el valor predeterminado, se puede redimensionar. Puede incluir un cuadro de control a la izquierda, barra de títulos y botones de maximizar y minimizar en la barra de título. Se puede redimensionar utilizando el borde del formulario o los botones de maximizar/minimizar
SizableToolWindow	Borde de ventana de herramienta de tamaño variable. Una ventana de herramientas no aparece en la barra de tareas ni en la ventana que aparece cuando el usuario presiona ALT+TAB.. Utilizada para herramientas de Windows, muestra un botón de cerrar y el título con letra más pequeña

3.2.6. Formularios Modales

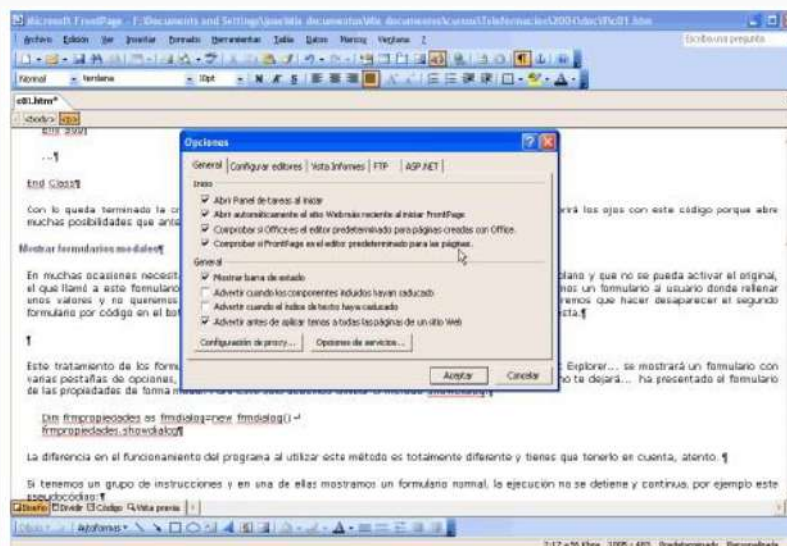


Debes conocer

Los formularios modales son los formularios que se presentan en primer plano de forma que no se pueda activar el original, el que llamó a este formulario.

Por ejemplo, en una recogida de datos para una consulta... le mostraremos un formulario al usuario donde rellenar unos valores y no queremos que se active la pantalla de debajo si le hace clic. En este caso tendremos que hacer desaparecer el segundo formulario por código en el botón "Aceptar" o "Cancelar" pero no podremos dejar la principal delante de esta.

Otro ejemplo sería, en la página de propiedades de FrontPage, que no me deja acceder a la ventana principal hasta que no pulso en "Aceptar" o "Cancelar":



Este tratamiento de los formularios es muy utilizado. Por ejemplo, si pulsamos en las Opciones de Word se mostrará un formulario con varias pestañas laterales de opciones. Si ahora intentamos hacer clic en el programa sin cerrar esta ventana el programa no nos dejará. Esto es porque ha presentado el formulario de forma "modal".

Para esto sólo debemos utilizar el método *ShowDialog*:

```
Form frmPropiedades = new Form();  
frmPropiedades.ShowDialog();
```

El funcionamiento del programa al utilizar este método es totalmente diferente y debemos tenerlo en cuenta.

Si tenemos un grupo de instrucciones y en una de ellas mostramos un formulario normal, la ejecución no se detiene y continúa.

Por ejemplo, este *pseudocódigo*:

```
calcula_datos  
muestra_formulario  
graba_datos termina_proceso
```

En este caso después de mostrar el formulario continúa con la ejecución del código siguiente. Sin embargo, en este caso:

```
calcula_datos  
muestra_formulario_MODAL  
graba_datos termina_proceso
```

El programa se detiene cuando se muestra el formulario modal y no continúa hasta que se ha ocultado o descargado de memoria. De ahí la importancia de utilizarlo.

Después del razonamiento anterior, sólo debemos saber la instrucción para mostrar un **formulario no modal**:

```
Form frmToolBox = new Form();  
frmToolBox.Show();
```

El ejemplo de uso de este tipo de formularios es de una barra de herramientas, es un formulario con iconos y que no hace falta cerrar para seguir trabajando, puede estar abierto a un lado de nuestra aplicación como formulario auxiliar.

3.2.7. Formularios de nivel superior (Top-Most)



Debes conocer

Un formulario de nivel superior es aquel que se muestra siempre en primer nivel aunque no esté activo.

Si en una aplicación tengo varios formularios, queda siempre en primer plano aquel sobre el que hacemos clic y los demás pasan a segundo plano.

En ocasiones, puedo querer que siempre haya uno en primer plano aunque ni siquiera esté activo, para esto activaré el formulario con la propiedad "**TopMost**":

```
frmToolBox.TopMost = true;
```

Un formulario de este tipo puede ser, por ejemplo, una barra de botones, si la ponemos flotante encima de la pantalla de trabajo, por ejemplo en Word, podemos seguir escribiendo y esta ventana seguirá visible en la posición donde la hayamos colocado.

3.2.8. Métodos de los formularios

Así como las propiedades determinan su aspecto y forma inicial, los métodos determinan su comportamiento. En el código los métodos se diferencian cuando los escribamos porque tienen el símbolo de un cubo en 3D.

Los métodos los utilizaremos durante la ejecución del programa para realizar acciones con los formularios. Por ejemplo, para ocultar un formulario utilizaremos el método "**hide**":

```
frmDatos.Hide();
```

Los métodos más importantes son:

Método	Descripción
Activate	Activa el formulario y le proporciona el enfoque
Close	Cierra el formulario.
Contains	Obtiene un valor que indica si el control especificado es un control secundario del control.
FindForm	Recupera el formulario en el que se encuentra el control.
Hide	Oculto el formulario al establecer el valor a False
LayoutMdi	Organiza los formularios MDI (interfaz de múltiples documentos) secundarios del formulario MDI principal.
Refresh	Obliga al control a invalidar su área cliente y, acto seguido, obliga a que vuelva a dibujarse el control y sus controles secundarios.
SendToBack	Envía el control al final del orden Z.
Show	Muestra el control
ShowDialog	Sobrecargado. Muestra el formulario como un cuadro de diálogo modal.
Update	Hace que el control vuelva a dibujar las regiones no válidas en su área de cliente.

3.2.9. Eventos de formularios



Debes conocer

Los eventos como hemos comentado en otras ocasiones suceden cuando hay cambios en los objetos o controles, por ejemplo, cuando un usuario abre, cierra, mueve o interactúa con el formulario. Estos eventos se "disparan" o activan cuando el usuario interactúa con el teclado o ratón.

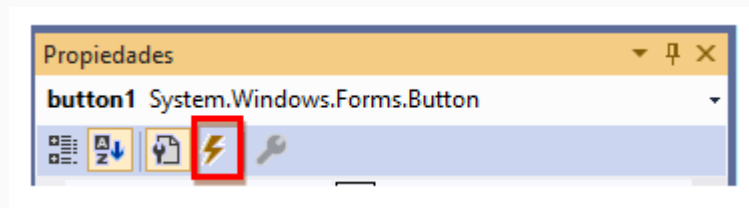
Existen muchos eventos pero sólo vamos a mostrar los más interesantes:

Evento	Descripción
Activated	Se produce cuando un formulario pasa a estar activo
Click	Se produce cuando se hace clic sobre el formulario
Closed	Se produce cuando se ha cerrado el formulario
Closing	Tiene lugar cuando se cierra el formulario.
Deactive	Se produce cuando el formulario pierde el enfoque y no está activo
Doubleclick	Se produce cuando se hace doble clic sobre el formulario
Gotfocus	Se produce cuando recibe el enfoque
KeyPress, KeyDown, KeyUp	Se producen cuando se pulsa una tecla, cuando se presiona y cuando se libera la tecla
LostFocus	Se produce cuando pierde el enfoque
MouseDown,MouseUp	Se produce cuando se está pulsando un botón y cuando se libera
MouseMove	Se produce cuando el ratón se mueve sobre el formulario
Move	Se produce cuando se mueve el formulario
Resize	Se produce cuando se cambia de tamaño el formulario

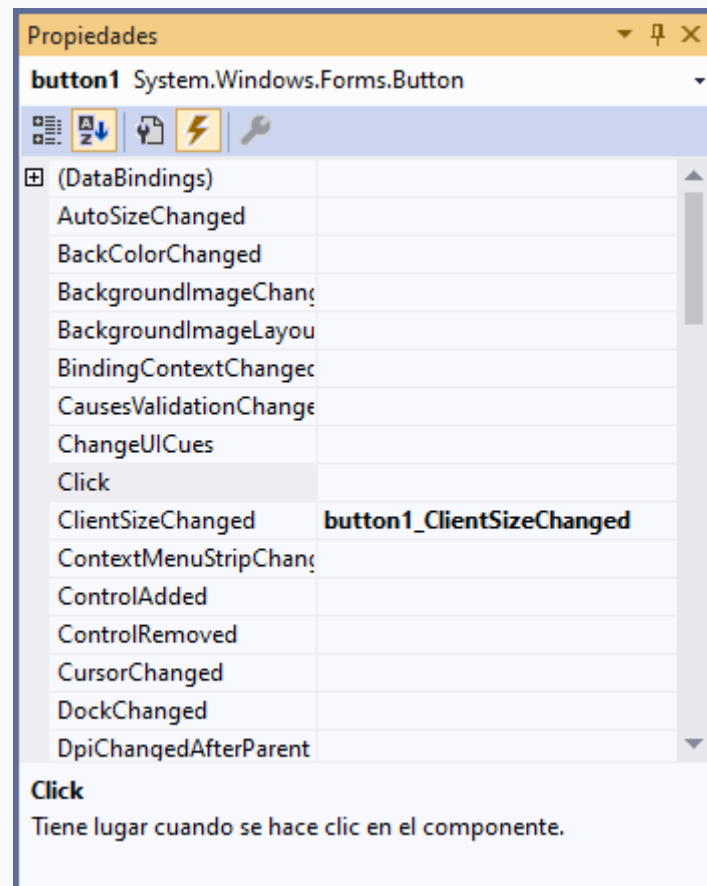
Ya hemos visto en alguna ocasión el evento Load(). Es muy interesante porque nos permite escribir instrucciones que se van a realizar en el proceso de carga del formulario, con lo que podremos rellenar de datos cuadros de listas y otros elementos para que cuando se muestre esté listo para el usuario.

a. ¿Cómo hacemos para escribir código en otros eventos?

Para poder usar otros eventos de un objeto, tenemos todos los eventos de cada objeto disponibles desde la ventana de propiedades del objeto.



Desde este botón, veremos todos los eventos del control que tengamos seleccionado.



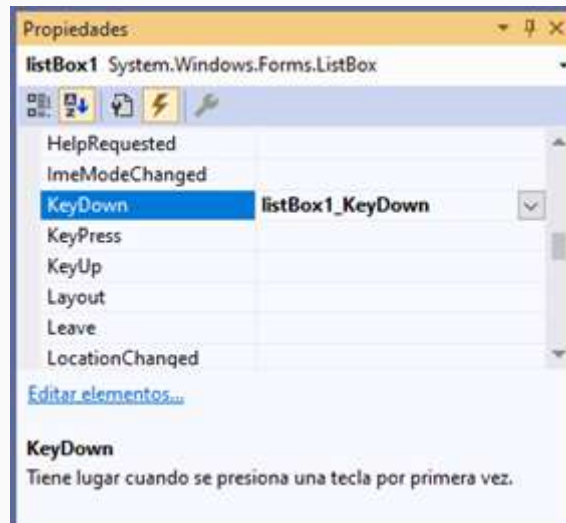
Si pulsamos doble-click sobre la parte de la izquierda, sobre cualquiera de los eventos, nos genera el método del evento. En la parte de la derecha, podemos establecer y cambiar el método que tenemos adjudicado para el evento.

Nota: Cada control tiene sus propios eventos. Muchos son comunes, por ejemplo, el evento clic está disponible en casi todos. Luego tiene los propios que dan funcionalidad distinta a cada control.

Cómo manejar eventos de teclado.

En el ejemplo anterior hemos controlado qué hacer cuando el usuario hace clic con el ratón en un elemento de la lista, ahora vamos a controlar qué hacer con determinadas teclas. Para esto vamos a utilizar el evento que controla cuándo se pulsan las teclas: el evento **KeyDown**.

Seguimos con el ejemplo anterior, buscamos el evento **KeyDown**:



El procedimiento que nos escribiré debajo es:

```
private void lstNombres_KeyDown(object sender, KeyEventArgs e)
{
}
}
```

Pero si hemos dicho que íbamos a utilizar el KeyPress ¿por qué cogemos ahora el KeyDown?. Bien, veamos los eventos que tenemos para manejar el teclado:

- **KeyPress**: Se produce cuando el usuario presiona y suelta una tecla
- **KeyDown**: Se producen cuando el usuario presiona una tecla
- **KeyUp**: Se producen cuando el usuario suelta una tecla

Cada uno de estos eventos tiene su utilidad. Por ejemplo, el KeyPress digamos que es el "normal", cuando se presiona una tecla normalmente y se suelta. Pero ¿y si queremos utilizar dos teclas simultáneas? Debemos saber si está la primera tecla pulsada, y en esa situación KeyDown será la opción adecuada, ya que con este evento se interceptan las pulsaciones de tecla cuando se empieza a pulsar la misma. Además existen muchas teclas especiales que sólo funcionan con KeyDown como es el caso de la tecla Supr (suprimir) que es una tecla especial y no se detecta en el evento KeyPress.

Con esto, ya sabemos cómo detectar cuando se pulsa la tecla suprimir (Supr/ Del), ahora debemos borrar el elemento de la lista que esté activo o

seleccionado:

```
private void lstNombres_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Delete)
    {
        lstNombres.Items.Remove(lstNombres.SelectedItem);
    }
}
```

La explicación es la siguiente: Cuando se ejecuta este procedimiento una variable llamada "e" indica el código de la tecla que se ha pulsado. La comparamos entonces para ver si la tecla de suprimir: e.KeyCode = Keys.Delete. Si es así, eliminamos el elemento que está seleccionado.

La variable "e" está en la definición del evento de este cuadro de lista:

```
private void lstNombres_KeyDown(object sender, KeyEventArgs e )
```

Cada evento nos proporcionará cierta información según sea su naturaleza. Está claro que si estamos seleccionando el evento de pulsar una tecla "KeyDown" nos tendría que decir qué tecla es la pulsada. Así es, lo dice en el parámetro de entrada:

KeyEventArgs e

que nos dice que nos proporciona una variable que se llama "e" que es de tipo "**KeyEventArgs**", es decir, de tipo evento de teclado o tecla.

Recuerda que **SelectedItem** nos indica el elemento que actualmente está seleccionado y usando el método **Remove** de la colección Items, lo quitamos de la lista.

Nota: El parámetro de entrada llamado "e" que veremos en los eventos tiene la información adecuada al evento. Si es de tecla, cuál es la que se ha pulsado. Si es de selección de un elemento de la lista, el elemento que se ha seleccionado, si es la de mover un cuadro por la pantalla tendrá las coordenadas de dónde se ha desplazado.

Por tanto, "e" puede tener un solo valor o varios, dependiendo del evento que esté detectando.

Selecciones múltiples en un cuadro de lista

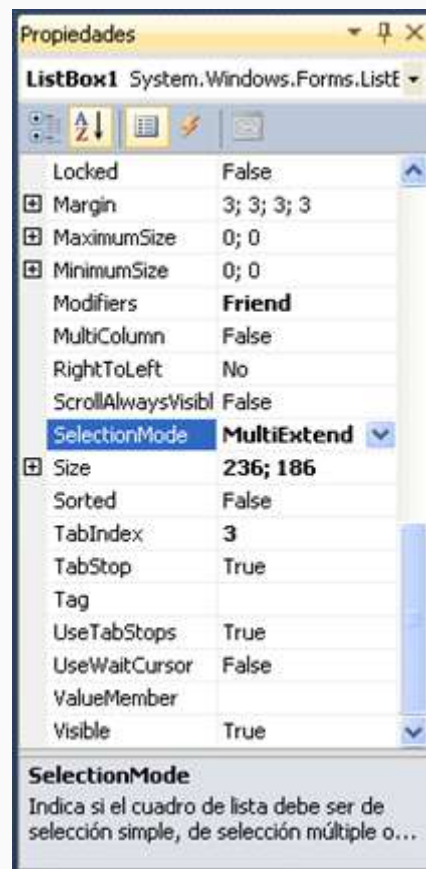
En este ejemplo queremos hacer una selección múltiple de un cuadro de lista y suprimir todos los que estén seleccionados. Con lo que sabemos tenemos ya una idea aproximada de cómo hacerlo pero nos falta alguna cosa. Veamos.

Una selección múltiple (seleccionar varios elementos a la vez de una lista) es algo habitual y que debemos tener en cuenta en nuestros programas. Cuando permitimos múltiple selección en un cuadro de lista (listBox), podemos seleccionar un elemento o varios, en este caso éstos pueden estar consecutivos o no. Por ejemplo, si seleccionas un elemento de la lista y manteniendo pulsada la tecla *Shift* (mayúsculas), pulsas en otro que está más arriba o más abajo, se seleccionan todos los elementos intermedios, (esto es habitual de Windows); también puedes seleccionar elementos no contiguos, si pulsas la tecla *Control* y con el ratón vas haciendo clic en elementos no consecutivos.

Lo primero que necesitamos saber es:

¿Cómo hacer que un listBox permita múltiple selección? Por defecto sólo se puede seleccionar un elemento a un mismo tiempo.

Para que un ListBox permita múltiple selección de elementos, hay que asignar a la propiedad **SelectionMode** el valor **MultiExtended**. Por tanto, selecciona el cuadro de lista ListBox1 y en la ventana de propiedades, asigna dicho valor a la propiedad SelectionMode.



Ahora tenemos que hacer más cosas cuando se detecta la pulsación de la tecla suprimir en el evento `KeyDown`, ya que tenemos que saber qué elementos están seleccionados para poder borrarlos. Es decir, de alguna manera debemos saber qué elementos están seleccionados.

Lo primero que tenemos que hacer es recorrer todos los elementos del ListBox para saber si está o no seleccionado, pero ese recorrido hay que hacerlo desde atrás hacia adelante... ¿por qué? Porque si lo hiciéramos desde el principio de la lista, al eliminar un elemento de dicha lista, el número de elementos variaría y tendríamos problemas cuando llegásemos al final, ya que no será el mismo

número de elementos después de haber borrado alguno, mientras que al recorrer los elementos desde el final hacia adelante, no importará que borremos alguno del final, ya que el siguiente que comprobaremos estará más al principio que el recién borrado y no tendremos problemas.

Veamos primero el código que habría que usar y después lo comentamos.

```
private void listBox1_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Delete)
    {
        // Elemento en la posición i del listBox
        int i;

        // Borra las palabras seleccionadas del listBox
        for (i = listBox1.SelectedItems.Count - 1; i >= 0; i--)
        {
            listBox1.Items.Remove(listBox1.SelectedItems[i]);
        }
    }
}
```

La parte del **if (e.KeyCode == Keys.Delete)** ya la vimos antes. Sirve para detectar la pulsación de la tecla de borrado.

int i indica al programa que vamos a usar un número y que ese número lo guarde en la *variable* i.

C# .Net nos indica en ese bucle los elementos que están seleccionados gracias a la colección de elementos seleccionados "SelectedItems.Count". Así que devolverá una colección con los elementos que ha seleccionado el usuario. Además todas las colecciones tienen una propiedad llamada **Count** que me dice cuántos elementos hay en esa colección. Pues bien, si hacemos:

for (i = listBox1.SelectedItems.Count - 1; i >= 0; i--)

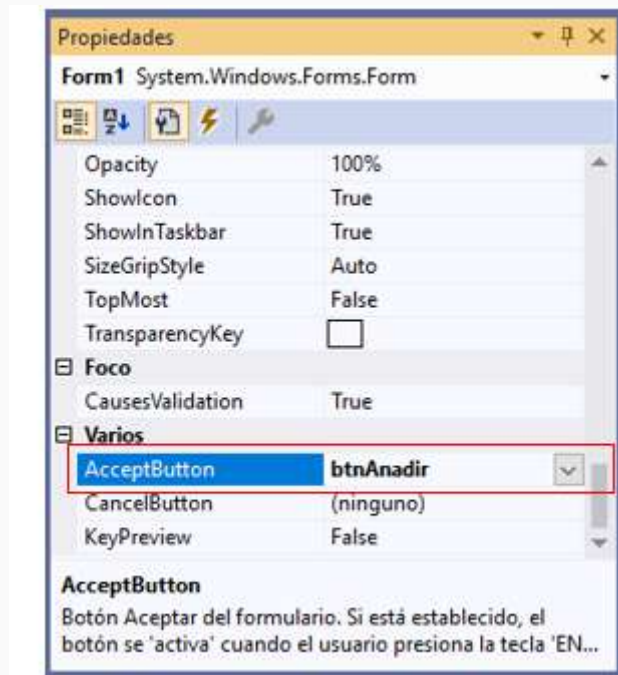
le estamos indicando que use la *variable* i para ir guardando los valores que resulten de contar desde el número de elementos que hay seleccionados hasta cero. El **i--** se usa para contar hacia atrás, (de mayor a menor), pero eso, al igual que el **for**, también lo veremos más adelante.

SelectedItems es una colección en la que están los elementos que hay seleccionados en el ListBox.

listBox1.Items.Remove(listBox1.SelectedItems[i]); Esto elimina el elemento que está seleccionado y que ocupa la posición i dentro de la colección de elementos

seleccionados.

Nos queda decirle al formulario que cuando se pulse el botón de aceptar haga lo mismo que si pulsamos el botón de Añadir. Para esto nos vamos al diseñador y buscamos la siguiente propiedad:



Esto es qué debe hacer cuando se pulse el botón Intro. Normalmente cuando trabajamos con formularios en cualquier programa, podemos movernos por los distintos elementos pulsando la tecla del tabulador. Además, la tecla de "intro" suele estar asociada a alguna acción de ese formulario: cerrar, añadir un elemento, guardar datos,...

Lo que queremos hacer en nuestro ejemplo es asociar el código de añadir el elemento con un botón de los que hemos puesto nosotros: *Añadir*. Como vemos en la imagen anterior, al pinchar en este desplegable el entorno es lo suficientemente inteligente como para mostrarme la lista botones que tengo en mi formulario: selecciono entonces *btnAñadir*.

Veamos ahora cómo le asociamos que al pulsar la tecla escape (*Esc*) termine el formulario. Para esto cambiamos la propiedad **CancelButton**.

Por tanto, selecciona de la lista desplegable el botón *btnCerrar* y así al pulsar *Esc* se cerrará la aplicación.

Probemos ahora todo el código y así lo comprobarás: Escribe algo en el cuadro de texto y pulsa Intro, verás que se añade a la lista, después pulsa ESC y verás que se cierra el formulario o ventana con lo que se da por terminada la aplicación.

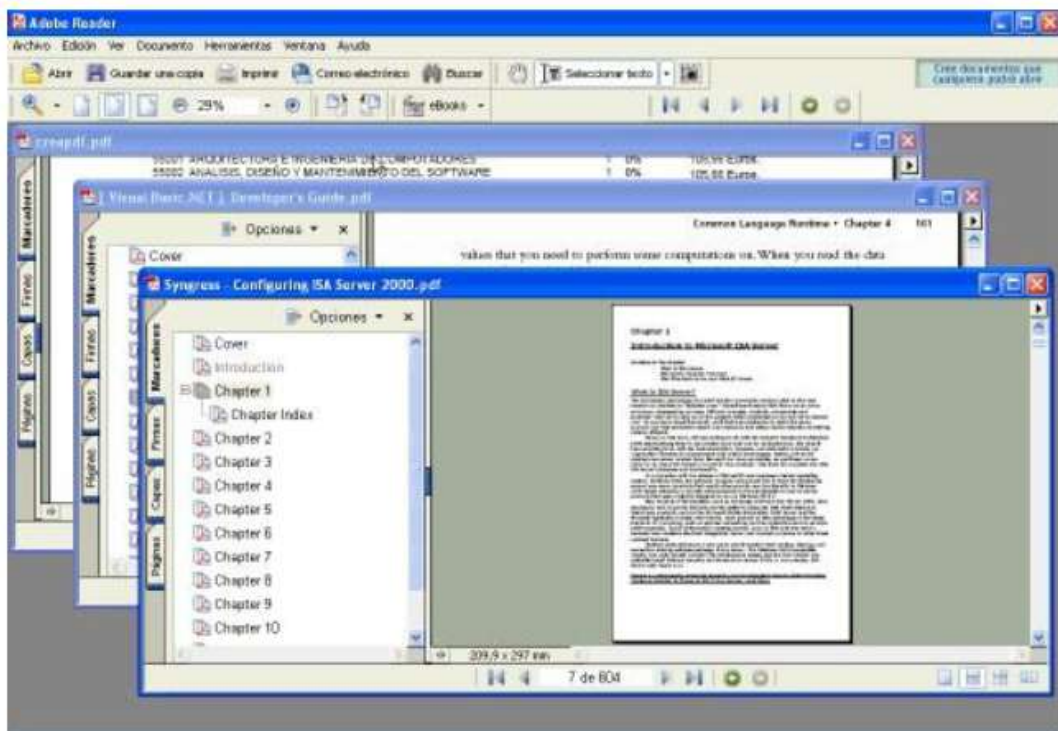
3.3 Interfaces de múltiples documentos: MDI



Debes conocer

Las aplicaciones **MDI** (*interfaz de múltiples documentos*) permiten mostrar varios documentos al mismo tiempo, cada uno de ellos en su propia ventana. Las aplicaciones MDI suelen tener un elemento de menú Ventana con submenús que permiten cambiar entre ventanas o documentos.

Por ejemplo:



Es una imagen del Acrobat Reader que tiene abiertos tres documentos simultáneos: interfaz de múltiples documentos MDI. Lo que comentábamos arriba es que suele ser habitual incluir un menú "Ventana" para permitir al usuario desplazarse por estas ventanas y ponerlas en varias disposiciones: cascada, mosaico, horizontal...

En este tipo de programas con varios formularios distinguiremos entre un formulario principal que es el que hace de contenedor y unos hijos que son cada una de las ventanas o formularios contenidos en él.

3.3.1. Crear un formulario MDI padre

Como hemos comentado el formulario MDI padre es el alma de una aplicación MDI. Es el contenedor de múltiples documentos (formularios hijos) dentro de una aplicación MDI. Puedes utilizar la propiedad "IsMdiContainer" para crear un formulario padre.

Sigue estos pasos para verlo:

1. Crea un nuevo formulario y abre la ventana de código
2. En las propiedades del formulario le indicamos que va a ser un MDI padre con:

```
this.IsMdiContainer = true; // es decir va a ser un contenedor MDI
```

Cuando se utilizan este tipo de formularios es conveniente iniciar la aplicación a pantalla completa o maximizada. Por ejemplo, estableciendo la propiedad "WindowState" a "Maximized".

3.3.2. Crear formularios MDI hijos

Los formularios MDI hijos son los que se muestran dentro del formulario padre en las aplicaciones MDI. Vamos a realizar un ejemplo paso a paso para ver cómo podemos crear formularios hijos dentro de un padre.

El código del evento es el siguiente:

```
private void button1_Click(object sender, EventArgs e)
{

    Form frmHijo = new Form();
    frmHijo.MdiParent = this; frmHijo.Show();

}
```

a. Determinar un MDI hijo activo

Debemos saber por supuesto qué formulario hijo está activo en cada momento. En una aplicación de este tipo el formulario activo es el que tiene el enfoque. Por ejemplo, si el usuario selecciona la opción Cerrar debemos cerrar el formulario activo. Para esto disponemos de la propiedad "**ActiveMDIChild**" del formulario padre que nos indica el formulario activo en ese momento.

Este ejemplo cierra el formulario MDI hijo activo:

```
this.ActiveMdiChild.Close();
```

b. Organizar formularios MDI hijos

Los MDI padres suelen tener una opción de menú que organiza los formularios hijos: en cascada, horizontal, vertical... Para ofrecer esta funcionalidad podemos utilizar el método "**LayoutMDI**" en el formulario padre en la enumeración "**MDILayout**". Esta lista de opciones (MDILayout) tiene estos valores:

Valor	Descripción
Arrangelcons	Muestra unos iconos que representa cada formulario hijo en la parte inferior de la ventana padre
Cascade	Muestra los iconos en cascada
TileHorizontal	Muestra los formularios acoplados horizontalmente.
TileVertical	Muestra los formularios acoplados verticalmente.

Por ejemplo, supón que ya tienes el menú creado y quieres poner el código para que se ajusten los formularios horizontalmente, el código sería:

```
this.LayoutMdi(MdiLayout.TileHorizontal);
```

3.4. Menús

Como es lógico nuestras aplicaciones también tendrán sus menús para que el usuario seleccione las opciones disponibles. Vamos a ver los menús desde dos sitios diferentes: uno desde el diseñador que incorpora el IDE de VS.NET y otro desde código.

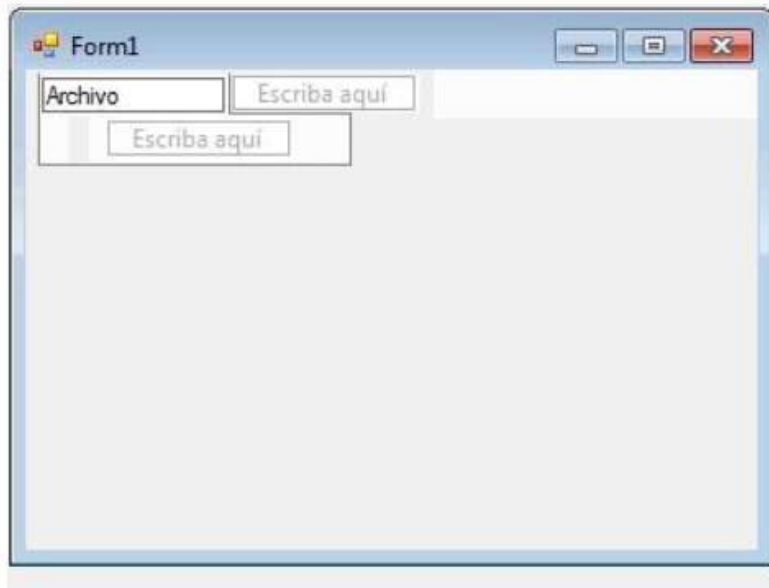
Por un lado, crearemos un menú en tiempo de diseño y por otro con la clase correspondiente controlaremos y manejaremos completamente el menú existente y otros nuevos como pueden ser los contextuales.

Los tipos de controles de menú son los siguientes:

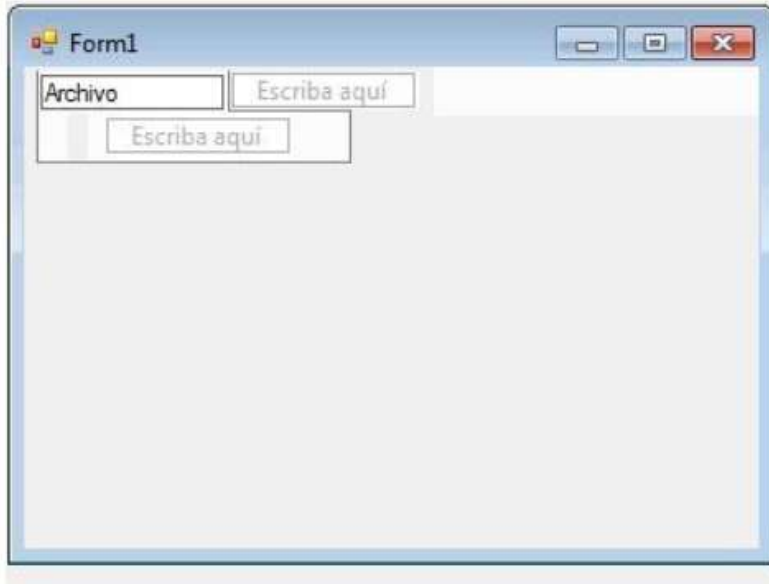
Control	Descripción
ContextMenuStrip	menús contextuales, que se activan con el botón derecho del ratón
MenuStrip	menú principal del formulario
StatusStrip	barra de estado
ToolStrip	barra de herramientas con botones
ToolStripContainer	contenedor de los diferentes botones que pueden componer las barras de herramientas.

3.4.1. Creación de menús con el IDE

Activamos un formulario donde queramos insertar el menú. Seleccionamos de la lista de controles disponible el llamado "MenuStrip" y lo arrastramos al formulario. Puesto que es un control oculto no tiene interfaz de usuario y aparecerá en la parte inferior:



Verás en la parte superior del formulario cómo está preparado para introducir opciones de menú. Vamos a hacer clic en el mensaje que indica "Escriba aquí" y escribimos "Archivo":

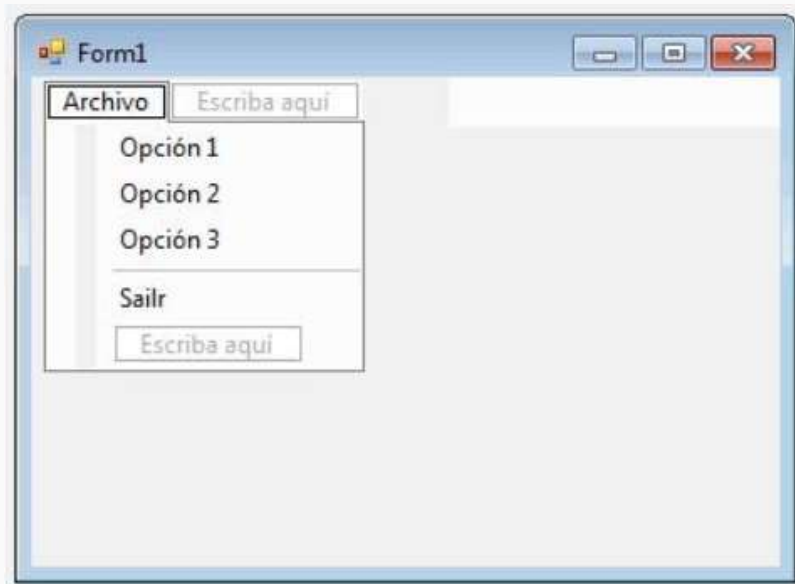


Se activan automáticamente las opciones de la derecha para poner otro menú y en la parte inferior para poner ya el primer elemento del desplegable de "Archivo". Ahora vamos a escribir en este menú una nueva opción "Opcion1":





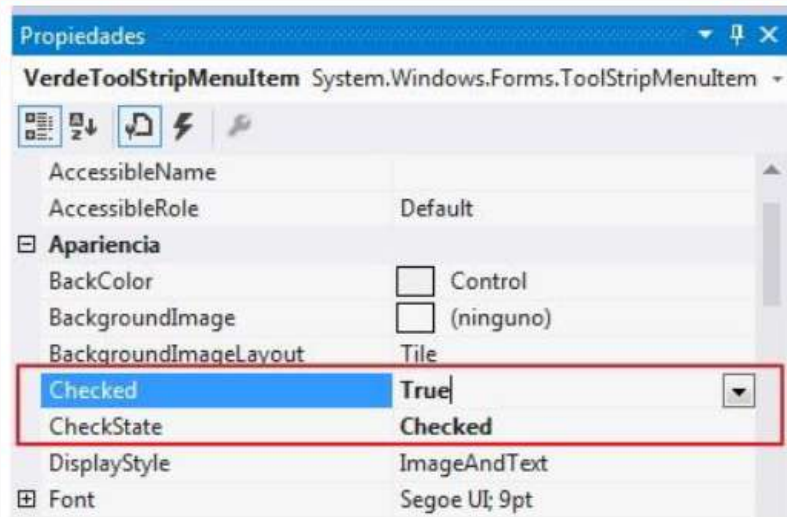
Verás además que se activa la posibilidad de desplegar un menú a la derecha de éste pero seguiremos escribiendo debajo dos opciones más: "Opcion2" y "Opcion3". Ahora quiero poner un separador entre las opciones de menú porque la última va a ser la opción "Salir" y quiero separarla de las demás. Para esto simplemente escribimos en el nombre un guión simple "-" y seguimos a la siguiente opción. Verás cómo ha dibujado una línea de separación. Finalmente introducimos la opción "Salir":



Sigamos con más opciones. Introduce un menú a la derecha que se llame Texto y que tenga tres opciones debajo que sean "Verde", "Azul" y "Rojo":



Pulsa en la opción de menú verde y fíjate en las propiedades de este elemento:



Estas son las propiedades disponibles... lo que yo quiero hacer con este menú es poner una marca a la izquierda como si estuviera activado. Y de hecho eso es lo que indica cuál de las tres opciones o colores está activo. Esto normalmente lo cambiaremos en el código pero debemos dejar una opción ya activada por defecto para que la vea el usuario. Para esto utilizaremos la propiedad "Checked" y la activaremos poniendo el valor "True". Verás como aparece la marca de activado a la izquierda de este elemento...

Ahora veremos las propiedades más importantes para seguir añadiendo opciones. Otra cosa es manejarlos en tiempo de ejecución que lo veremos más adelante.

Propiedad	Descripción
Nombre	Como todos los elementos u objetos de nuestra interfaz debe tener un nombre único. Es muy recomendable ponerle unos nombres legibles para facilitar luego el código
Text	Texto que muestra al usuario
Enabled	Permite habilitar/deshabilitar la opción de menú. Si se ha deshabilitado aparece en color gris como no activa.
Checked	Coloca una marca en la opción de menú.
ShortcutKeys	Son los conocidos atajos de teclado. Son combinaciones de teclas que activan esa opción sin tener que utilizar el menú.
ShowShortcutKeys	Indica si debe mostrar los atajos de teclado.
Visible	Muestra u oculta una opción de menú.
MdiWindowListItem	Es una opción especial cuando se utilizan formularios MDI, es decir, el formulario principal hace de contenedor de varios formularios. Especifica el elemento cuyo DropDown mostrará la lista de ventanas MDI.
DisplayStyle	Para activar que se puedan poner gráficos en las opciones del menú.

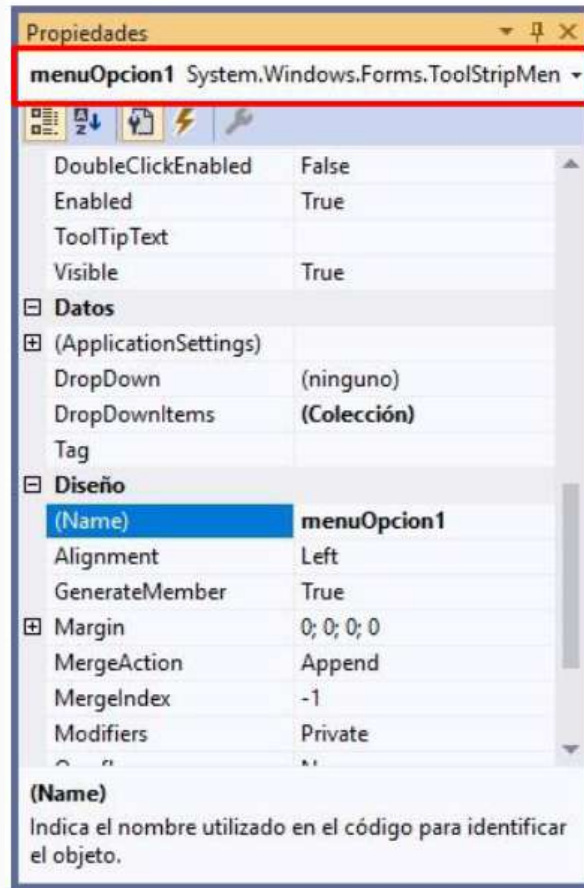
Para dejar una letra del menú subrayada para que se active junto con la tecla "Alt" le pondremos delante el símbolo "&" en la propiedad Text.

Por ejemplo "&Abrir" o "&Cerrar".

Si queremos insertar imágenes utilizaremos la propiedad "Image", como hemos visto en un ejemplo anterior.

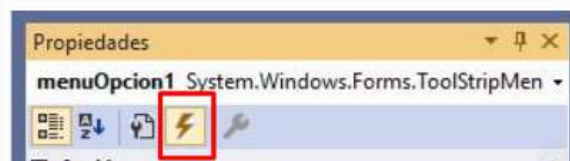
3.4.2. Ejecutar el código de las opciones de menú.

Cuando ya tenemos nuestro menú con todas las opciones tenemos que decirle el código que debe ejecutar. Para esto debemos indicar en nuestro código los eventos adecuados del menú. Al añadir los menús, el IDE les ha asignado un nombre genérico "ToolStripMenuItem", muy incómodo para utilizarlo en el código. Vamos a cambiar su nombre para que sea más legible:

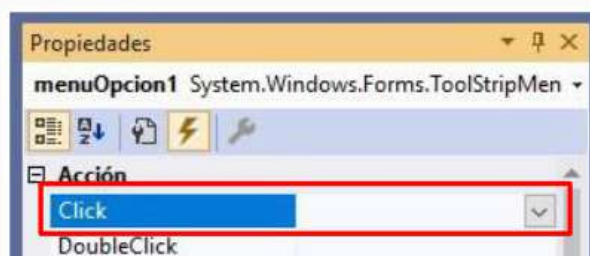


Ahora debemos localizar en nuestro código dónde está este objeto y en qué evento debo poner el código. Ya lo hemos hecho anteriormente.

Primero en la ventana de propiedades seleccionamos el nombre del objeto en el desplegable superior. Luego seleccionamos los eventos:



Y luego seleccionamos el evento adecuado haciendo clic sobre él. En este caso lógicamente el evento "Click".





Con lo que nos escribirá el procedimiento que se ejecutará cuando se haga clic en esta opción.

```
private void menuOpcion1_Click(object sender, EventArgs e)
{
    ...
}
```

En el caso del evento Click, también podíamos haber llegado hasta aquí haciendo doble clic sobre la opción de menú deseada en el diseño del formulario.

3.4.3. Manejo de los menús desde código

En muchas ocasiones tendremos que manejar dinámicamente el menú, es decir, a lo largo del programa y según en la opción en la que estemos puede que algunas opciones de menú no deban estar habilitadas. Vamos a ver cómo manejamos desde el código las propiedades que hemos visto antes.

Habilitar una opción de menú

Ya hemos visto antes que la propiedad para esto debe ser **Enabled**. Luego si queremos habilitar una opción:

```
mnuEjemplo.Enabled = true;
```

Y para deshabilitarla

```
mnuEjemplo.Enabled = false;
```

Para marcar una opción utilizamos la propiedad **Checked**:

```
mnuEjemplo.Checked = true;
```

Mostrar u ocultar una opción de menú:

```
mnuEjemplo.Visible = true;
```

o para ocultar

```
mnuEjemplo.Visible = false;
```

Si esto lo ponemos en una de las opciones del menú principal se queda visible o invisible todo el menú desplegable.

Cambiar el texto de un menú

Muy fácil, utilizaremos la propiedad **Text**:

```
mnuEjemplo.Text = "Este es el nuevo texto";
```

Añadir y eliminar elementos de menú por código

Queremos añadir y eliminar elementos de nuestro menú en tiempo de ejecución. Para esto no utilizaremos las propiedades ya que éstas sirven para modificar su aspecto y comportamiento, lo que tendremos que utilizar son los métodos disponibles en los **"MenuItem"**.

Añadir más elementos de menú:

```
ToolStripMenuItem mnuPrimero = new ToolStripMenuItem();  
mnuPrimero.Text = "Otra opción";  
// Otro elemento  
ToolStripMenuItem mnuSegundo = new ToolStripMenuItem();  
mnuSegundo.Text = "Opción 1"; mnuSegundo.Checked = true;  
// Otro elemento  
ToolStripMenuItem mnuTercero = new ToolStripMenuItem();  
mnuTercero.Text = "Opción 2";  
// Ya están creados.  
// Ahora voy a enlazar los dos últimos para que dependan de mnuPrimero.  
mnuPrimero.DropDownItems.AddRange(new  
ToolStripMenuItem[] { mnuSegundo, mnuTercero });
```

```
// Ahora añadimos mnuprimero a la lista de menús de primer nivel  
Menu.Items.AddRange(new ToolStripMenuItem[] { mnuPrimero});
```

Eliminar elementos de menú:

```
Menu.Items.Remove(mnuSegundo)
```

3.4.4. Menús contextuales

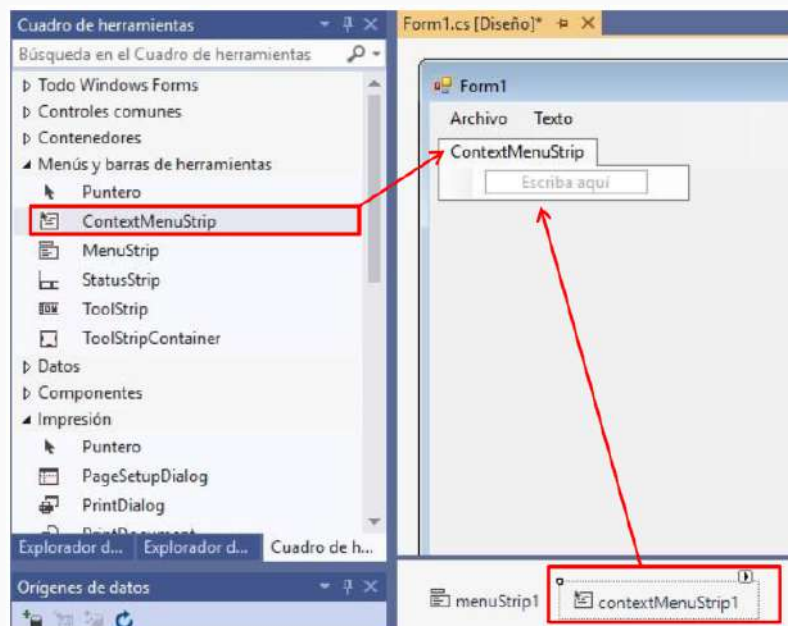


Debes conocer

Como ya hemos comentado son los menús que se activan con el botón derecho del ratón y dependiendo del contexto, pantalla o formulario en donde nos encontremos podemos seleccionar un menú u otro.

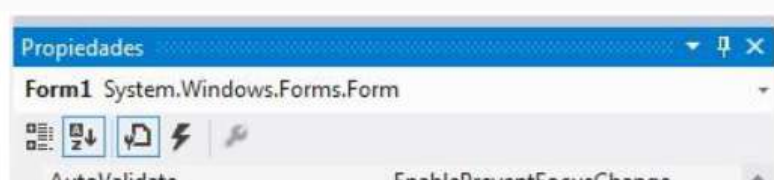
El componente **ContextMenuStrip** proporciona a los usuarios acceso a los comandos de menú de uso frecuente. Con frecuencia, los menús contextuales están asociados a un control para adecuar su menú más específicamente a las necesidades del usuario (por ejemplo, los comandos de menú **Deshacer**, **Cortar**, **Copiar**, **Pegar**, **Eliminar** y **Seleccionar todo**, que suelen formar parte del menú contextual del control **TextBox**).

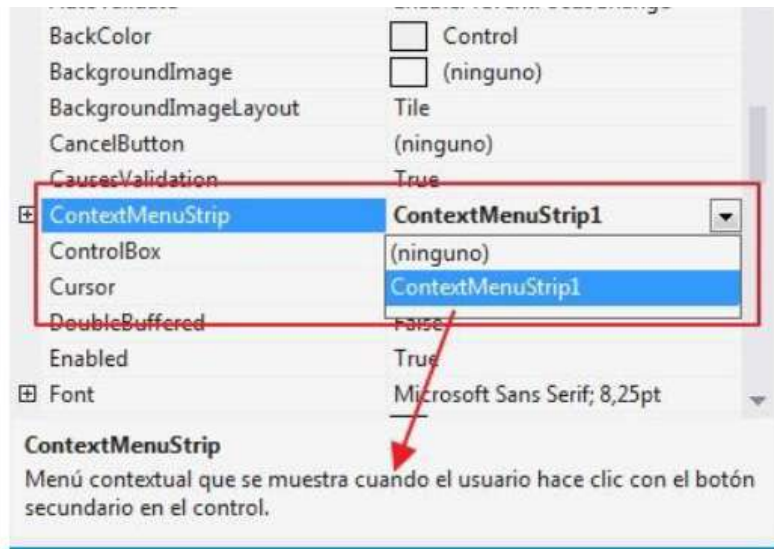
Procederemos de la misma forma que con los otros menús. Añadimos un control de tipo "**ContextMenuStrip**" a nuestro formulario y escribimos las opciones que nos interesen:



Vemos en la parte inferior cómo hemos añadido este control al formulario. Una vez creado ahora tenemos que asociarlo con un control. Podemos asociarlo a un cuadro de texto, un gráfico, un formulario,... lo que queramos.

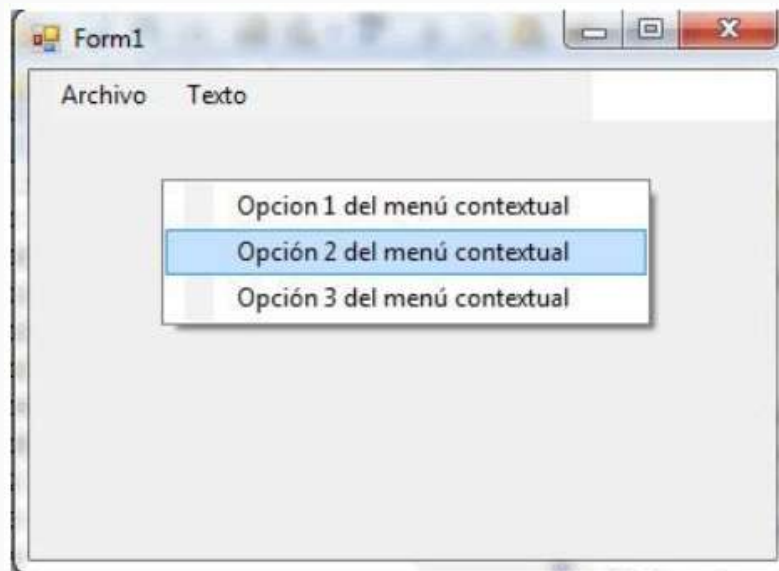
Supongamos que lo queremos asociar al formulario, es decir, cuando se haga clic con el botón derecho del ratón en el formulario queremos que se active nuestro menú. Para esto vamos a utilizar la propiedad **ContextMenuStrip** que está disponible en muchos controles. Búscala en las propiedades del formulario:





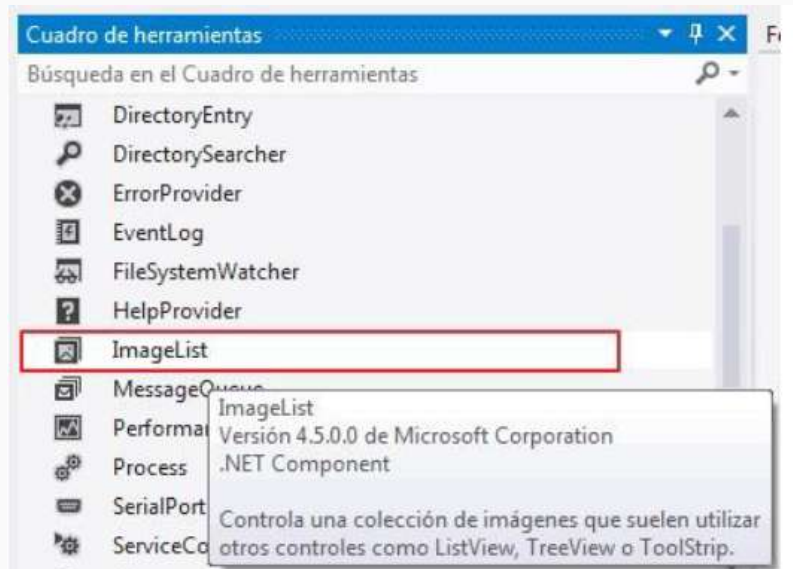
Al desplegar la lista verás que aparece el menú contextual que hemos creado antes.

Perfecto, sólo falta probarlo, ejecutamos el programa y:

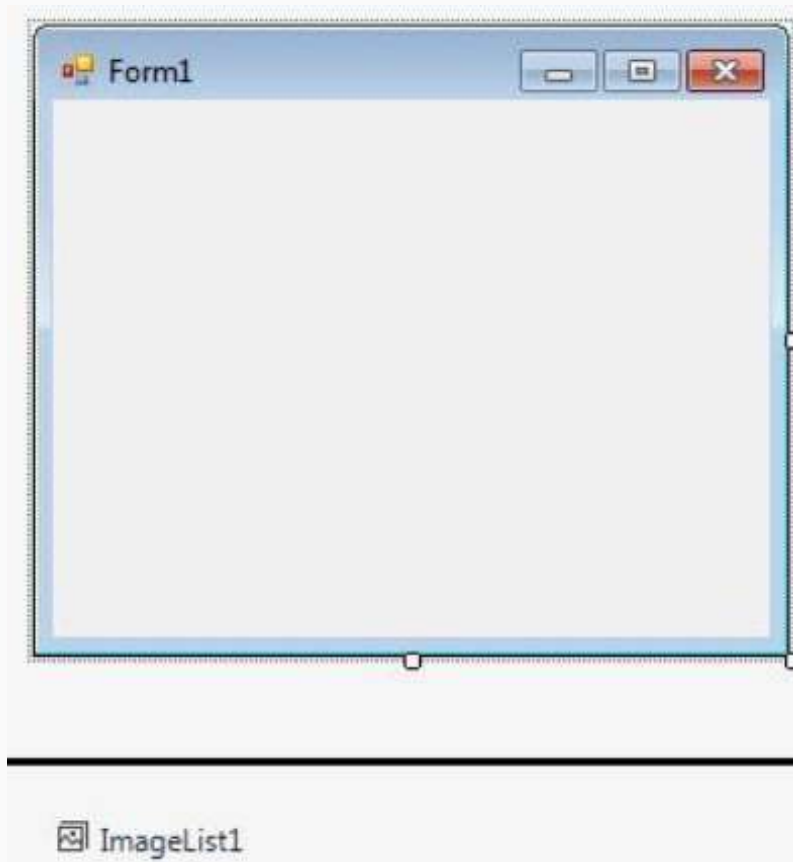


3.5. Control de lista de imágenes. ImageList

Este control es muy sencillo ya que simplemente es un contenedor de imágenes, es decir, almacena una colección de imágenes. Para seleccionarlo lo encontraremos en el cuadro de herramientas:



Al añadirlo a nuestro proyecto aparecerá en la parte inferior como control no visible. Es decir, no muestra nada al usuario pero es utilizado por otros controles para utilizar las imágenes que tiene dentro almacenadas:

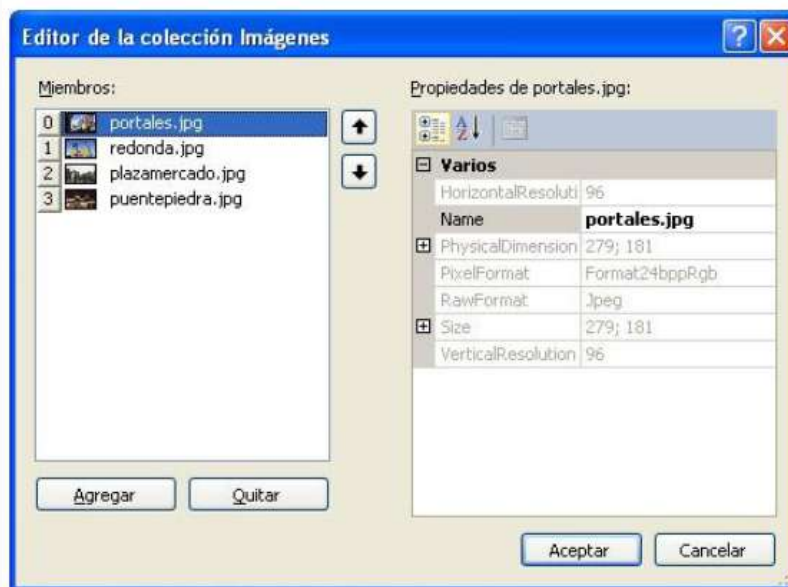


Tiene varias propiedades aunque la más importante es "**Images**" que es la colección de las imágenes.

Las otras propiedades son:

Propiedad	Descripción
ColorDepth	Número de colores utilizado para procesar las imágenes
Images	Colección de imágenes
ImageSize	Tamaño de las imágenes individuales en la lista de imágenes
TransparentColor	Color tratado como transparente

La primera propiedad indica la profundidad de color o números de colores, puede ser de 8 bits para iconos o logotipos o de hasta 32 bits para fotografías. Pero veamos la propiedad "Images", al pulsarla tenemos como en otras colecciones la lista de miembros y sus propiedades:



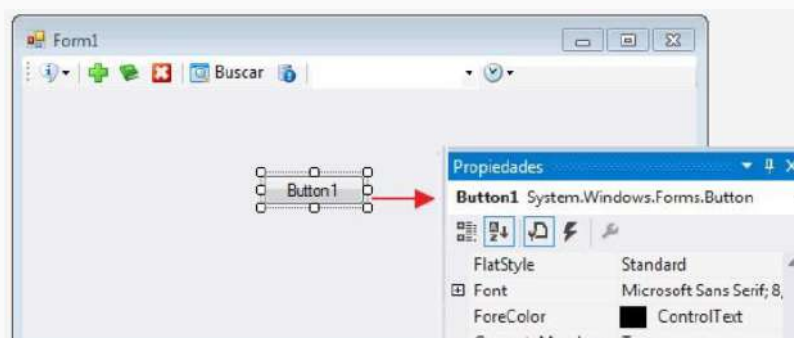
En la lista de la izquierda iremos añadiendo las imágenes y a la derecha podremos ver las propiedades de cada una de ellas.

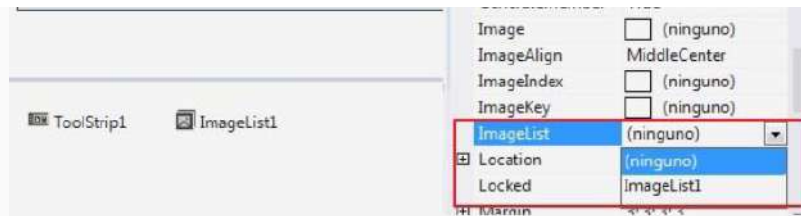
También podemos añadir más elementos mediante código:

```
this.ImageList1.Images.Add(new Bitmap("ejemplo.gif"));
```

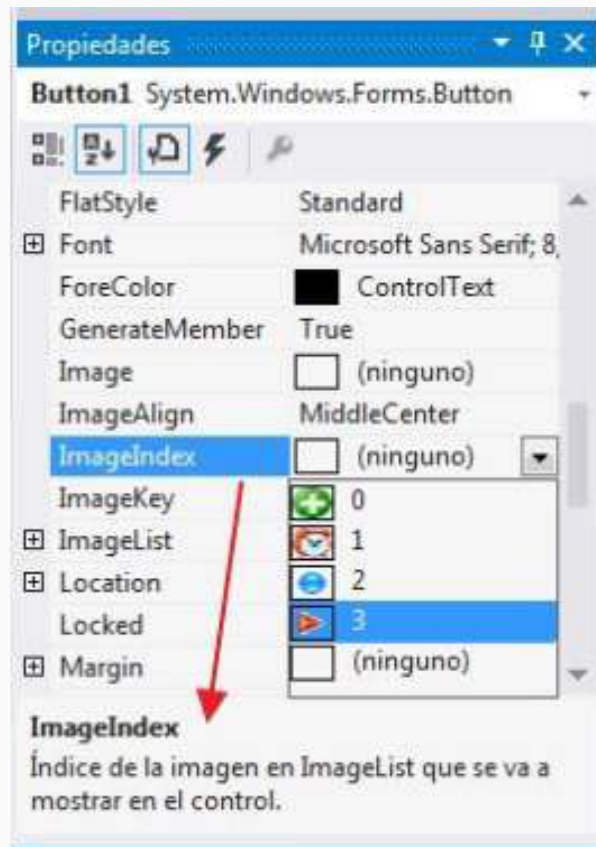
Pero lo utilizaremos pocas veces ya que es en tiempo de diseño cuando preparamos las imágenes para los iconos de las barras u otros elementos de nuestra interfaz.

Veamos cómo se asocia a un control. Colocamos un botón en el formulario e indicamos la propiedad que apunta a la lista de imágenes que acabamos de crear:

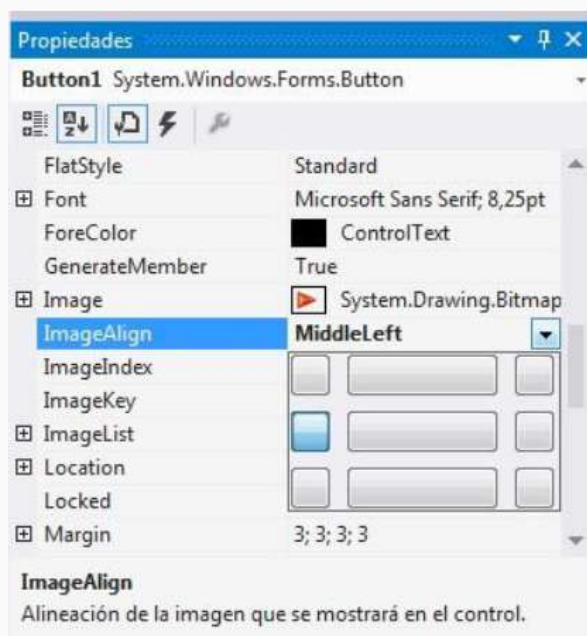




Le estamos indicando que se asocie con un control de este tipo que ya existe y que actúa como contenedor de imágenes. Así que ahora debemos elegir la imagen de este contenedor mediante la propiedad **"ImageIndex"**. Al pulsar en el desplegable nos muestra los iconos directamente:



Y luego le indicamos la posición con la propiedad **"ImageAlign"**:



Para darnos como resultado el botón así:

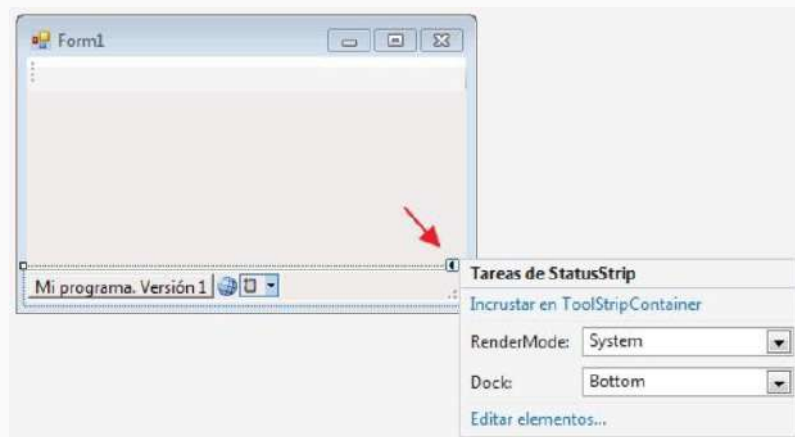


Y recuerda que, como ya hemos comentado antes, también puede alimentarse de imágenes mediante la propiedad "**Image**" que accede a la sección de recursos del proyecto.

3.6. Barras de estado y herramientas

Una barra de herramientas proporciona botones en la parte superior de la ventana para acceder más rápidamente a determinadas opciones del programa. Por otro lado, la barra de estado se sitúa en la parte inferior de la ventana y muestra distintos tipos de opciones.

Las posiciones de estos dos controles la podemos cambiar para colocarlos donde queramos. Habitualmente las barras de botones se colocan en la parte superior y la barra de estado en la inferior. Vamos a añadir a un proyecto nuevo estos dos controles:

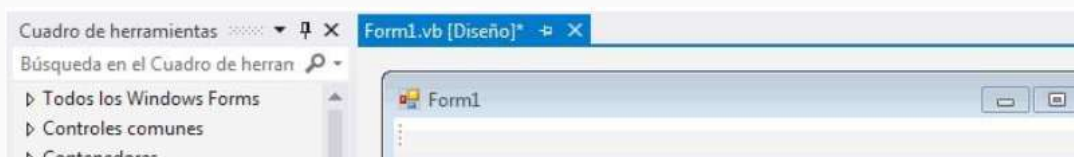


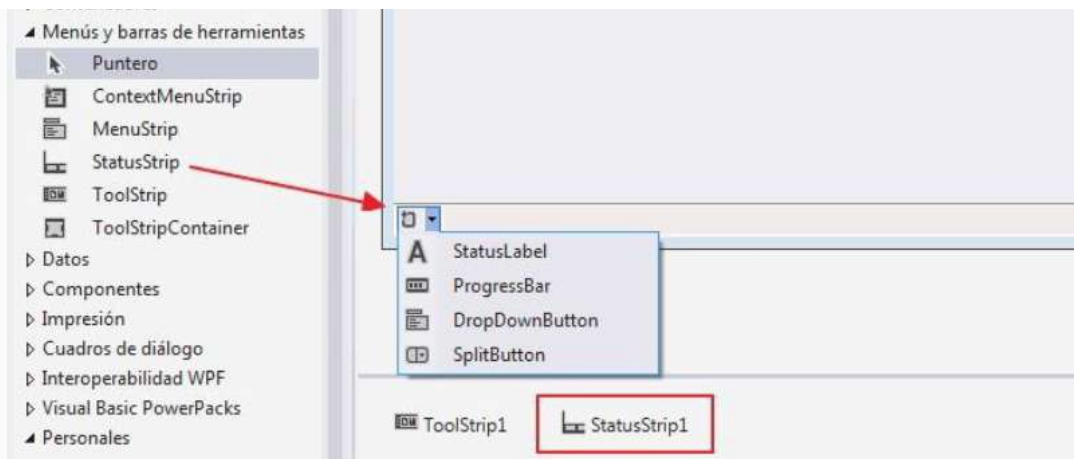
Vamos a añadir los distintos tipos de controles que podemos utilizar:



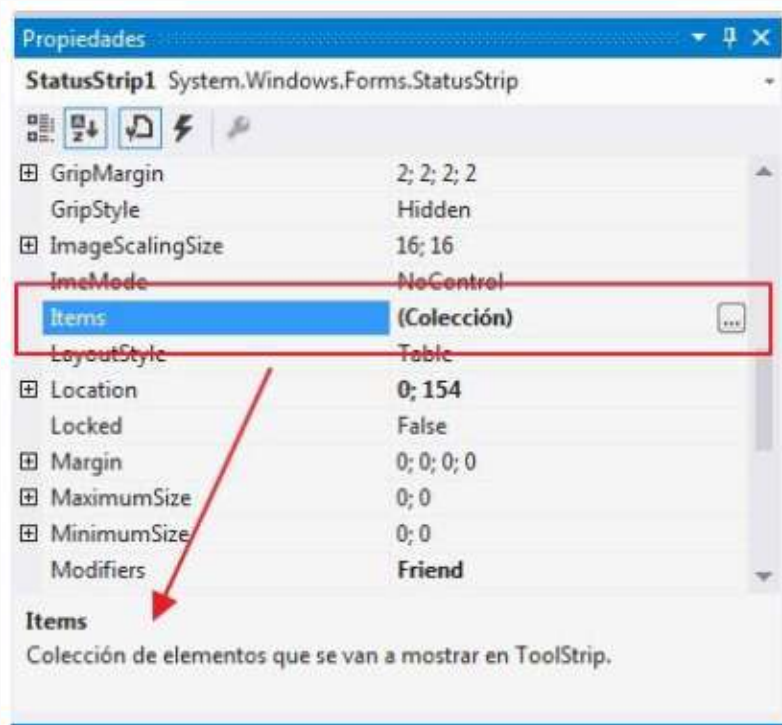
Las barras de botones han ido evolucionando mucho, ya no solo hay botones sino que además hay cuadros desplegables para los tipos de letra, por ejemplo. Hay cuadros de texto y hasta barras de progreso.

Ahora hacemos lo mismo con la barra de estado. Añadimos una, veremos que el funcionamiento es muy similar pero con algún control menos:

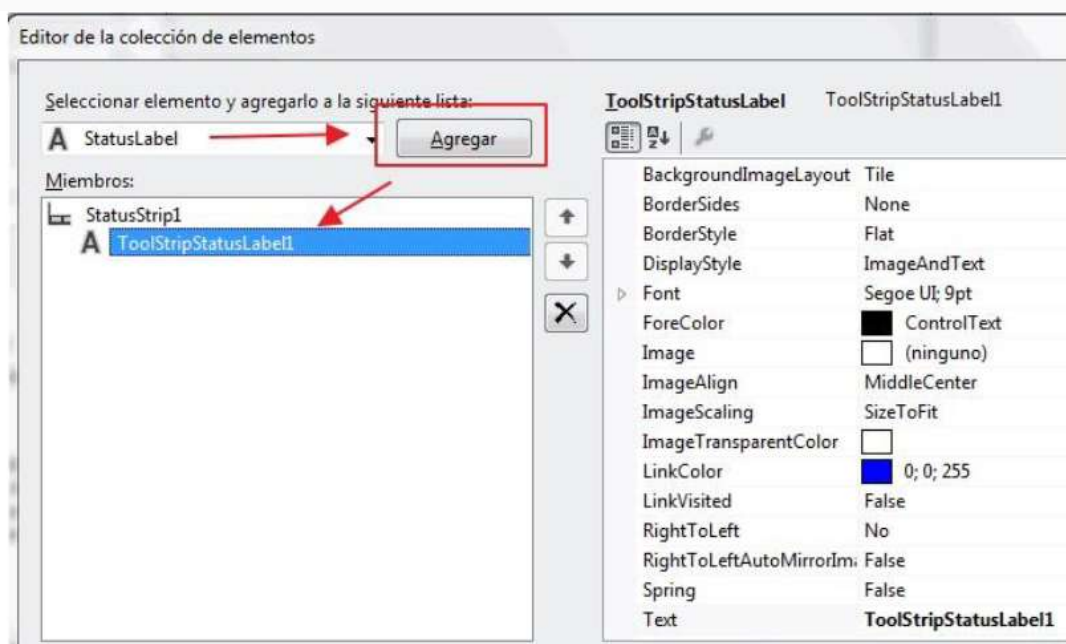


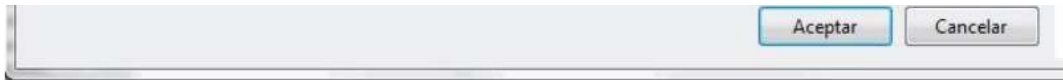


Para añadir elementos, podemos escribir directamente, como cuando hicimos los menús anteriores, o acceder a la propiedad "items":

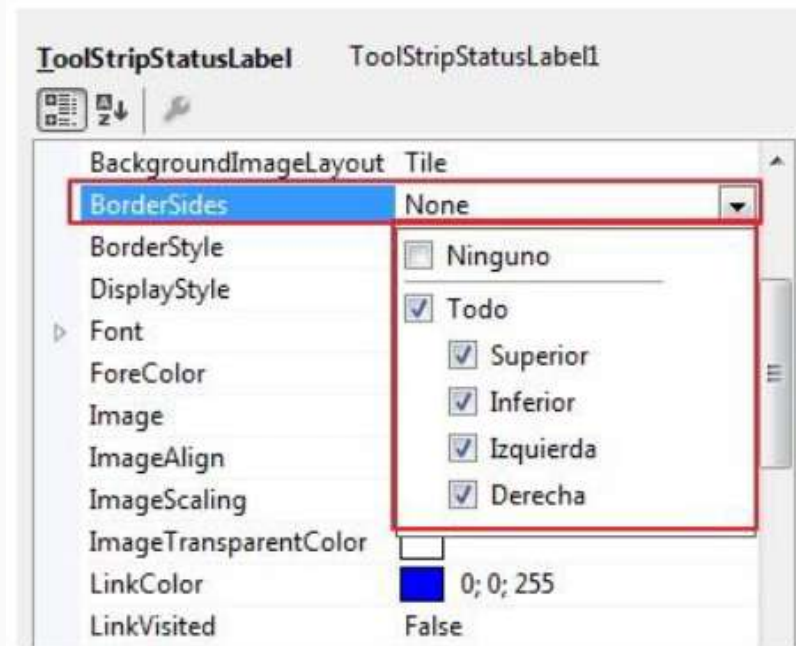


Que nos permitirá editar los elementos de una forma más ordenada:

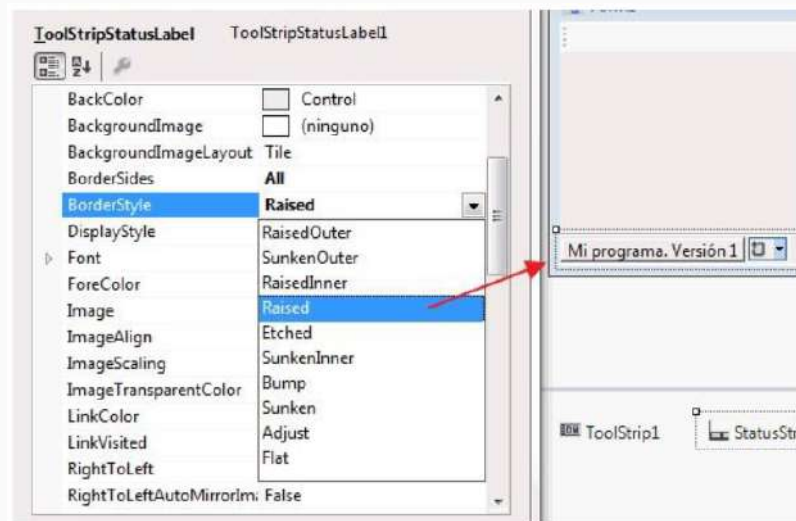




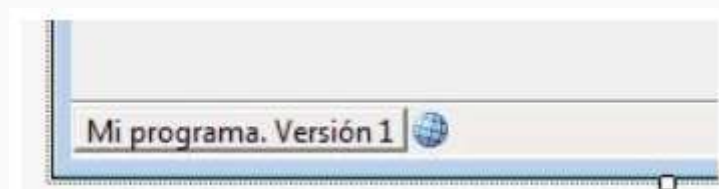
Vamos a añadir varios paneles de ejemplo de tipo texto y les ponemos algún borde:



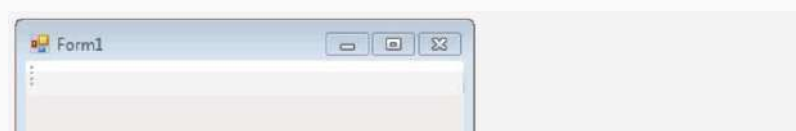
También en el estilo del contorno "borderStyle" podemos ponerle más estilos:

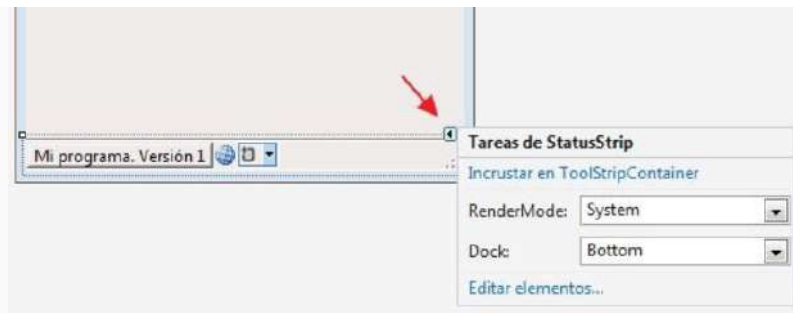


E incluso añadir imágenes:

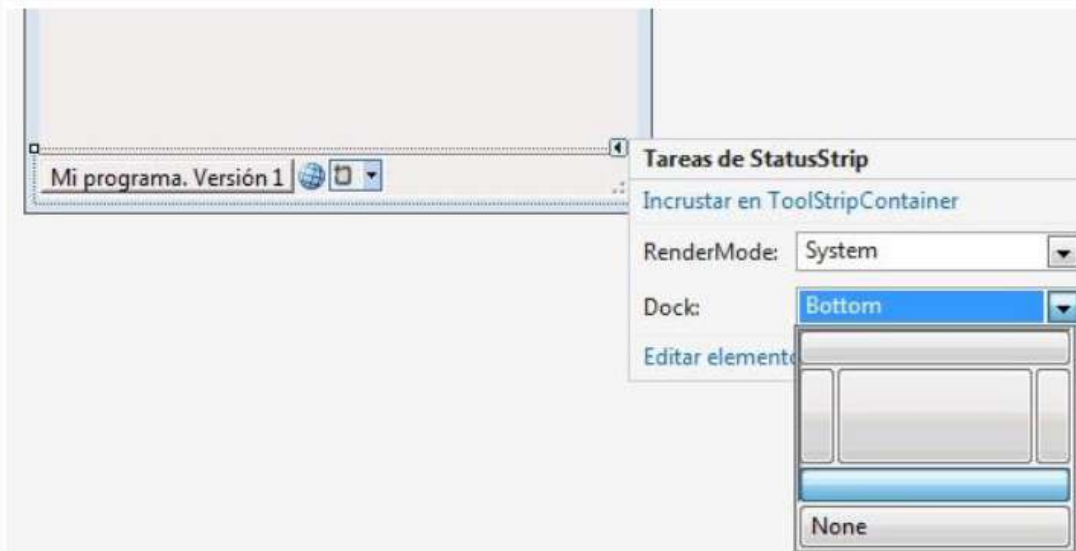


Poco a poco vamos componiendo la barra de estado. Ahora nos fijamos en una pequeña flecha que aparece en la parte derecha y que nos muestra algunas propiedades:





Se trata de opciones generales del control, donde podremos, por ejemplo, anclar el control en la parte de la pantalla que queramos:

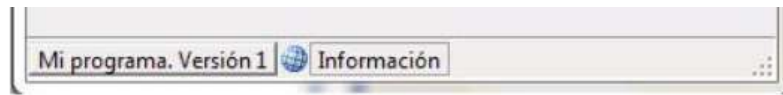


La primera opción nos coloca este control en otro del tipo "ToolStripContainer" que no vamos a ver de momento. La opción de "RenderMode" permite:

- **ToolStripSystemRenderer** proporciona el aspecto y estilo de su sistema operativo.
- **ToolStripProfessionalRenderer** proporciona el aspecto y estilo de Microsoft Office.
- **ToolStripRenderer** es la clase base abstracta para las otras dos clases de representación.

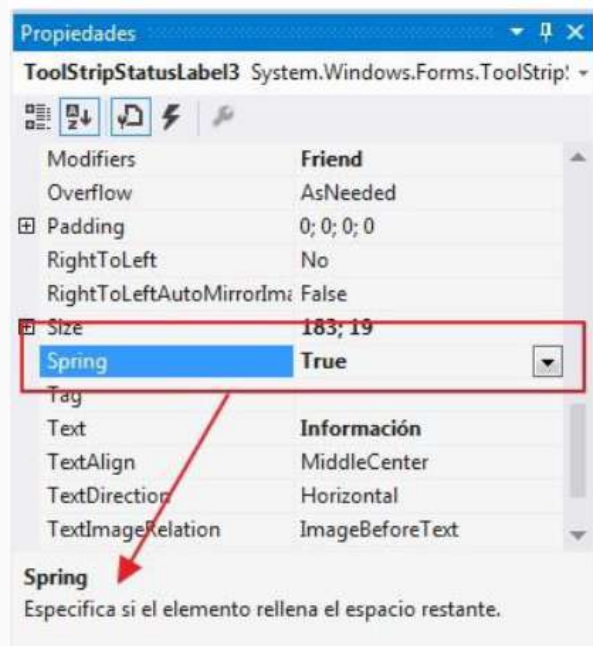
Ahora nos fijaremos en el comportamiento del ajuste de los controles. Si ejecutamos este ejemplo, que tiene dos paneles de texto, y lo ampliamos vemos que su tamaño es fijo:





Pero queremos que ese segundo control se ajuste al total del ancho de la pantalla.

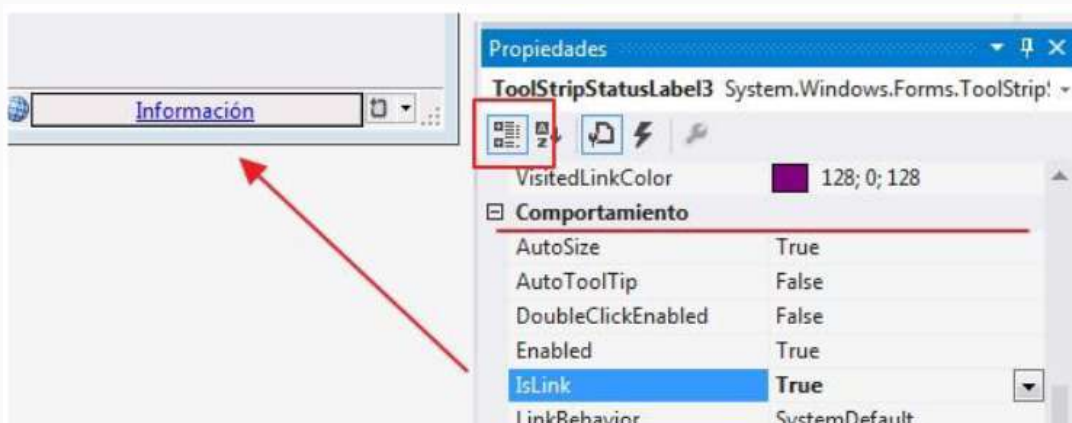
Para esto vamos a cambiar la propiedad "**Spring**":

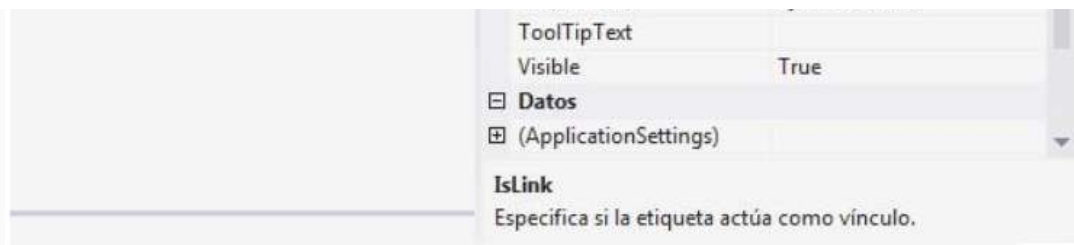


Ejecutamos ahora el programa. Podremos comprobar que ese segundo control ahora tiene su tamaño dinámico:



Si agrupamos las propiedades por categorías, podemos ver las que afectan al comportamiento. Por ejemplo, incluso si queremos que el texto se comporte como un hipervínculo:

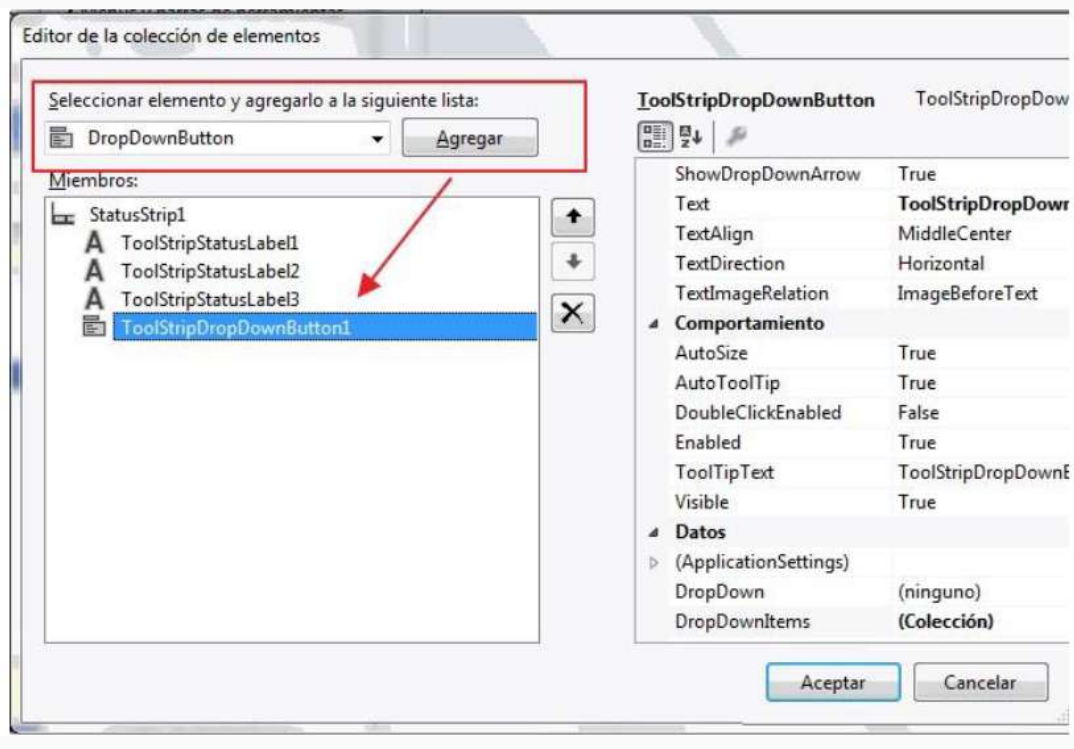




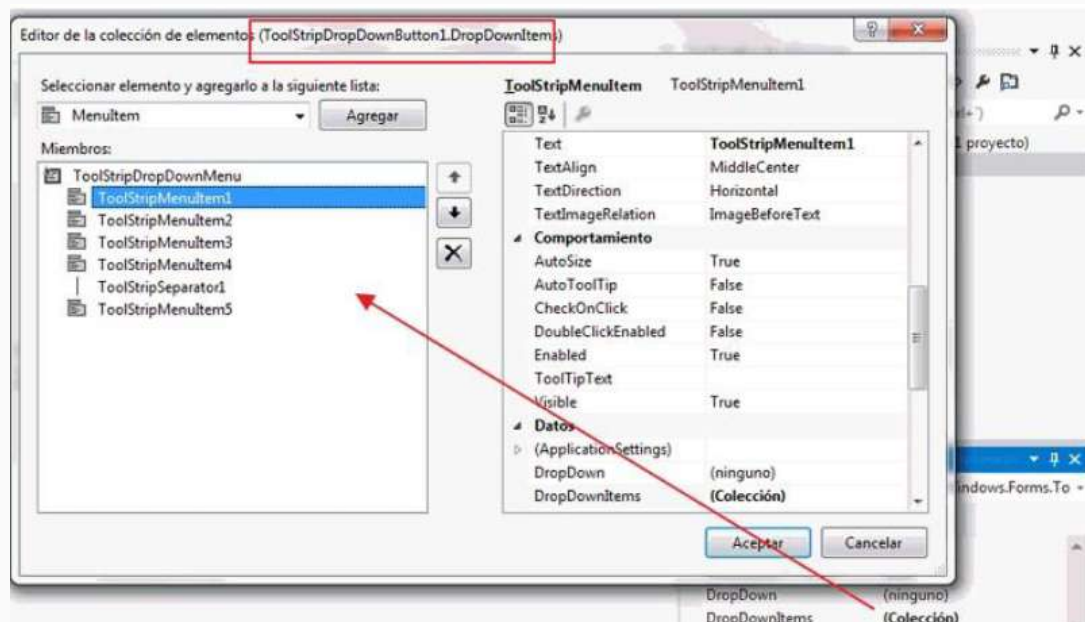
Explorando las propiedades, podemos ver incluso que puede tener un menú contextual. Las posibilidades son ilimitadas.

3.6.1. Control "DropDownButton"

Veamos ahora el control de tipo "DropDownButton".

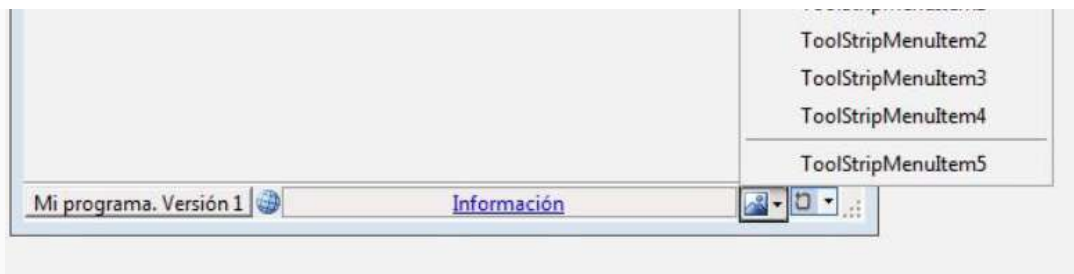


Este objeto se puede personalizar como el anterior. Si lo marcamos podemos ir a la ventana de propiedades para ver los "DropDownItems":

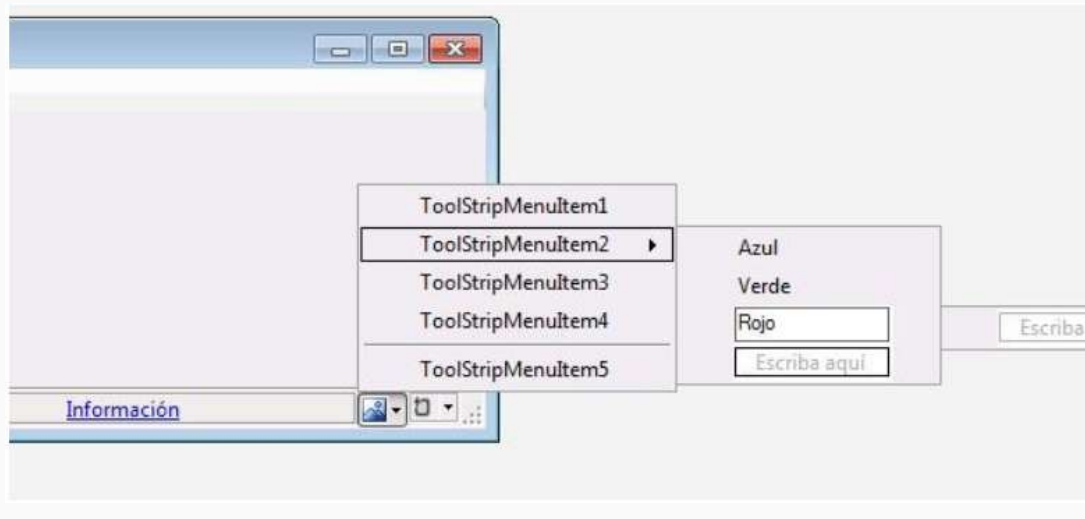


Como vemos en la pantalla, podemos añadir esta otra colección de objetos a este desplegable de la barra de estado. Tener la posibilidad de añadir elementos como si fuera un menú que al ejecutar la página:



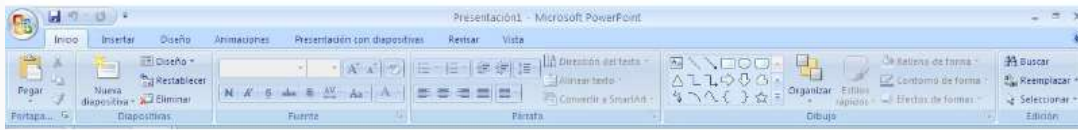


Podemos insertar controles similares a los del menú:



3.6.2. Barras de herramientas ToolStrip

El otro elemento habitual en los formularios principales de las aplicaciones es la barra de herramientas. Por ejemplo, en PowerPoint:

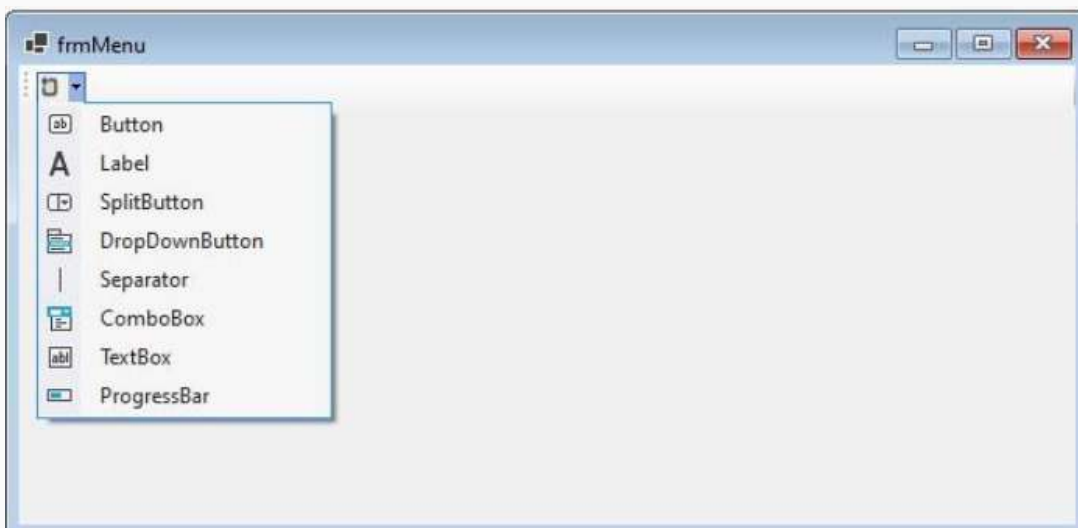


Las barras de herramientas se componen de botones e iconos que luego despliegan opciones, cuadros de texto, cuadros desplegables, botones que se excluyen entre sí (justificación de textos), separadores, etc. Y todo además en unos contenedores que podemos mover y cambiar de sitio.

Los elementos que tenemos para utilizar en esta barra de botones son:

Elemento	Descripción
ToolStripButton	Crea un botón de barra de herramientas que admite texto e imágenes
ToolStripLabel	Representa una clase ToolStripItem no seleccionable que representa texto e imágenes y opcionalmente con hipervínculos
ToolStripSplitButton	Representa una combinación de un botón estándar situado a la izquierda y un botón de lista desplegable situado a la derecha
ToolStripDropDownButton	Es como el ToolStripButton, pero muestra un área desplegable cuando el usuario hace clic en él
ToolStripSeparator	Representa una línea utilizada para agrupar elementos
ToolStripComboBox	Representa un cuadro combinado
ToolStripTextBox	Representa un cuadro de texto
ToolStripProgressBar	Representa una barra de progreso

Coinciden con las opciones gráficas que ya hemos visto anteriormente:



Ejercicio Resuelto

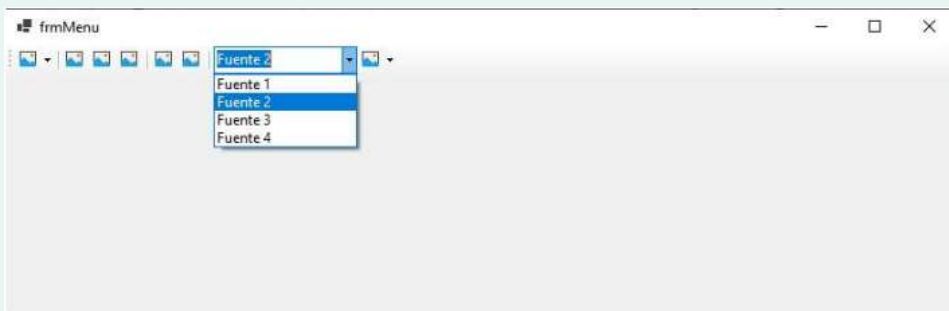
Como ejemplo vamos a intentar reproducir una barra completa de un

programa, empezaremos con la opción de un icono que despliega un menú:

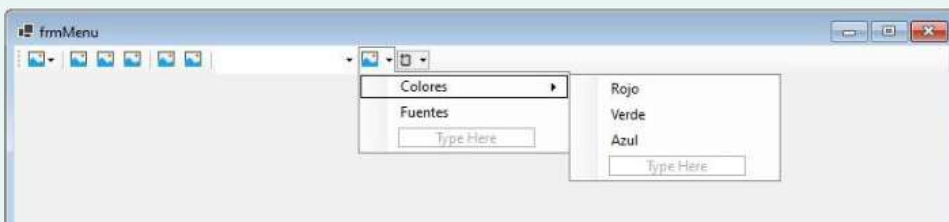
Mostrar retroalimentación



Lo que despliega es un objeto del mismo tipo que los menús, así que las opciones son las mismas. A continuación hay unos botones con iconos y con unos separadores para dejarlos agrupados. Después tenemos un cuadro desplegable:



Donde podríamos seleccionar tipos de letra, por ejemplo. Luego tenemos un control del tipo "**SplitButton**" que permite menús más complejos como:



Si ahora encojemos la ventana para provocar que no quepan los controles:

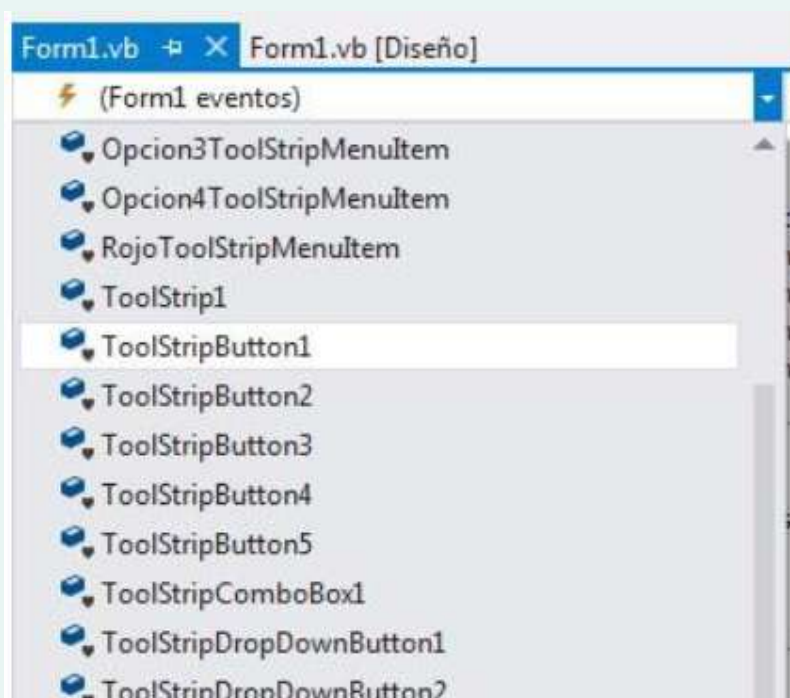


Vemos que entra en acción la propiedad que, de forma predeterminada está activa para controlar el overflow, o desbordamiento. En la parte derecha ha puesto una flecha para poder continuar dibujando el menú debajo.

Como vemos las posibilidades son ilimitadas y además del diseño podemos tener en cuenta las propiedades más importantes de **ToolStrip** que son:

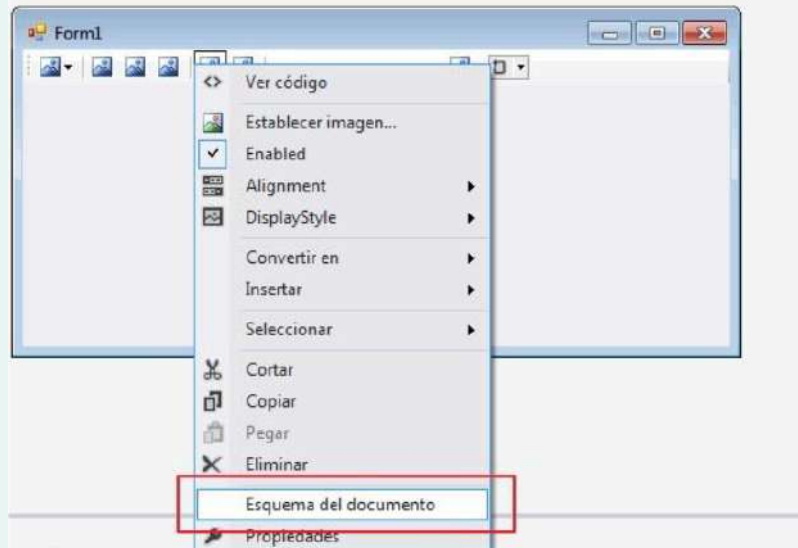
Nombre	Descripción
AllowItemReorder	Obtiene o establece un valor que indica si la clase controla de forma privada las operaciones de arrastrar y colocar, y la reordenación de elementos
AllowMerge	Obtiene o establece un valor que indica si se pueden combinar varios ToolStrip, ToolStripDropDownMenu, ToolStripMenuItem y otros tipos
BackColor	Obtiene o establece el color de fondo del ToolStrip
BackgroundImage	Obtiene o establece la imagen de fondo que se muestra en el control (Se hereda de Control)
BackgroundImageLayout	Obtiene o establece el diseño de la imagen de fondo tal como se define en la enumeración ImageLayout (Se hereda de Control)
CanOverflow	Obtiene o establece un valor que indica si pueden enviar elementos de ToolStrip a un menú de desbordamiento
ContextMenu	Obtiene o establece el menú contextual asociado al control (Se hereda de Control)
ContextMenuStrip	Obtiene o establece el ContextMenuStrip asociado a este control (Se hereda de Control)
DefaultBackColor	Obtiene el color de fondo predeterminado del control (Se hereda de Control)
DefaultDropDownDirection	Obtiene o establece un valor que representa la dirección predeterminada en la que se muestra un control ToolStripDropDown con respecto a ToolStrip
DefaultFont	Obtiene la fuente predeterminada del control (Se hereda de Control)
DefaultForeColor	Obtiene el color de primer plano predeterminado del control
DockPadding	Obtiene la configuración de relleno de acople para todos los bordes del control. (Se hereda de ScrollableControl)
ForeColor	Obtiene o establece el color de primer plano del control ToolStrip
GripDisplayStyle	Obtiene la orientación del controlador de movimiento de ToolStrip
GripMargin	Obtiene o establece el espacio que hay alrededor del controlador de movimiento de ToolStrip
GripRectangle	Obtiene los límites del controlador de movimiento de ToolStrip
GripStyle	Obtiene o establece si el controlador de movimiento de ToolStrip es visible o está oculto
ImageList	Obtiene o establece la lista de imágenes que contiene la imagen mostrada en un elemento de ToolStrip
Items	Obtiene todos los elementos que pertenecen a ToolStrip
Orientation	Obtiene la orientación de ToolStripPanel
OverflowButton	Obtiene el ToolStripItem que es el botón de desbordamiento para un ToolStrip con desbordamiento habilitado
Padding	Obtiene o establece el relleno dentro del control (Se hereda de Control)
ShowItemToolTips	Obtiene o establece un valor que indica si se mostrará información sobre herramientas en los elementos de ToolStrip
Stretch	Obtiene o establece un valor que indica si ToolStrip se estira de extremo a extremo en ToolStripContainer
TextDirection	Obtiene o establece la dirección en la que se dibujará el texto en ToolStrip

Hay un tema importante que debemos tener en cuenta y es poner nombres legibles a toda la cantidad de elementos que podemos añadir. De esta forma será mucho más fácil acceder a ellos. Sino al dejar los nombres genéricos tendremos un buen lío de nombres:

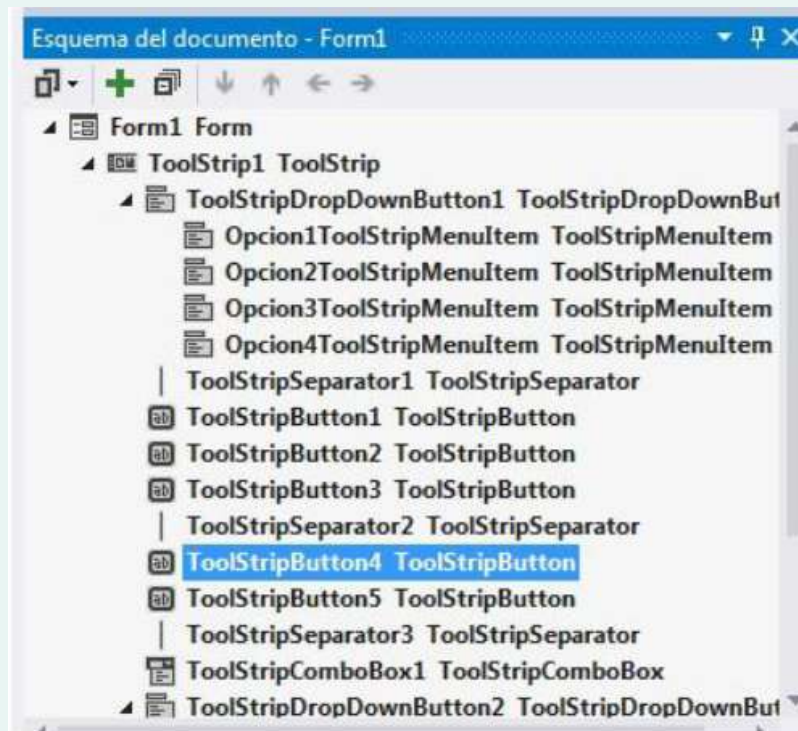




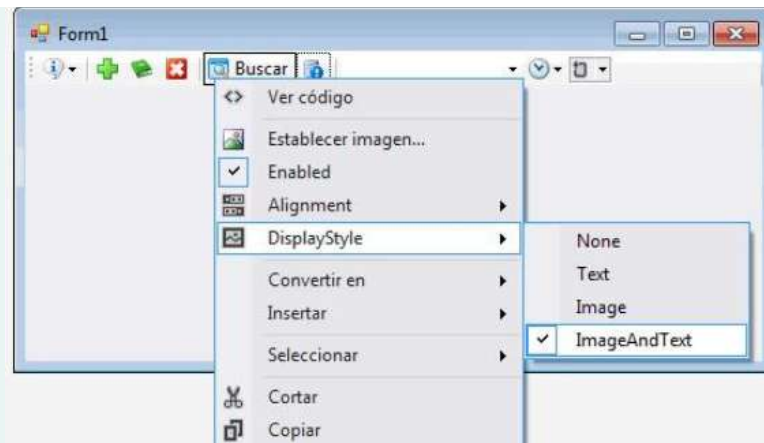
Sería muy difícil de depurar así que es mejor invertir un poco de tiempo en poner los nombres. En esta ocasión puede ser interesante la vista jerárquica del formulario. Así podemos ver la dependencia de unos controles y otros. Y, si tenemos controles anidados, es decir, unos dentro de otros, podemos verlos jerárquicamente:



Nos muestra en uno de los laterales, el esquema jerárquico de los controles del formulario:



En la personalización, podemos indicar que nos muestre el texto junto a los iconos.
Por ejemplo:



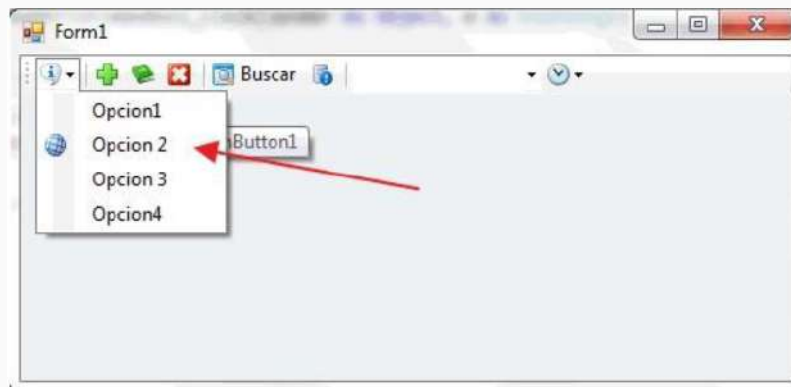
En el botón se ha incluido el texto "Buscar", indicando en sus propiedades que sea del tipo "ImageAndText": imagen y texto.

3.6.3. Detectar las opciones en el código

Obviamente una vez definida nuestra barra de herramientas debemos detectar las opciones del usuario en nuestro código para así poder realizar las acciones necesarias. Para esto debemos introducir el código necesario en los distintos eventos. En el ejemplo anterior es fácil detectar los eventos porque cada elemento es un objeto de los que vemos enumerados en el gráfico anterior. Por tanto basta con seleccionar el elemento y luego el evento. Por ejemplo un clic en la segunda opción del menú:

```
private void opción2ToolStripMenuItem_Click(object sender, EventArgs e)
{
    MessageBox.Show("Has hecho clic en la segunda opción");
}
```

Al hacer clic en la opción:



Se dispara el evento y podremos hacer la operación que queramos:



En el cuadro combinado debemos coger el valor seleccionado en su propiedad "value" utilizando el evento que ya conocemos para detectar que se ha

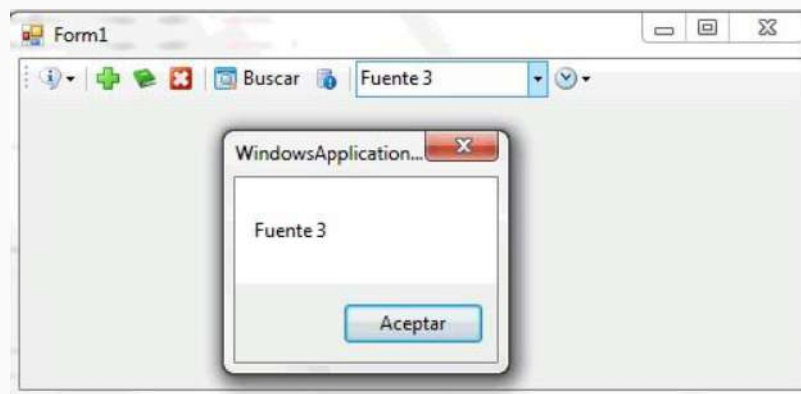
seleccionado un elemento: **SelectedIndexChanged**. Por tanto seleccionamos el objeto del cuadro combinado y luego este evento para escribir el valor actual que corresponde con la selección:

```
private void toolStripComboBox1_SelectedIndexChanged(object sender,
EventArgs e)
{

    MessageBox.Show(toolStripComboBox1.SelectedItem.ToString());

}
```

Se dispara el evento y podemos capturar el valor seleccionado:



Esta técnica común de los menús, barras de botones y de estado ha simplificado enormemente la programación de todos estos elementos y, sobre todo, hacer que sean verdaderamente intuitivos.

